

综合课程设计I —桌面用电监控系统

实验指导书

刘 政



重庆理工大学
CHONGQING UNIVERSITY OF TECHNOLOGY

计算机科学与工程学院
物联网工程专业

2025. 03

目录

第1章 嵌入式系统开发基础	3
1.1 嵌入式系统的组成	3
1.1.1 嵌入式系统的硬件	4
1.1.2 嵌入式系统的软件	5
1.2 嵌入式系统的开发	5
1.2.1 嵌入式系统的开发流程	5
1.2.2 嵌入式系统的开发和运行环境	7
1.2.3 嵌入式系统的硬件设计	8
1.2.4 嵌入式系统的软件设计	9
1.2.5 嵌入式应用程序的下载	10
1.3 思考与练习	11
第2章 常用软件介绍	12
2.1 Altium Designer电路设计	12
2.1.1 原理图设计	12
2.1.2 PCB设计	20
2.1.3 PCB板的制作	27
2.1.4 原理图库的建立	29
2.1.5 创建PCB元器件封装	30
2.2 keil软件安装及使用	31
2.2.1 keil μ vision5基本介绍	31
2.2.2 Keil μ Vision5的安装	32
2.2.3 第一个STM32程序	34
2.3 ST-LINK 下载软件的使用	39
2.4 思考与练习	41
第3章 实验硬件平台介绍	42
3.1 MiniSTM32开发板	42
3.2 miniIO扩展板	54
第4章 PWM频率测试信号产生与测频	57
4.1 基于PWM的频率信号源产生	57
4.2 基于捕捉功能的频率测量	59
第5章 STM32F103的DMA传送	62
5.1 DMA主要特性	62
5.2 DMA请求映像	63
5.3 DMA配置步骤	64
5.4 DMA传输示例	65
5.5 DMA中断	66
5.6 注意事项	67
5.7 常见问题	67
第6章 测试正弦波的生成	68
6.1 STM32F103的DA介绍	68
6.2 正弦波DA输出数组的生成	69
6.3 设置幅值频率相位	69
6.4 DAC输出步骤	70

6.5 串行通信协议	72
6.5.1 通信格式	72
6.5.2 演示程序通讯命令字	73
6.5.3 软件设计	74
第7章 交流信号采样	77
7.1 ADC介绍	77
7.2 ADC主要特征	77
7.3 ADC功能描述	77
7.3.1 规则通道与注入通道	78
7.3.2 ADC启动方式	79
7.3.3 STM32的双ADC模式	79
7.3.4 DMA传输	80
7.4 ADC配置过程	81
第8章 交流电参量计算方法	85
8.1 交流电参量的直接计算方法	85
8.2 交流电参量的DFT计算方法	86
附录A 常用工具使用方法	89
附录B 常见元器件识别	91
B-1 对于插针式的电阻	91
附录C 万用表使用方法	93
C.1 万用表的外表结构	93

第 1 章 嵌入式系统开发基础

嵌入式系统是一种专用的计算机系统，通常嵌入在更大的设备中，用于控制、监测或执行特定任务。它以微控制器（如STM32）或微处理器为核心，集成了硬件、实时操作系统（RTOS）和专用软件，具有高可靠性、低功耗和实时性强的特点。

嵌入式系统广泛应用于智能家居、工业自动化、医疗设备、汽车电子和消费电子等领域。例如，智能手表中的传感器数据处理、汽车中的ABS控制系统、工业机器人的运动控制等，都依赖嵌入式技术。

与通用计算机不同，嵌入式系统通常针对特定需求优化，资源受限但高效稳定。其开发涉及硬件设计、固件编程和软硬件协同调试，是物联网（IoT）和智能化时代的关键技术基础。

1.1 嵌入式系统的组成

我们通常看到的智能控制系统（如空调、微波炉、电视机、手机等）都是一个嵌入式系统装置，取下后盖，我们能够看到一块布满元件的控制板，这就是该智能装置的核心，如图1-1所示。



图1-1 各种智能设备及其控制器

各种智能装置能够工作，主要是由控制板来控制其他部件，如空调控制器可以控制压缩机制冷，电视机控制能够显示和接收信号，汽车可以控制车轮前行等。因此一个嵌入式系统一般都由两部分组成：嵌入式控制系统和被控对象，如图1-2所示。

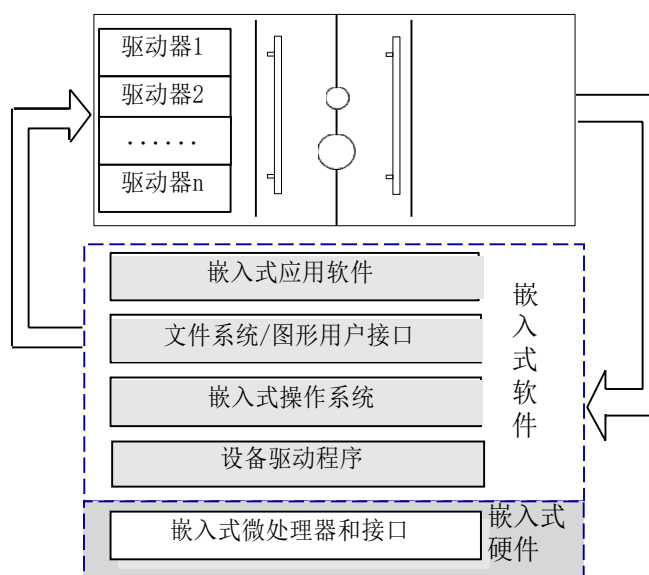


图1-2 嵌入式系统

被控对象接收嵌入式控制系统发出的控制命令，执行所规定的操作或任务。嵌入式控制系统通过各种传感器来获得被控对象的状态，进而产生控制命令控制被控对象。

根据应用场景的不同，嵌入式控制系统可以很简单，也可以很复杂。嵌入式控制系统是整个嵌入式系统的核心，它的核心是一个微处理器，运行相应的软件进行被控对象的控制，因此嵌入式控制系统包括嵌入式硬件和嵌入式软件两部分。简单的嵌入式控制系统在微处理器上直接运行设备驱动程序和应用程序，复杂的嵌入式控制系统还带有操作系统，有图形界面和文件系统等，如图 1-2 所示。

被控对象可以很简单，如手机上的一个微小型电机，当手机处于震动接收状态时打开；也可以很复杂，如工业控制机器，上面集成了多个中大型电机和多种传感器，从而可以执行各种复杂的动作和接收各种状态信息。

1.1.1 嵌入式系统的硬件

嵌入式系统的硬件主要包括电源、嵌入式处理器、存储器（程序存储器和数据存储器）模块、可编程逻辑器件、嵌入式系统周边元器件、各种 I/O 接口、总线、必备的接插件以及外部设备等。如图 1-3 所示。

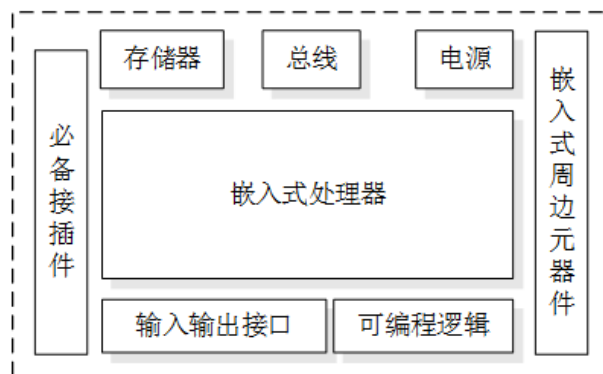


图 1-3 嵌入式硬件系统的构成

嵌入式系统的硬件以嵌入式处理器为核心，简单的嵌入式系统使用 8 位处理器，如 51

单片机系列。在一些复杂的应用中，已经从 8 位的控制器到 32 位的数字信号处理器（DSP），

到 64 位的芯片，嵌入式处理器的种类繁多。目前一般应用场合采用嵌入式微处理器（如 51 系列单片机、ARM7 或 ARM9 等），在信息处理能力要求比较高的场合，可采用嵌入式 DSP，以完成高性能信号处理。越来越多的嵌入式系统开始采用分布式微处理器网络，这种网络可以通过有线或无线的方式通信，由常规的网络管理通信协议远程监管和控制。RISC（精简指令集计算机）芯片在高端嵌入式应用中常见，甚至在不算很高档的应用中也是如此。例如，Sega 公司的一种游戏机有三个 32 位 Hitachi SHRISC 处理器。小型、可嵌入、完全基于 PC 的板卡常见于中等规模系统中；而超小型的微控制器则统治低端系统，如 PHILIPS 的 P89C51 系列和 MICROCHIP 公司的 PIC 系列。处理器的速度、总线宽度的增加，以及制造商能集成到一个芯片上的外设数量的增多都增加了嵌入式系统可用芯片的种类，以及其对应用的适应能力。

有些应用场合要求具有 USB 接口、I2C 总线接口、SPI 接口、CAN 总线接口、以太网接口以及 A/D、D/A、PWM 等接口，因此，嵌入式系统的硬件要根据实际应用选择或裁剪，以最少成本满足应用系统的要求。

1.1.2 嵌入式系统的软件

简单的嵌入式系统在微处理器上直接写设备驱动程序和嵌入式应用程序，如基于 STM32F103RC 系列单片机设计的嵌入式控制系统。对于大部分复杂的嵌入式系统，由于性能要求越来越高，通常需要嵌入式操作系统。因此嵌入式系统的软件包括设备驱动程序、嵌入式操作系统、应用程序接口 API 层（文件系统、图形用户接口等）以及实际应用软件层，如图 1-2 所示。

系统安装的硬件设备必须经过驱动才能使用，设备驱动程序为上层软件提供调用该硬件的操作接口。上层软件只需要调用驱动程序提供的接口，而不必关心设备内部的具体操作，就可以控制硬件设备。

嵌入式操作系统在复杂的嵌入式系统中发挥着非常重要的作用，有了嵌入式操作系统，进程管理、进程间的通信、内存管理、文件管理、驱动程序、网络协议等都可实现。常用的嵌入式操作系统有 VxWorks、pSOS、PalmOS、 μ C/OS-II、 μ CLinux、WinCE 等。

应用程序接口层为上层应用提供接口，如文件系统和图形用户接口等。应用软件是在嵌入式操作系统支持下通过调用 API 函数，结合实际应用编制的用户软件。

1.2 嵌入式系统的开发

1.2.1 嵌入式系统的开发流程

由于很多人只是在厨房、客厅或是汽车中遇到嵌入式系统，所以通常认为嵌入式系统就是硬件。其实嵌入式系统还需要开发软件、设计被控对象才能实现其功能，而且要把程序烧录到系统附带的 ROM 中，以便它能实现自己的使命。

嵌入式系统的开发原则是“物尽其用”，其软硬件的设计都必须高效，所以要量体裁衣，以提高系统整体的性能、效率和性价比。由于嵌入式系统涉及到硬件和软件，以及开发工具的特殊性，所以嵌入式系统的开发过程和通用系统的开发过程稍有不同。应该包括：（1）总体论证、需求分析，确定系统实现的功能以及实现的技术指标，确定系统的组成方案；（2）系统总体设计，系统功能分配，软硬件任务及相互关系，系统体系结构设计，单片机选型；（3）系统详细设计，硬件设计、软件设计、被控对象设计，硬件设计包括设计硬件电路原理图、关键电路的试验和确定、元器件的分配、PCB 版的设计、组装并进行初步电气检查；软件设计包括绘制软件功能图、确定算法和数据结构、划分程序模块、画程序流程图、代码编写、编译、连接、软件仿真调试；被控对象设计可以包括在硬件设计中，也可以单列出来，主要是各种被控对象的选型；（4）系统集成、联机调试，检查硬件排除故障，装入软件运行，并调试；反复调试直至符合设计要求；（5）系统交与用户进行测试，使用和维护，如图 1-4 所示。

确定了设计任务和设计目标后，通过需求分析提炼出需求分析规格说明书，作为正式设计指导和验收的标准。系统的需求一般分功能性需求和非功能性需求两方面。功能性需求是系统的基本功能，如输入输出信号，操作方式等；非功能性需求包括系统性能，成本，功耗，体积，重量等因素。

嵌入式系统的体系结构设计任务是描述系统要实现所需的功能和非功能需求的系统框架，包括对硬件、软件和被控对象的功能划分，系统的软件、硬件选型，系统最终呈现的形状等。一个好的体系结构是嵌入式系统开发成功与否的关键。

体系结构的设计并不包括系统的实现方式，只说明系统做些什么以及由哪部分做来达到系统的功能需求和非功能需求。体系结构是系统整体结构的一个规划和描述。

硬件、软件和被控对象的设计就是基于嵌入式系统结构，对系统的硬件、软件和执行机构进行详细设计。为了缩短产品开发周期，硬件、软件和执行机构往往是并行进行设计的。硬件设计就是确定嵌入式处理器型号、外围接口及外部设备、绘制相应的硬件系统的电路原理图和印制板图。被控对象的设计主要是选型，选择合适的被控对象，配置相应的驱动器以及传感器、放大器、信号变换电路等，并考虑与硬件的连接方法。在整个嵌入式系统软硬件的设计过程中，嵌入式系统设计的工作大部分都集中在软件设计上。面向对象技术、软件组件技术、模块化设计是现代软件工程经常采用的方法。

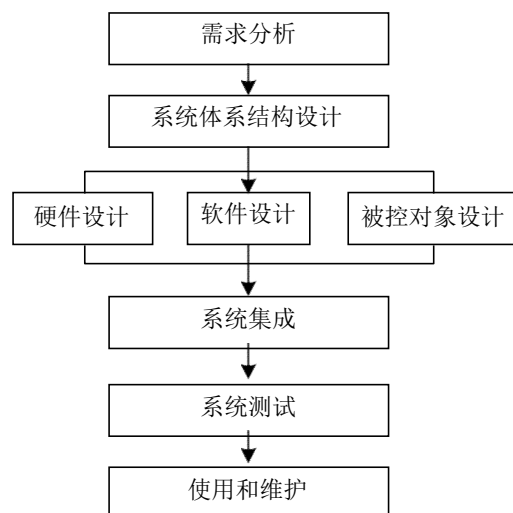


图1.4 嵌入式系统开发流程

系统集成是指当所有的软硬件和被控对象设计完成后，将它们集成为一个能运转的系统，这个阶段不仅仅是把所有构件插在一起，还要进行综合的测试，在系统集成中通常可以发现错误。只有通过集成测试成功的系统才能交给用户使用。

因为嵌入式系统使用的调试工具比在桌面系统中可找到的工具有限得多，因此，需要在系统集成时更多地发现问题，详细地观察系统以准确确定错误。在集成之前，必须确保在体系结构和各构件设计阶段尽可能容易地按阶段组装系统和相对独立地测试系统功能。按阶段架构系统并且正确运行选好的测试，经常能更容易地找到这些错误。如果每次只对一部分模块排错，很可能更容易发现和识别简单的错误。只有在早期修正这些简单的错误，才能发现那些只有在系统高负荷时才能确定的、比较复杂或是含混的错误。

系统测试是指对设计并集成好的系统进行测试，看其是否满足需求规格说明书中给定的功能和非功能需求。经过测试的系统就可以投入使用，在使用过程中进行维护。

1.2.2 嵌入式系统的开发和运行环境

通用计算机的软件开发一般都是以本地（Native）编译方式进行的，即在同一台计算机上开发、调试、测试和运行，开发环境和运行环境是一致的。然而，由于嵌入式系统一般嵌入在设备中运行，一般不支持这种本地环境开发，那么如何实现它们呢？嵌入式系统上的软件开发通常采用交叉开发（Cross Compile）方式。交叉开发环境建立在宿主机（Host）上，宿主机又称开发机、上位机，一般是一台通用计算机，如 PC。对应嵌入式系统称为目标机。目标机是指各式各样的嵌入式设备，如手机、掌上电脑等，或者是嵌入式厂商提供的一套专用于开发的评估板（又称开发板），甚至是基于软件的模拟器。一般来说，嵌入式系统的调试也常采用交叉的方式。

因此，从运行平台来分，嵌入式软件可以分为：

- 运行在开发环境上的软件：设计、开发、测试工具等。它们运行在上位机

上，如PC上。

- 运行在运行环境上的软件：嵌入式操作系统、应用程序、驱动程序及部分开发工具。它们运行在嵌入式设备上。

嵌入式软件的开发和运行环境的关系如图 1-5 所示。

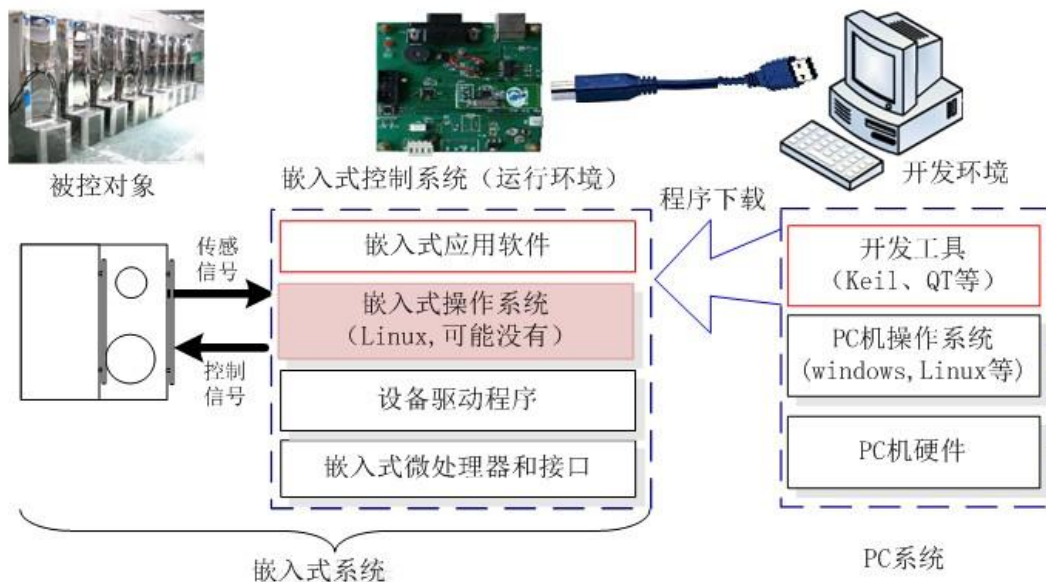


图1-5 嵌入式系统开发和运行环境

具体来说，嵌入式应用软件开发的过程分为三个大的阶段：（1）先在宿主机上编写程序；可以用任何文本编辑器或集成环境中的编辑器编写源程序（一般使用汇编语言或 C / C++）；（2）使用宿主机上的交叉编译、汇编和链接工具生成只有在目标机（嵌入式系统）中才能执行的二进制代码，这包括编译、链接和重定位阶段。编译阶段就是对每一个源文件进行交叉汇编或编译，生成只有嵌入式系统的处理器才能执行的目标代码，通常将这样的编译称为交叉编译；链接与重定位阶段将所有产生的目标文件链接成一个目标文件，即可重定位程序，进行重定位，把物理存储器 RAM 的地址指定给可重定位程序，这一阶段的任务是产生一个在嵌入系统上可执行的二进制映像文件；（3）下载调试运行阶段，将第二阶段生成的可执行二进制映像文件下载到目标机的 RAM 中进行调试运行，如果文件太大而 RAM 太小，则必须下载到 FLASH 中运行；下载完成后，复位即可运行。如果运行结果达不到要求，将进一步返回修改程序，重复上述过程。

在早期的嵌入式系统中，代码的长度和执行速度都非常重要。由于汇编语言能够最有效地简化代码，加速它们的执行，所以汇编语言一直是嵌入系统的专用语言。甚至当 C 语言盛行，出现了跨平台的 C 编译器后，许多程序员仍然继续使用汇编语言。C 语言比汇编程序大约长和慢 25%，但是比较容易编写。即使是今天 ROM 越来越大时，我们仍然希望代码越短越好，所以汇编语言程序仍在使用中。

1.2.3 嵌入式系统的硬件设计

嵌入式系统的硬件设计主要是指设计并制作控制板。设计控制板的一般过程

是：（1）画电路控制的原理图；（2）生成 PCB 图；（3）制作 PCB 板；（4）焊接元器件。

早期使用手工画原理图，后来使用专门的 EDA（Electronic Design Automation，电路设计自动化）软件来画原理图，EDA 是指将电路设计中各种工作交由计算机来协助完成。如电路原理图（Schematic）的绘制、印刷电路板（PCB）文件的制作、执行电路仿真（Simulation）等设计工作。随着电子技术的蓬勃发展，新型元器件层出不穷，电子线路变得越来越复杂，电路的设计工作已经无法单纯依靠手工来完成，电子线路计算机辅助设计已经成为必然趋势，越来越多的设计人员使用快捷、高效的 CAD 设计软件来进行辅助电路原理图、印制电路板图的设计，打印各种报表。使用较多的 EDA 软件有 Protel 和 Altium Designer 等软件。

Protel 是 Altium 公司在 80 年代末推出的 EDA 软件，在电子行业的 CAD 软件中，它当之无愧地排在众多 EDA 软件的前面。它较早就在国内开始使用，在国内的普及率也最高，有些高校的电子专业还专门开设了课程来学习它，几乎所有的电子公司都要用到它，许多大公司在招聘电子设计人才时在其条件栏上常会写着要求会使用 Protel。早期的 Protel 主要作为印制板自动布线工具使用，运行在 DOS 环境，对硬件的要求很低，在无硬盘 286 机的 1M 内存下就能运行，但它的功能也较少，只有电路原理图绘制与印制板设计功能，其印制板自动布线的布通率也低，而现今的 Protel 已发展到 DXP 2004，是个庞大的 EDA 软件，完全安装有 200 多 M，它工作在 WINDOWS 环境下，是个完整的板级全方位电子设计系统，它包含了电路原理图绘制、模拟电路与数字电路混合信号仿真、多层印制电路板设计（包含印制电路板自动布线）、可编程逻辑器件设计、图表生成、电子表格生成、支持宏操作等功能，并具有 Client/Server（客户/服务器）体系结构，同时还兼容一些其它设计软件的文件格式，如 ORCAD，PSPICE，EXCEL 等，其多层印制线路板的自动布线可实现高密度 PCB 的 100% 布通率。在国内 Protel 软件较易买到，有关 Protel 软件和使用说明的书也有很多，这为它的普及提供了基础。

Altium Designer 是原 Protel 软件开发商 Altium 公司推出的一体化的电子产品开发系统，主要运行在 Windows 操作系统。这套软件通过把原理图设计、电路仿真、PCB 绘制编辑、拓扑逻辑自动布线、信号完整性分析和设计输出等技术的完美融合，为设计者提供了全新的设计解决方案，使设计者可以轻松进行设计，熟练使用这一软件必将使电路设计的质量和效率大大提高。目前最新版本为：Altium Designer 16.1.12。

Altium Designer 除了全面继承包括 Protel99SE、ProtelDXP 在内的先前一系列版本的功能和优点外，还增加了许多改进和很多高端功能。该平台拓宽了板级设计的传统界面，全面集成了 FPGA 设计功能和 SOPC 设计实现功能，从而允许工程设计人员能将系统设计中的 FPGA 与 PCB 设计及嵌入式设计集成在一起。由于 Altium Designer 在继承先前 PROTEL 软件功能的基础上，综合了 FPGA 设计和嵌入式系统软件设计功能，Altium Designer 对计算机的系统需求比先前的版本要高一些。

1.2.4 嵌入式系统的软件设计

嵌入式系统开发中最重要的是仍然是软件设计。软件设计一般要基于硬件选型。如果选用 ARM 系列单片机作为微处理器，一般使用 C51 系列软件编写 C 语言或汇编程序。Keil C51 是美国 KeilSoftware 公司出品的 C51 系列兼容单片机 C 语言软件开发系统，与汇编相比，C 语言在功能、结构性、可读性、可维护性上有明显的优势，因而易学易用。Keil 提供了包括 C 编译器、宏汇编、连接器、库管理和一个功能强大的仿真调试器等在内的完整开发方案，通过一个集成开发环境（ μ Vision）将这些部分组合在一起。运行 Keil 软件需要 WIN XP 及后续版本的操作系统。如果你使用 C 语言编程，那么 Keil 几乎就是你的不二之选，即使不使用 C 语言而仅用汇编语言编程，其方便易用的集成环境、强大的软件仿真调试工具也会令你事半功倍。

如果选用 ARM 系列微处理器，可以选用 QT、IAR、Eclipse 等软件编写程序。QT 是一个 1991 年由奇趣科技开发的跨平台 C++ 图形用户界面应用程序开发框架。它既可以开发 GUI 程序，也可用于开发非 GUI 程序，比如控制台工具和服务器。QT 是面向对象的框架，使用特殊的代码生成扩展（称为元对象编译器（MetaObjectCompiler, moc））以及一些宏，易于扩展，允许组件编程。2008 年，奇趣科技被诺基亚公司收购，QT 也因此成为诺基亚旗下的编程语言工具。2012 年，QT 被 Digia 收购。2014 年 4 月，跨平台集成开发环境 QtCreator 3.1.0 正式发布，实现了对于 iOS 的完全支持，新增 WinRT、Beautifier 等插件，废弃了无 Python 接口的 GDB 调试支持，集成了基于 Clang 的 C/C++ 代码模块，并对 Android 支持做出了调整，至此实现了全面支持 iOS、Android、WP。

Android 是一种基于 Linux 的自由及开放源代码的操作系统，主要使用于移动设备，如智能手机和平板电脑，由 Google 公司和开放手机联盟领导及开发。第一部 Android 智能手机发布于 2008 年 10 月。Android 逐渐扩展到平板电脑及其他领域上，如电视、数码相机、游戏机等。

1.2.5 嵌入式应用软件的下载

当完成嵌入式硬件和应用软件的开发后，必须将编译后的二进制映像文件下载到嵌入式微处理器中去运行。

往微控制器里下载程序有很多种方法，并且不同的微控制器又有不同的下载方法。一般可分为烧录（或烧写）式和网络式。烧录式是用某种机制将目标代码映像烧录到目标平台上的 Flash 中，这种方式不方便，一般用在开发过程晚期的系统集成测试阶段和最后的发布阶段；网络式是通过某种机制，让目标机直接通过网络加载所需的目標代码映像到内存，这种方式方便，无需烧录 Flash ROM，但通常需要操作系统的支持，一般用在开发过程的前期和中期。

在单片机系统的开发中通常只能使用烧录式，Flash 芯片的烧写又分为脱机（Offline）方式和在线方式（In-system-programming），脱机方式需要将 Flash ROM 芯片从目标板上取下，一般在目标板上做成插座式安装，使用专门的编程器和配套的软件来烧写；在线方式无需将 flash ROM 芯片从目标板上取下，直接通过 JTAG 等下载器，将所需烧录的目标映像下载到目标板的 RAM 或 ROM 中。

不管哪种方法，都需要有相应的下载电路，不同的方法的下载电路也都不一样，

常用下载接口有 JTAG, ICSP, 还有串口, USB 等。具体设计下载电路还要参考芯片的数据手册。有了下载电路后, 下载器也是必要的, 除了 USB 接口外, 不同厂家的微控制器芯片, 所提供的下载器也是不一样的, 如 Microch, TI, STC, AVR……。使用不同的下载器, 需要参考相关厂家的使用说明。

1.3 思考与练习

- (1) 通过在网上查阅资料, 了解嵌入式系统的发展。
- (2) 一个嵌入式系统一般由哪几个部分构成。
- (3) 了解嵌入式应用的开发流程, 和一般的应用软件的开发有何不同。
- (4) 嵌入式系统的开发环境和运行环境分别是什么。

第2章 常用软件介绍

本章主要介绍在嵌入式系统设计中常用的电路EDA设计软件AD，和软件开发平台keil μvision5。

2.1 Altium Designer电路设计

Altium Designer软件是Altium公司推出的一体化的电子产品开发系统，主要运行在Windows操作系统。它基于一个软件集成平台，把为电子产品开发提供完整环境所需的工具全部整合在一个应用软件中。Altium Designer包含所有设计任务所需的工具：原理图和HDL设计输入、电路仿真、信号完整性分析、PCB设计、拓扑逻辑自动布线、基于FPGA的嵌入式系统设计和开发等。

使用Altium Designer设计印制电路板大致分为四个步骤：原理图设计、创建PCB文件、PCB设计、PCB板制作。详细的流程图如图2-1所示。

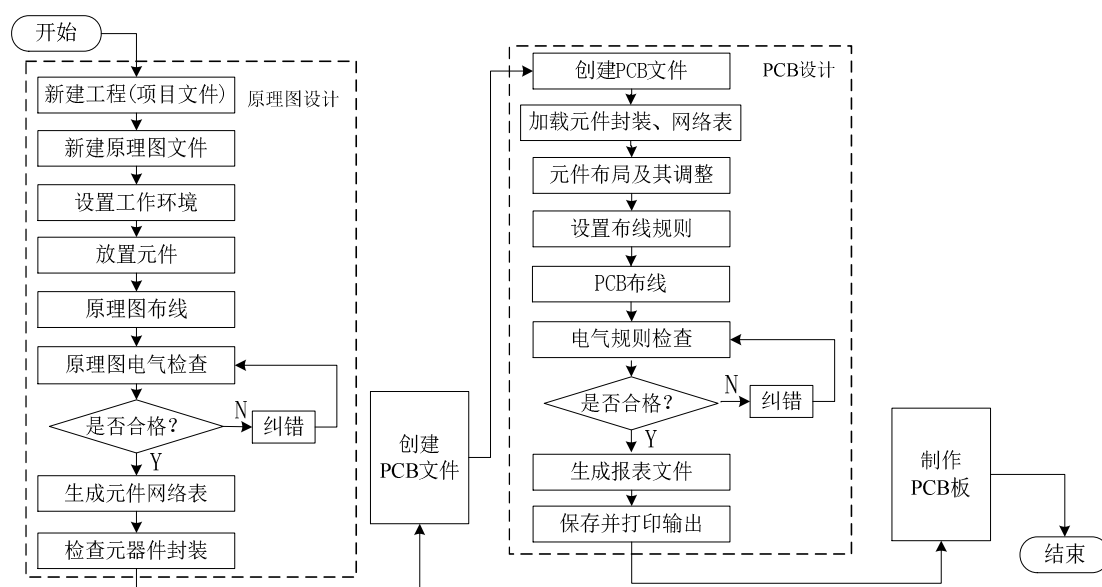


图2-1 制作印制电路板的流程

2.1.1 原理图设计

使用Altium Designer来设计原理图的步骤如图2-1左边虚线框部分，大致包括如下几个步骤。

(1) 创建PCB工程（项目文件）

启动AD15后，选择菜单【文件】/【新建】/【Project】/【PCBProject】命令；完成后如图2-2所示。

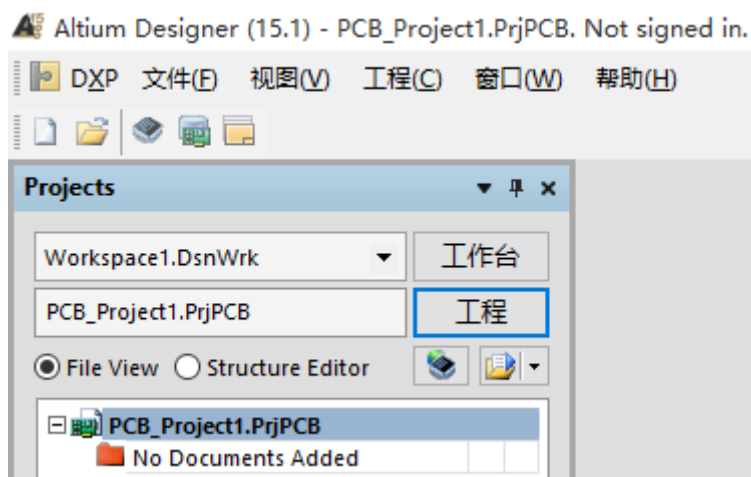


图2-2创建PCB工程

选择【文件】/【保存工程】菜单命令，弹出保存对话框

【Save[PCB_Project1.PrjPCB]AS...】对话框如图2-3所示；选择保存路径后在【文件名】栏内输入新文件名将PCB工程保存到自己自己建立的文件夹中。

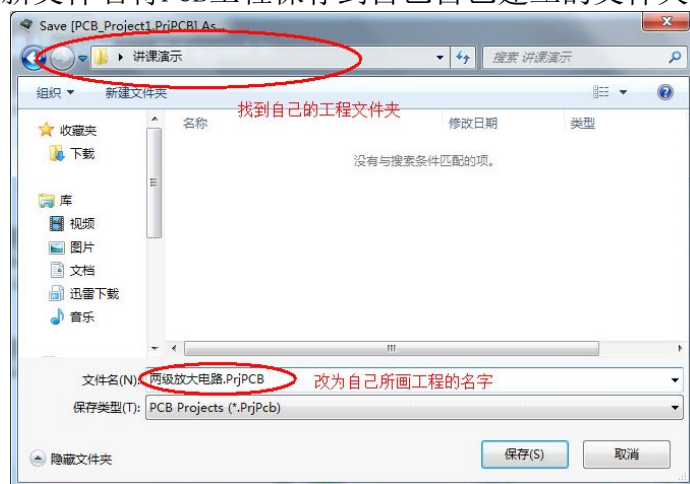


图2-3保存PCB工程

(2) 创建原理图文件

在新建的PCB项目（工程）下，选择菜单【文件】/【新建】/【原理图】命令；完成后如图2-4所示。

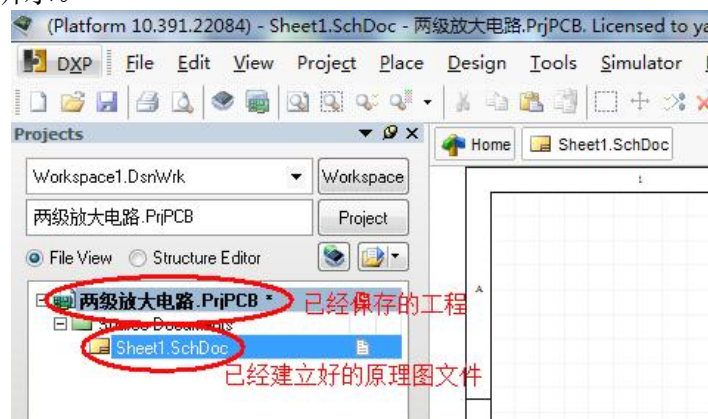


图2-4创建原理图文件

然后选择【文件】/【保存】菜单命令，弹出保存对话框

【Save[Sheet1.SchDoc]AS...】对话框如图2-5所示；选择保存路径后在【文件名】

栏内输入新文件名将原理图文件保存到自己建立的文件夹中。

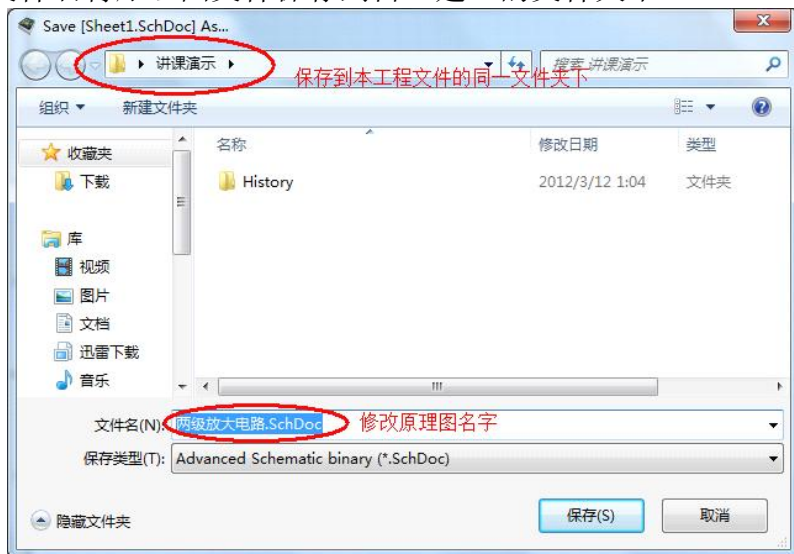


图2-5保存原理图文件

(3) 放置元器件

根据设计功能放置相应的元器件。在放置元件之前需要加载所需要的库（系统库或者自己建立的库，主要是看需要的元件在哪个库中）。

如果知道自己所需要的元件在哪一个库，则只需要直接将该库加载，具体加载方法如下：选择【设计】/【添加/移除库】菜单命令，弹出【可用库】对话框，如图2-6所示，单击安装找到库文件即可。

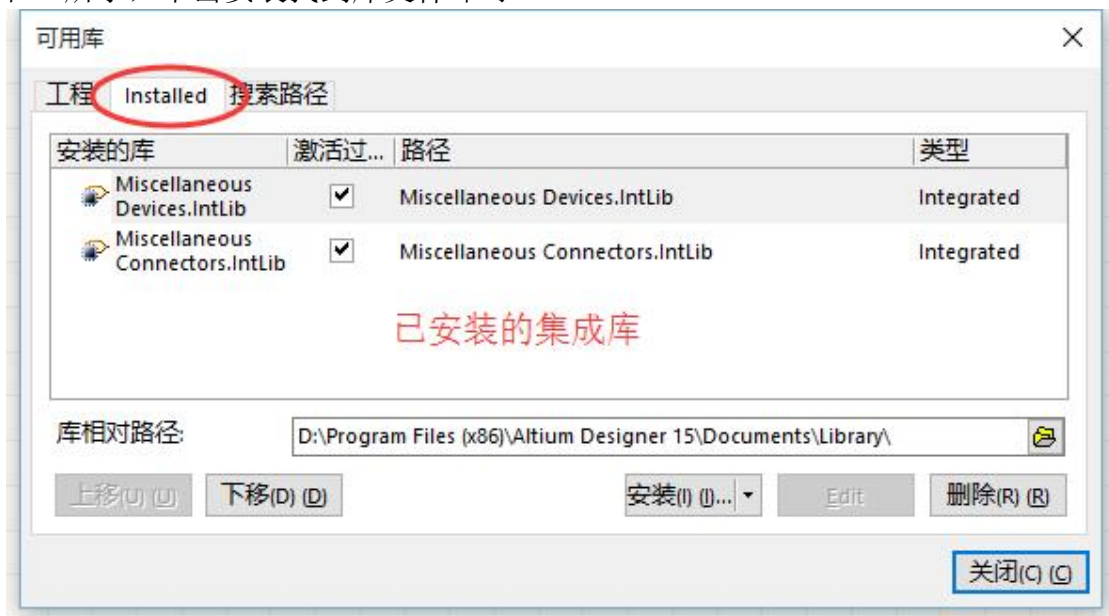


图2-6 加载库文件

加载库文件后，可直接在工具栏中选取元器件。假定设计如图2-7所示的二级放大电路。

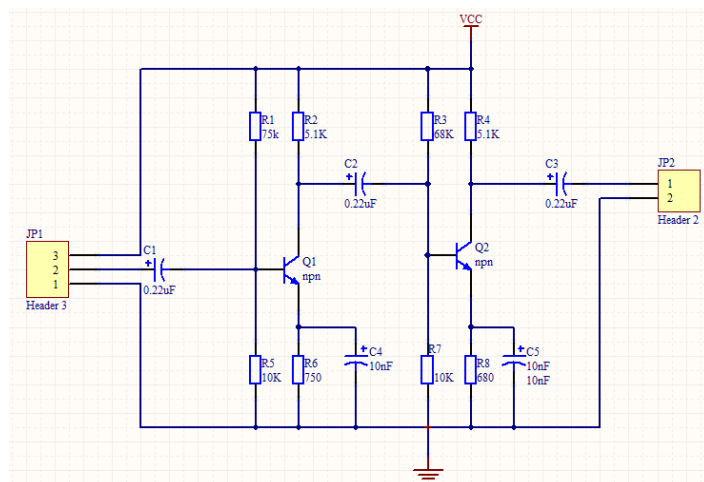


图2-7二级放大电路

如果我们不知道某个需要用的元件在哪一个库的情况下，可以采用搜索元件的方式进行元件放置。具体操作如下：选择【放置】/【器件】菜单命令，弹出【放置端口】对话框如图2-8所示。

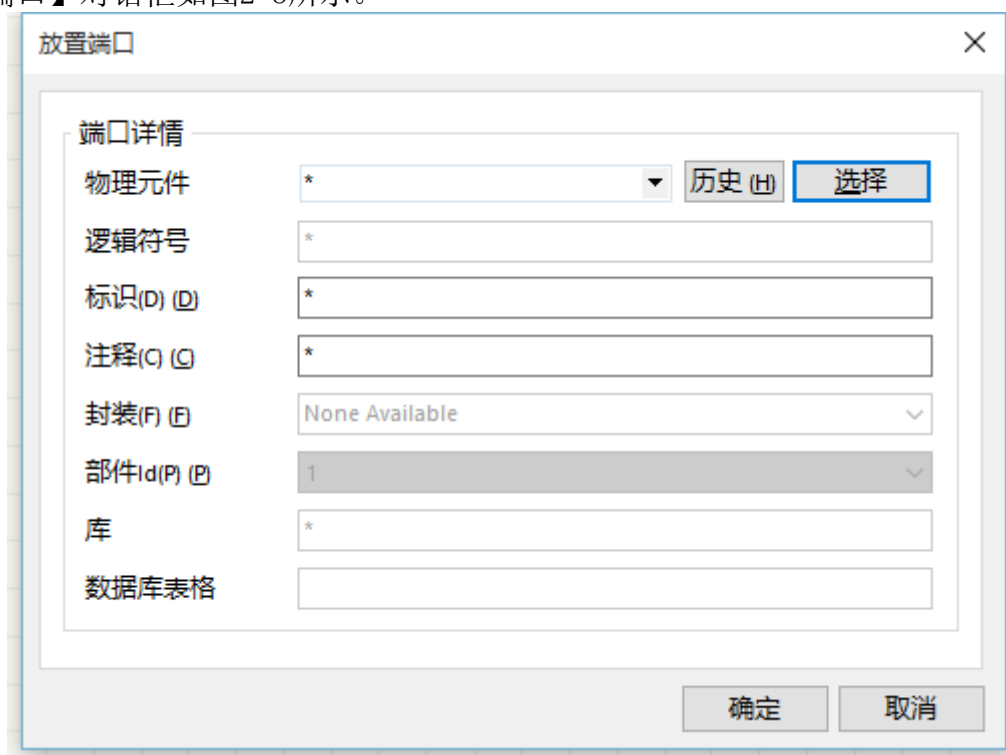


图2-8 选择元器件对话框

选择【选择】，弹出【浏览库】对话框如图2-9所示。在【对比度】输入元器件名称进行筛选。

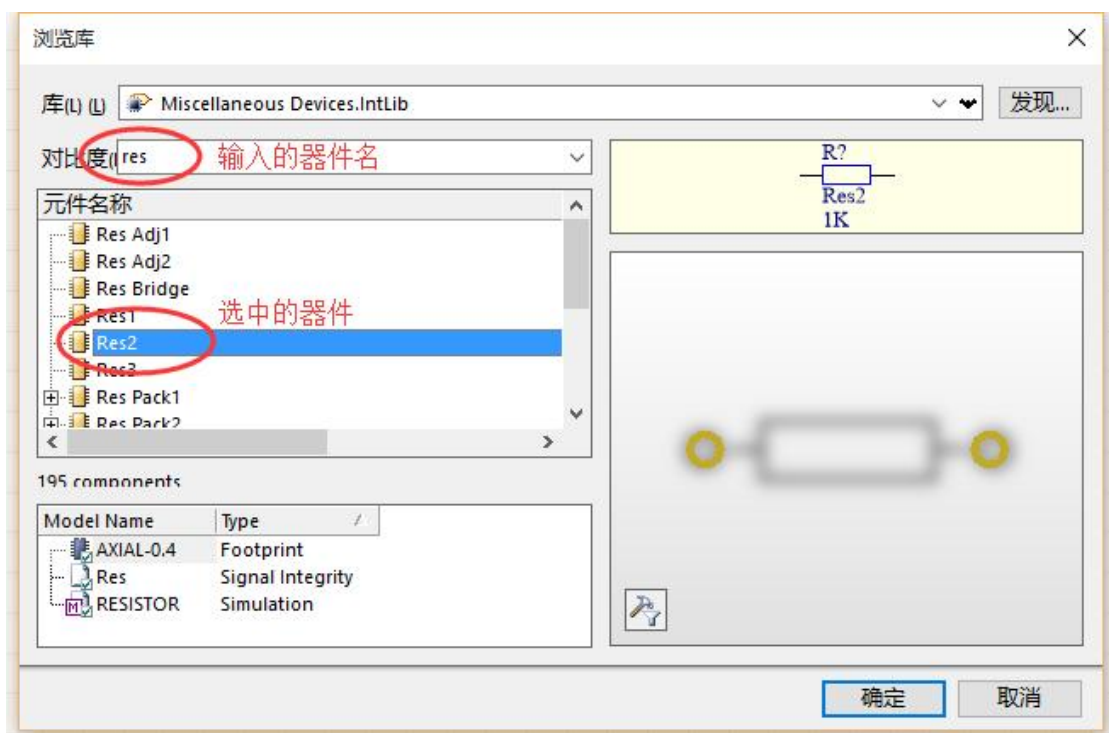


图2-9 元器件浏览并筛选

选中所需的元件后单击【确定】后操作如图2-10所示。

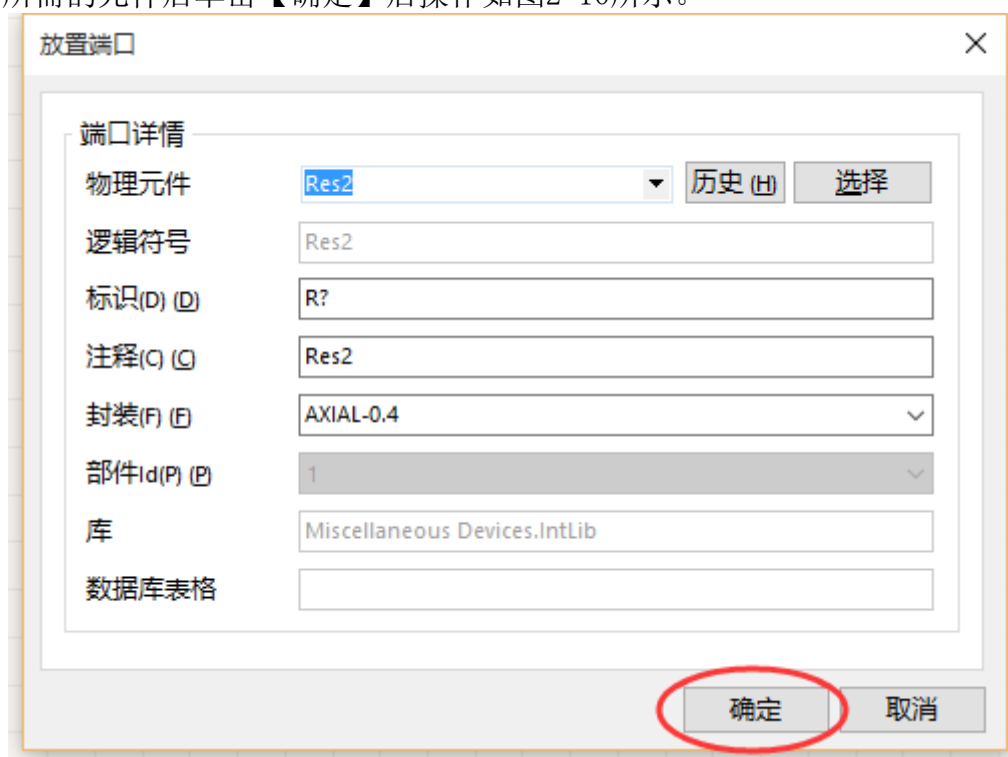


图2-10 放置元器件对话框

此时元件就粘到了鼠标上，并呈现“十”字，单击鼠标左键即可放置元件。如图2-11所示。

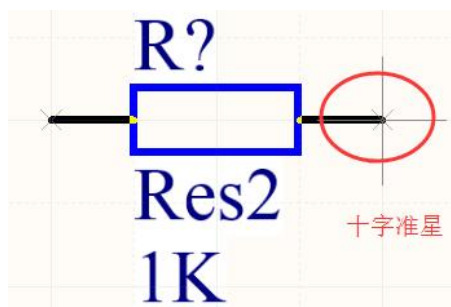


图2-11 元件粘贴到鼠标上

在放置好元件后需要对元件的位置、名字、封装、序号等进行修改和定义。元件属性修改方法如下（除元件位置外其他修改也可以放到布线以后再进行）：

在元件上双击鼠标左键，弹出【Properties for Schematic Component in Sheet [原理图文件名]】对话框，属性修改如图2-12所示。

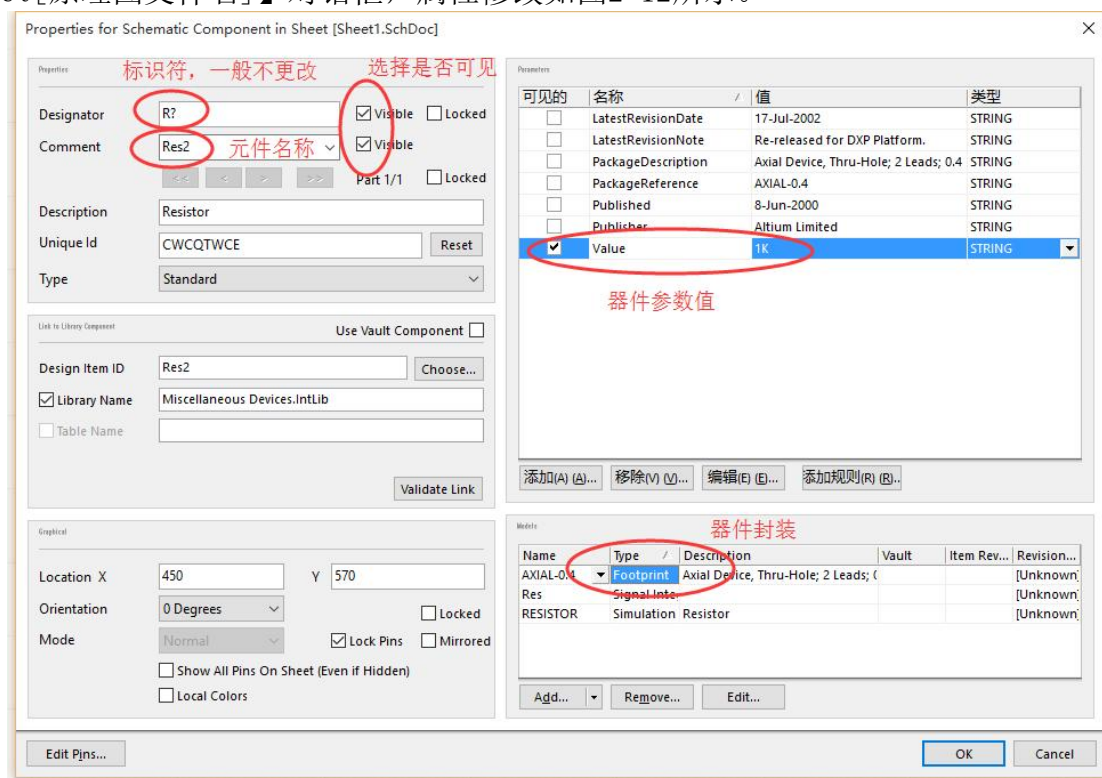


图2-12元器件属性对话框

封装形式的修改过程：在如图2-12所示右下角对话框中的models模块，点击框中所示的Footprint，，然后点击edit，过程如图2-13所示。

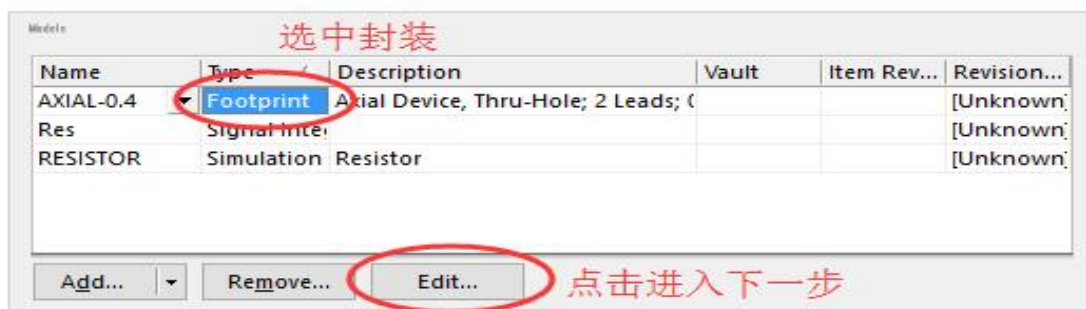


图2-13 models模块

点击Edit按钮进入下一步，进入PCB模型界面，如图2-14所示。



图2-14 PCB模型

如果当前原件的封装不符合需求，点击浏览，进入浏览库界面，如图2-15所示。

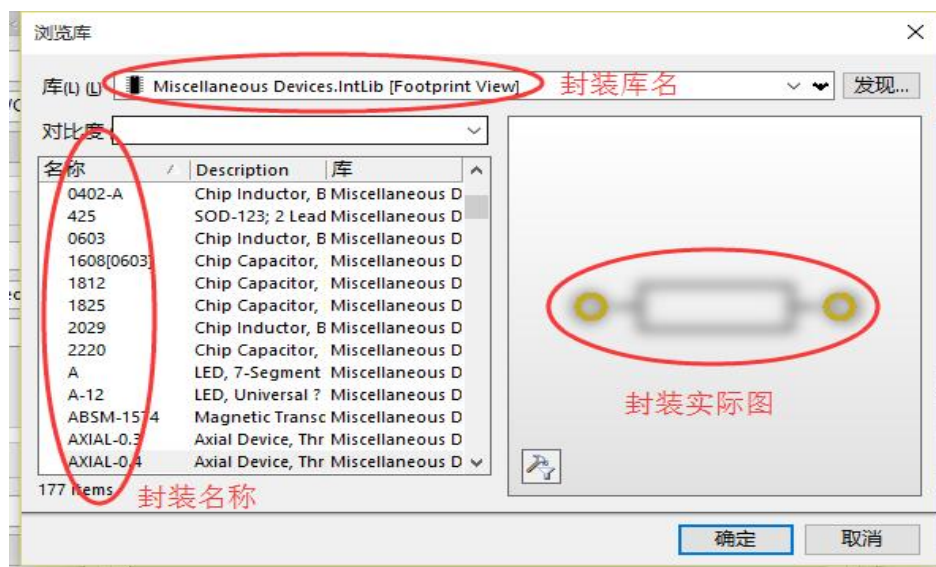


图2-15 修改封装

在放好元件位置后即可对原理图进行布线操作。选择【放置】/【线】菜单命

令，此时将带十字型的光标放到元件引脚位置单击鼠标左键即可进行连线（注意拉线过程不应一直按住鼠标左键不放），将导线拉到另一引脚上单击鼠标左键即放完一根导线，放置完导线单击右键或者【Esc】键结束放置。

（4）检查原理图的正确性

原理图电气连接是否正确，需要通过编译进行检查。选择【工程】/【CompilePCBProject[工程名]】；若无错误提示，即通过电器规则检查，如有错误，则需找到错误位置进行修改调整。（电气检查规则建议初学者不要更改，待熟练后再更改）

（5）对原理图中的元器件进行注解

通过编译后，需对元器件进行注解，其实就是为器件设置标号。选择【工具】/【注解】菜单命令，弹出【注解】对话框，如图2-16所示。

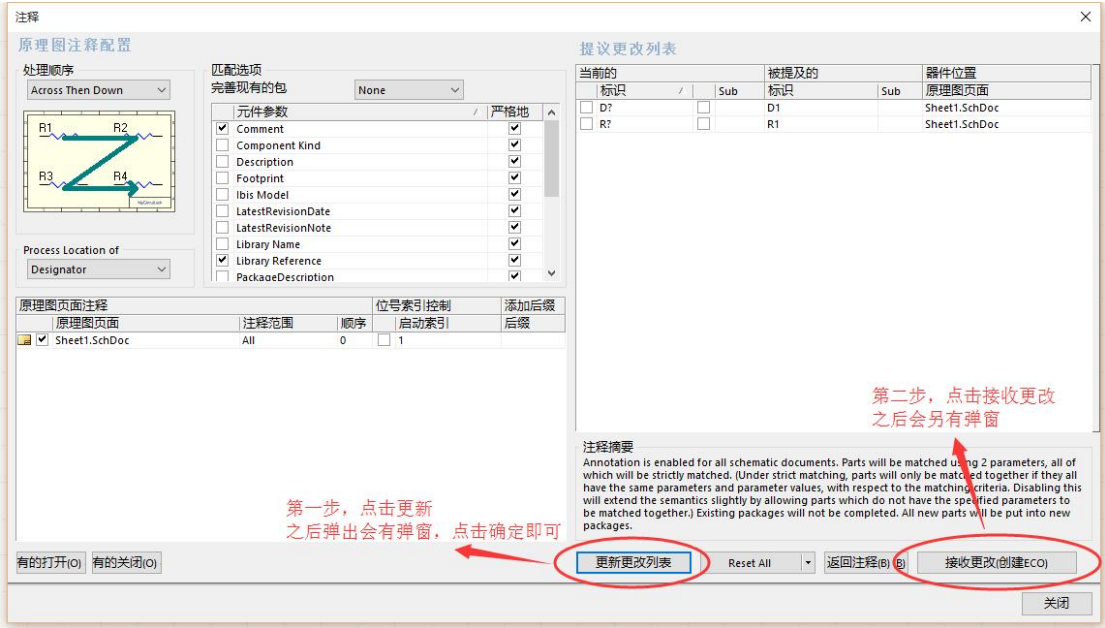


图2-16 注解原件

点击【更新更改列表】按钮，之后弹出弹窗点击确定即可，然后点击【接受更改】按钮，弹出【工程更改顺序对话框】，如图2-17所示。

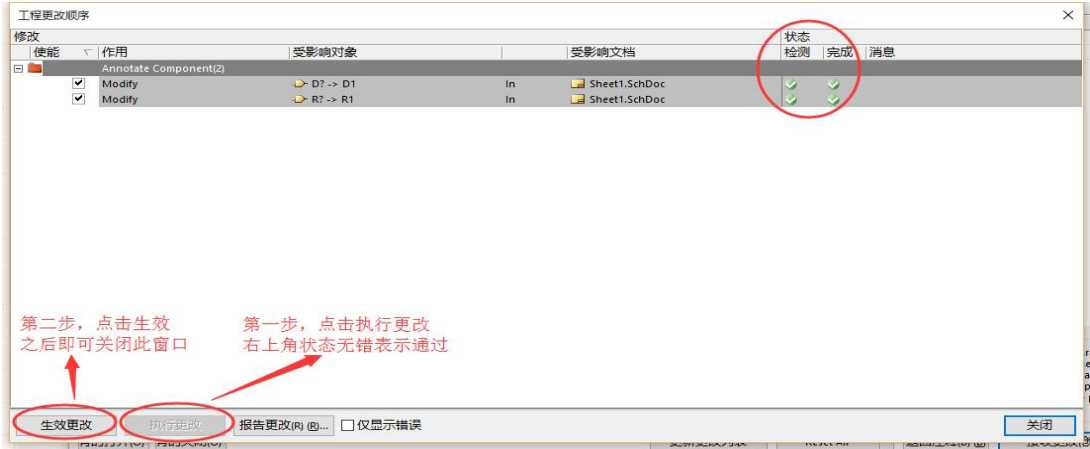


图2-17 执行更改

在【工程更改顺序】对话框下，第一步点击执行更改，右上角状态无错表示通过，第二步点击生效更改之后即可关闭次窗口。

(6) 元器件封装检查

原理图中的元器件，只是一种表示符号，然而这些符号需要对应实际生产的元器件，就是封装，所以需要进行封装检查。选择【工具】/【封装管理器】菜单命令，弹出【Footprint Manager-[工程名]】对话框，如图2-18所示。

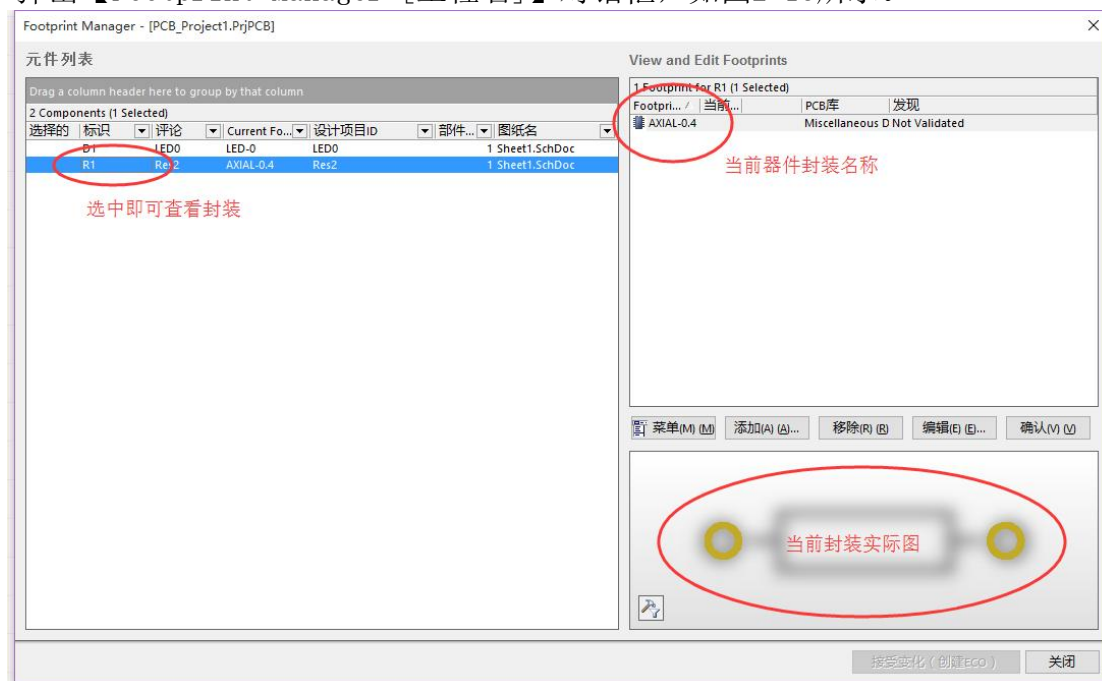


图2-18 封装管理

(7) 保存输出

到此为止，原理图设计完成，可以保存输出了。选择【文件】/【保存】将设计好的原理图保存起来。

2.1.2 PCB设计

在PCB (Printed Circuit Board, 印制电路板) 出现之前，电子元器件之间的互连都是依托电线直接连接完成的。而如今，电线仅用在实验室做试验应用而存在；印刷电路板在电子工业中已占据了绝对控制的地位。PCB是通过在绝缘程度非常高的基材上覆盖上一层导电性良好的铜膜，采用刻蚀工艺，根据PCB 的设计，在敷铜板上经腐蚀后保留铜膜形成电气导线，一般在导线上再附上一层薄的绝缘层，并钻出安装定位孔、焊盘和过孔，适当剪裁后供装配使用。

日常使用最多的电路板是双面板，下边就只介绍双面板的绘制。利用原理图创建PCB的流程如图2-1所示右部的虚线框中。设计过程中涉及到一些重要的概念和规则。

(1) 创建PCB文件

在新建的PCB项目（工程）下，选择菜单【文件】/【新建】/【原理图】命令；完成后如图2-19所示，之后再保存。

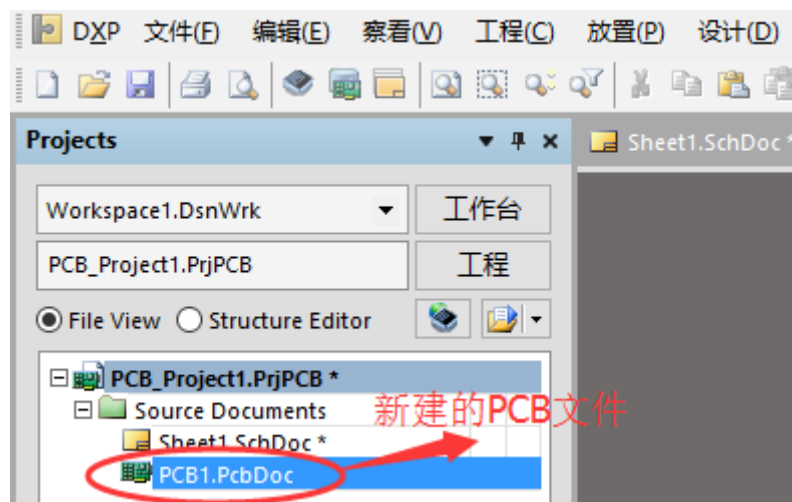


图2-19 新建的PCB文件

(2) 更新PCB文件

先回到原先的原理图文件，选择菜单【设计】/【Update PCB Document...】命令，弹出【工程更改顺序】窗口，如图2-20所示。

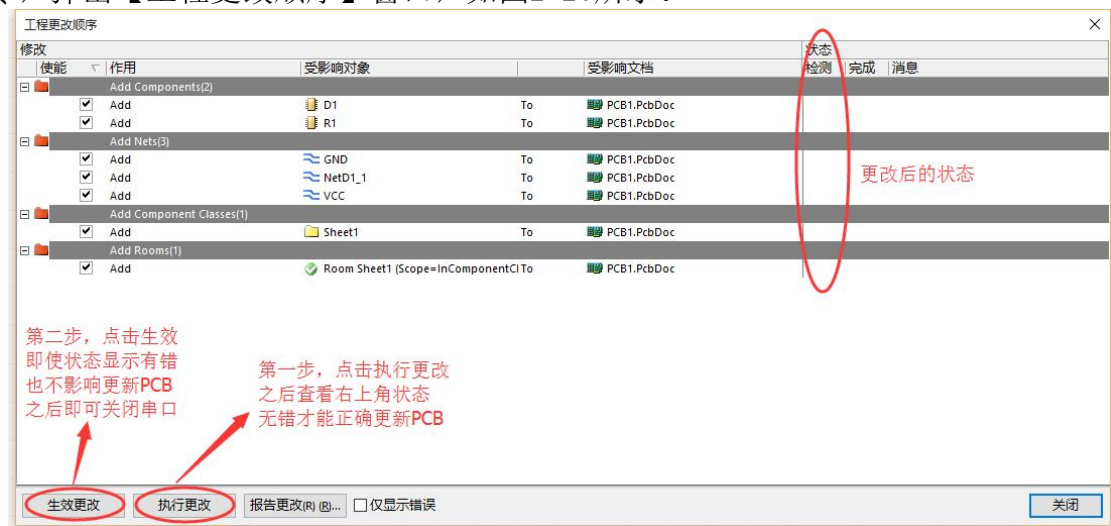


图2-20 更新PCB文件

(3) PCB板层简单介绍，图纸最底部即可任意切换板层，如图2-21所示。



图2-21 PCB板层

- TopLayer（顶层）顶层布线层，用来画元件之间的电气连接线。
- BottomLayer（底层）底层布线层，作用与顶层布线层。
- MidLayer1（中间层1）作用是在制多层板时在此层也会绘制电气连接线，不过多层板成本比较高。
- Mechanical Layers(机械层) 可用来绘制 PCB 印制板的外形，及需挖孔部位，也可用来做注释 PCB 尺寸等，可用来绘制 PCB 印制板的外形，及需挖孔部位，注意 PCB 外形，挖空部位和 PCB 的注释尺寸不要用同一机械层，比如机械层 1 用来绘制 PCB 外形及挖空，机械层 13 用来注释尺寸等，分开后印

制板厂家的技术人员会根据此层的东西自己分析是否需要将此层制作出来。

- Top Overlay(顶层丝印层)(黄色)就是板子正面的字符,对应 Top Layer(顶层)单面板就用到这层字符就可以了, Bottom Overlay(底层丝印层)(褐色)对应 BottomLayer(底层)就是板子背面的字符,双面板时候用到以上两层字符。
- Keep Out Layer(禁止布线层)作用是绘制禁止布线区域,如果印制板中没有绘制机械层的情况下,印制板厂家的人会以此层来做为 PCB 外形来处理。如过 KEEP OUT LAYER 层和机械层都有的情况下,默认是以机械层为 PCB 外形,但印制板厂家的技术人员会自己去区分,但是区分不出来的情况下他们会默认以机械层当外形层。
- Multilayer(多层)(银色)所有布线层都包括,一般单双面的插件焊盘就在这层,花条线条就是所有层都画上了。

(4) 原件布局(左键选中即可任意拖动器件)

元件布局不仅影响PCB 的美观,而且还影响电路的性能。在元件布局时应注意:

- 先布放关键元器件(如单片机、DSP、存储器等),然后按照地址线 and 数据线的走向布放其他元件。
- 高频元器件引脚引出的导线应尽量短些,以减少对其他元件及其电路的影响。
- 模拟电路模块与数字电路模块应分开布置,不要混合在一起。
- 带强电的元件与其他元件距离尽量要远,并布放在调试时不易触碰的地方。
- 对于重量较大的元器件,安装到 PCB 上要安装一个支架固定,防止元件脱落。
- 对于一些严重发热的元件,必须安装散热片。
- 电位器、可变电容等元件应布放在便于调试的地方。

(5) 设计PCB设计规则

可以通过规则编辑器设置各种规则以方便后面设计中使用,执行菜单命令【设计】/【规则】,在弹出的规则编辑对话框,在上面进行逐一设置。如图2-22 所示。一般需要设置安全间距、导线线宽、电源线宽、敷铜间歇设置。其中安全距离在 clearancese 上设置;导线线宽在 width 上设置,先设定所有线宽,将其最大值设置为 1mm,最小值设置为 0.3mm,优先值为 0.6;电源线宽的设置需要在 width 上单击鼠标右键,在弹出的对话框中选择 New Rule,在新建的 With_1 中进行设置;敷铜间歇是在 Plane 下的 polygon Connect Style 上设置的。

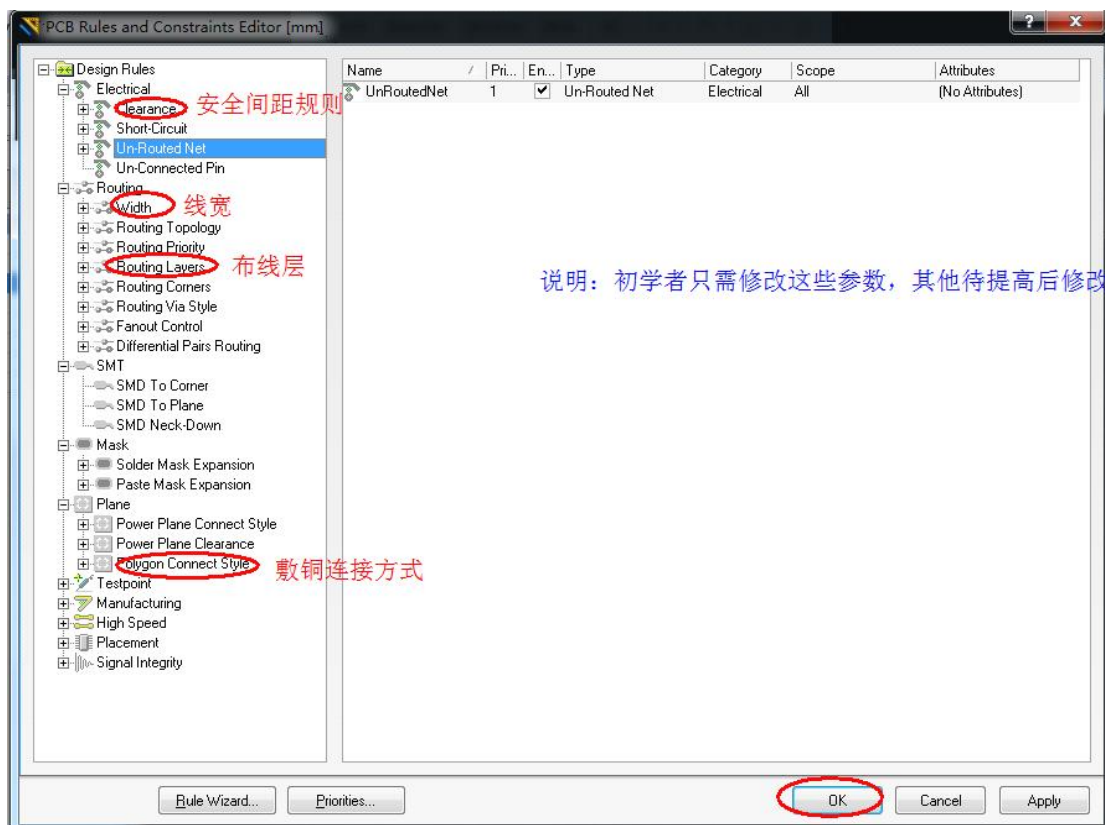


图2-22 规则设计对话框

其他规则可以保持默认设置。

(6) 在PCB板上布线

调整好元件位置，并设定好布线规则后即可进行PCB布线。执行【放置】/【走线】菜单命令，或者单击  图标，将鼠标移动到焊盘位置，此时光标会呈现十字形，在焊盘处单击鼠标左键即可开始连线（此时按下Tab 键可以修改导线属性）。连线完成后单击鼠标右键一次或者Esc 键结束该条导线放置，单击两次鼠标右键结束导线放置状态。

在布线过程中有时需要放置安装孔。如要求孔径2mm，焊盘大小3mm。执行菜单命令【放置】/【孔】，此时焊盘会粘在鼠标上，按下Tab 键进行修改属性和大小。如图2-23所示。



图2-23 设置安装孔属性

布线结束后，放置好导线的PCB如图2-24所示。

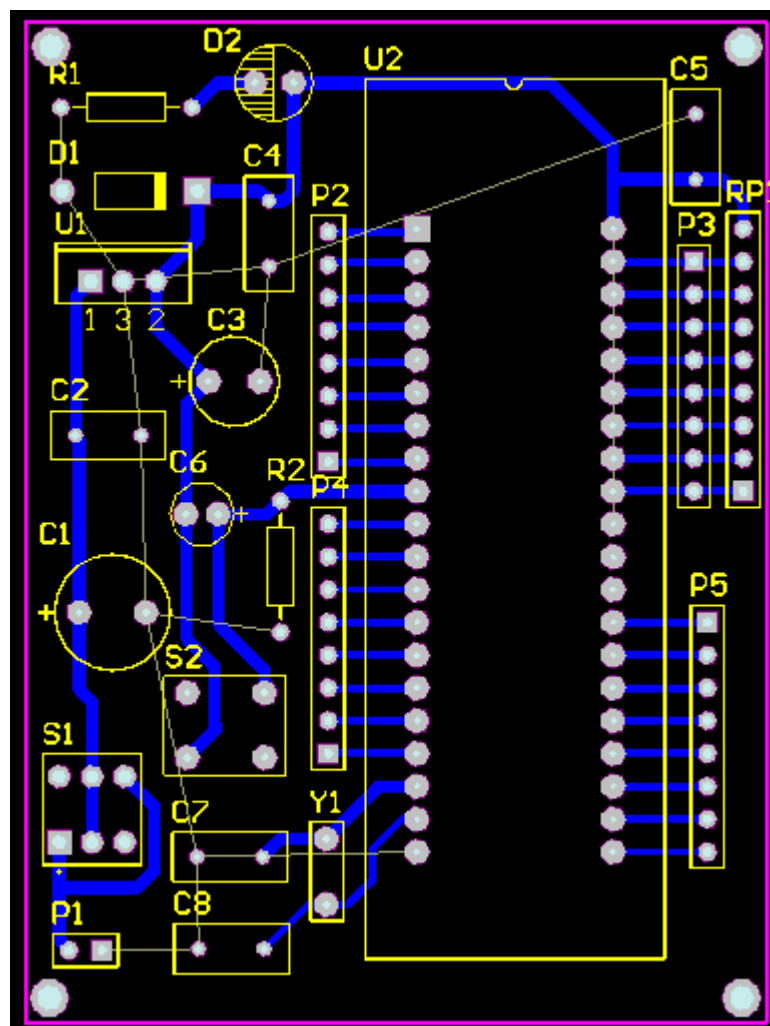


图2-24 绘制好导线的PCB

布线应遵循以下基本原则：

- 输入端导线与输出端导线应尽量避免平行布线，以免发生耦合。
- 在布线允许的情况下，导线的宽度尽量取大些，一般不低于 10mil（密耳，千分之一英寸，1mil=0.001=0.0254mm 英寸）。
- 导线的最小间距是由线间绝缘电阻和击穿电压决定的，在允许布线的范围内应尽量大些，一般不小于 12mil。
- 微处理器芯片的数据线和地址线应尽量平行布线。
- 布线时尽量少转弯，若需要转弯，一般取 45 度走向或圆弧形。在高频电路中，拐弯时不能取直角或锐角，以防止高频信号在导线拐弯时发生信号反射现象。
- 电源线和地线的宽度要大于信号线的宽度。

（7）对地线敷铜

地线的连接一般采用敷铜的方式连接。执行菜单命令【放置】/【多边形敷铜】，弹出敷铜选项对话框，设置网络为GND，敷铜层为Bottom Layer，如图2-25所示。



图2-25 敷铜选项

单击确定，此时光标会变成“十”字形状，将光标移动到电气约束线的一个角单击鼠标左键，移动鼠标到另一个角后在单击鼠标左键。直到在电路板上画成一个框，然后单击鼠标右键结束敷铜。放置好敷铜后的PCB 如图2-26所示。

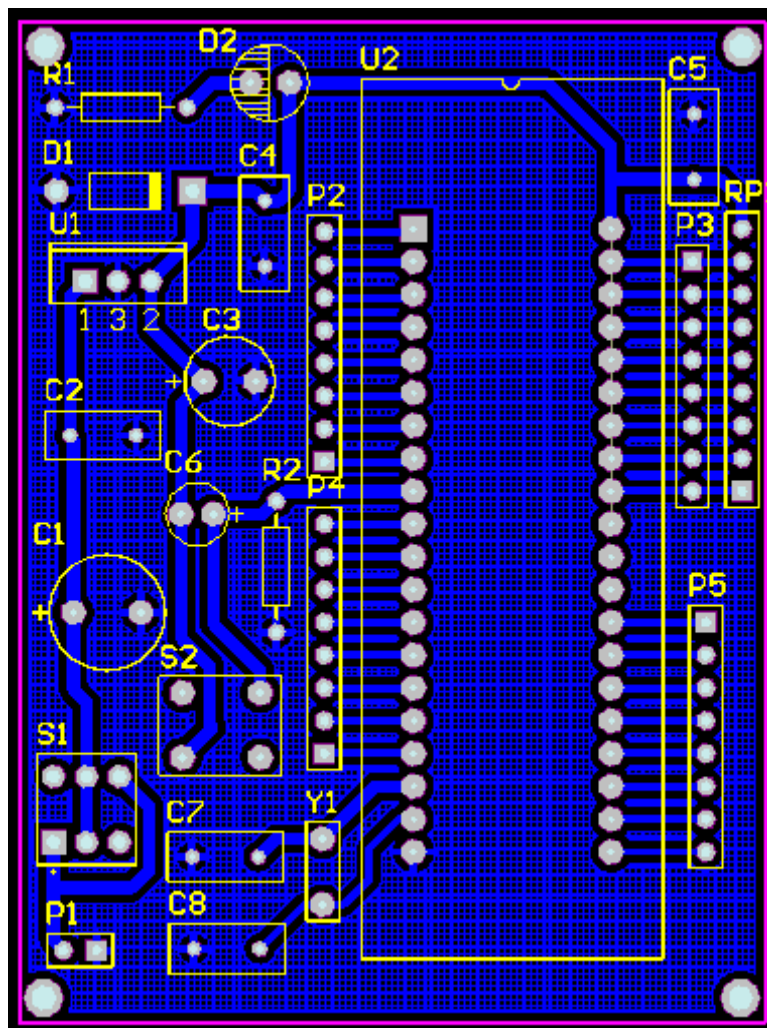


图2-26 放置好敷铜后的PCB

至此，完成了整个PCB电路板的绘制。下面就可以实际制作PCB板了。

2.1.3 PCB板的制作

实际制作PCB板是根据设计好的PCB文档进行的。一般有两种制作方式，一种是将设计好的PCB文档发到相应的生产厂家制作印制电路板，另一种方式是自己利用热转印技术制作印制电路板。

下面介绍后一种方式来自制PCB板。热转印法就是使用激光打印机，将设计好的PCB图形打印到热转印纸上，再将转印纸以适当的温度加热，转印纸上原先打印上去的图形就会受热融化，并转移到敷铜板上面，形成耐腐蚀的保护层。通过腐蚀液腐蚀后将设计好的电路留在敷铜板上面，从而得到PCB。

(1) 打印设置

为了制作电路板，还需要对PCB进行打印。执行菜单命令【File】/【Page Setup】，在弹出的对话框中设置成打印参数。如图2-27所示。

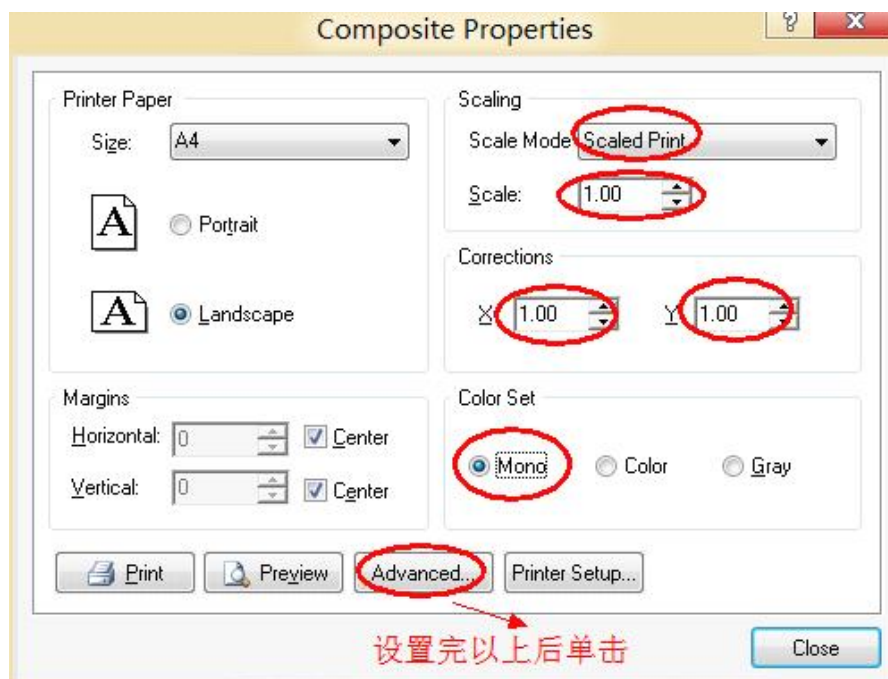


图2-27 打印设置

然后单击Advanced，进入PCB Printout Properties 对话框，将Top Overlay、Top Layer 删除。选中需要删除的层，单击鼠标右键，在弹出的对话框中选择Delete，单击Yes 即可删除。如果需要插入某个层，只需在空白的地方单击鼠标右键，选择Insert Layer，在弹出的对话框中找到需要插入的层即可。设置完成后如图2-28 所示。

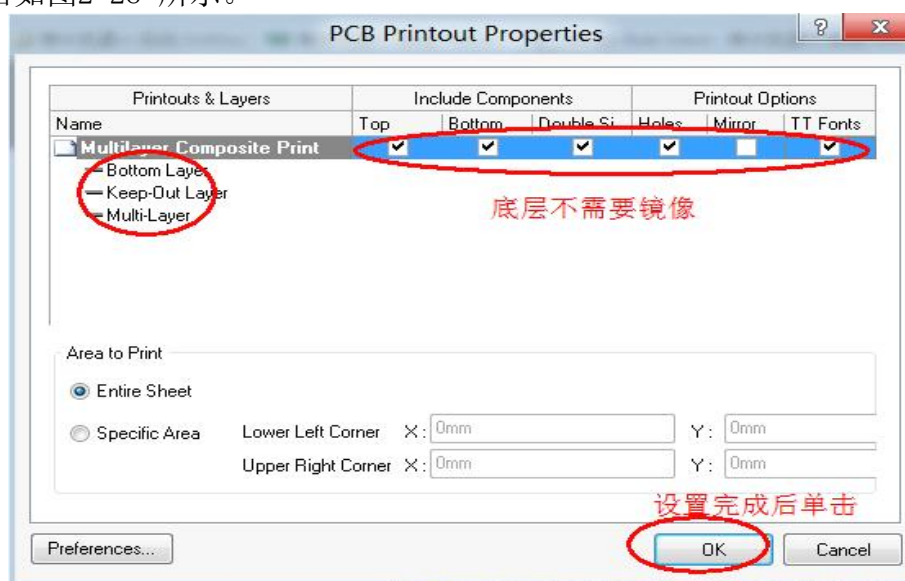


图2-28 打印属性修改

依次单击OK就设置好了。

(2) 热转印制作PCB板

准备材料：激光打印机一台、TPE-ZYJ 热转印机一台、剪板机一台、热转印纸一张、150W 左右台钻一台、敷铜板一块、钻花数颗、砂纸一块、工业酒精、松香水、腐蚀剂若干。

在打印机上放入热转印纸，执行菜单命令【File】/【Print】即可打印。打印的结果如图2-29所示。

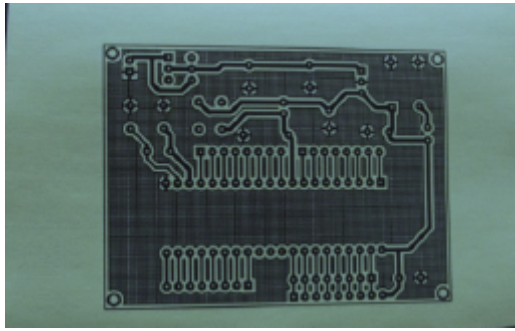


图2-29 打印到热转印纸上的PCB图

再将敷铜板根据实际电路大小裁剪出来，并用砂纸将敷铜板打磨干净后，用酒精进行清洗，晾干备用。将打印好的热转印纸的有图的那一面帖到打磨干净的敷铜板上，将敷铜板和热转印纸一同放到热转印机中进行转印，如图2-30所示。



图2-30 进行热转印

将转印好的敷铜板放到腐蚀液里进行腐蚀。如图2-31所示。



图2-31 腐蚀电路板

当电路板腐蚀好后，用酒精清洗，晾干后再进行打孔。

将顶层和顶层丝印层打印（需要镜像），后一同样的方法转印到电路板正面，此时在电路板上涂上一层松香水即完成整个电路板制作。

这种自制的方法制作的PCB板一般为单层板，如果设计的是双层或者多层板，都要拿到专门生产PCB的厂家生产。

2.1.4 原理图库的建立

在上面原理图的设计过程中，是从已有的元件库中选择元件。然而，并不是所有元件在库中都能找到，或者能找到但与实际元件引脚标号不一致，或者元件库里面的元件的符号大小或者引脚的距离与原理图不匹配等，都需要对找不到的库或者某些元件重新进行绘制，以完成电路的绘制。

原理图中的元件主要由标识图和引脚组成。标识图主要用来提示元件功能，无电气特性；引脚是元件的核心，有电气特性。建立原理图新元件的方法有两种：在原有的库中编辑修改；或者自己重新建立一个库文件。如图2-32所示是自己制作元

件的流程。具体操作可以查看AltiumDesigner 的操作说明或帮助信息。

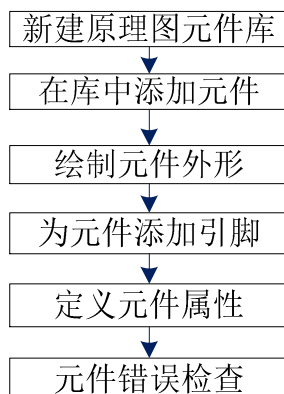


图2-32 自制原理图库并添加元件的流程

2.1.5 创建PCB元器件封装

由于新器件、和特殊器件的出现，某些器件导致在ProtelDXP集成库中没有办法找到，因此需要手工创建元器件的封装。

元器件封装只是元器件的外观和焊点的位置，纯粹的元器件封装只是空间的概念，因此不同的元器件可以共用一个封装，不同元器件也可以有不同的元件封装，所以在画PCB时，不仅需要知道元器件的名称，还要知道元器件的封装。

元件封装大体可以分为两大类：双列直插式（DIP）元件封装和贴片式（STM）元件封装。双列直插式元器件实物图和封装图如图2-33所示。

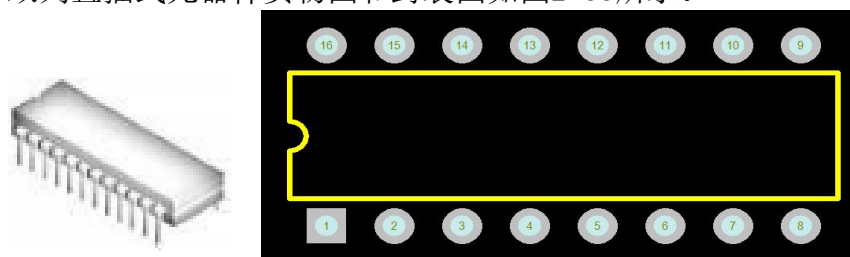


图2-33 双列直插式元器件实物图和封装图

贴片式元件实物图和封装图如图2-34所示。

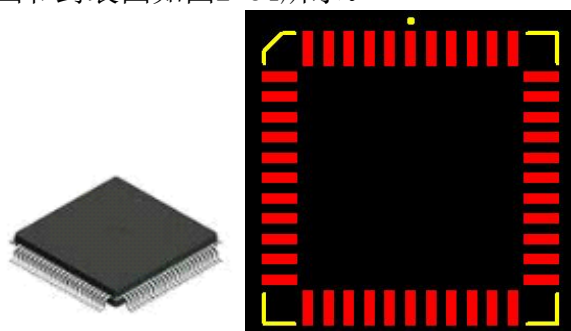


图2-34 贴片式元件实物图和封装图

元器件封装的编号一般为元器件类型加上焊点距离（焊点数）再加上元器件外形尺寸，可以根据元器件外形编号来判断元器件包装规格。比如AXAIL0.4 表示此元件的包装为轴状的，两焊点间的距离为400mil。DIP16 表示双列直插式的元器件封装，两排共16 个引脚。

创建封装库的大体流程如下如图2-35 所示。

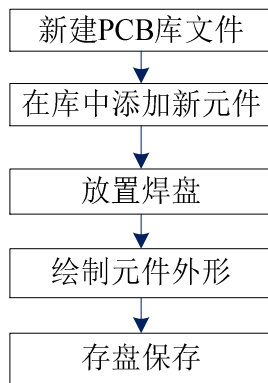


图2-35 创建封装库的流程

执行菜单命令【文件】/【新建】/【库】/【PCB 元件库】，就可以打开PCB 元器件封装库编辑器。下面有两种方式来创建封装库，一种是通过向导来完成，在PCB 元件库编辑器编辑状态下，执行菜单命令【工具】/【元件属性】，以后按照提示一步一步操作就可以了。

另一种方式是通过执行菜单命令来创建。在PCB 元件库编辑器编辑状态下，执行菜单命令【文件】/【保存为】，就可以将新建的库命名为自己的库文件。

在库中添加新的元器件的方法是，选择【PCB 元件库】标签，菜单【工具】列表中的【新的空元件】，绘制完毕后，执行菜单命令【工具】/【元件属性】对话框，在【名称】处输入要建立元件封装的名称；在【高度】处输入元件的实际高度后确认。

放置焊盘是通过执行菜单命令【放置】/【孔】（或者单击绘图工具栏的按钮）来实现的。此时光标会变成十字形状，且光标的中间会粘浮着一个焊盘，移动到合适的位置（一般将1 号焊盘放置在原点[0,0]上），单击鼠标左键将其定位。

绘制元件外形是通过工作层面切换到顶层丝印层，（即【TOP-Overlay】层），执行菜单命令【放置】/【线】，此时光标会变为十字形状，移动鼠标指针到合适的位置，单击鼠标左键确定元件封装外形轮廓的起点，到一定的位置再单击鼠标左键即可放置一条轮廓，以同样的方法只到画完位置。执行菜单命令【放置】/【圆弧】可放置圆弧。

最后需要设定器件的参考原点。执行菜单命令【编辑】/【设置参考】/【1 脚】，元器件的参考点一般选择1 脚。

在绘制焊盘或者元件外形时，可以不断的重新设定原点的位置以方便画图。操作为：【编辑】/【设置参考】/【中心】，此时移动鼠标到所需要的新原点处单击鼠标左键即可。详细的内容可以参考Altium Designer 15的参考资料。

2.2 keil软件安装及使用

2.2.1 keil μ vision5基本介绍

Keil是美国Keil Software公司出品的51系列兼容单片机C语言软件开发系统，与汇编相比，C语言在功能上、结构性、可读性、可维护性上有明显的优势，因而易学易用。Keil提供了包括C编译器、宏汇编、连接器、库管理和一个功能强大的仿真调试器等在内的完整开发方案，通过一个集成开发环境（ μ Vision）将这些部分组合在一起。运行Keil软件需要WINXP或WIN10等操作系统。如果你使用C语言编程，那么Keil几乎就是你的不二之选。

Keil生成的目标代码效率非常之高，多数语句生成的汇编代码很紧凑，容易理解。在开发大型软件时更能体现高级语言的优势。

2.2.2 Keil μ Vision5的安装

(1) 所需软件: Keil μ Vision5安装包、STC-ISP v6.85E。

(2) 安装Keil μ Vision5, 安装方式很简单, 选好安装路径, 一路【next】, 直到【Finish】。安装完成后不要打开, 继续以下操作。

(3) 安装STM32芯片包

图解:

第一步: 获取keil5的安装包, 到keil的官网下载

<https://www.keil.com/download/product/>, 如图2-36所示。

Download Products

Select a product from the list below to download the latest version.



Keil products use a [License Management](#) system - without a current license the product runs as a Lite/Evaluation edition with a few [Limitations](#).

图2-36 获取安装包

第二步: 双击安装包, 开始安装, next。接下来演示安装MDK5.33, 大家根据需要下载安装包并安装。如图2-37所示。

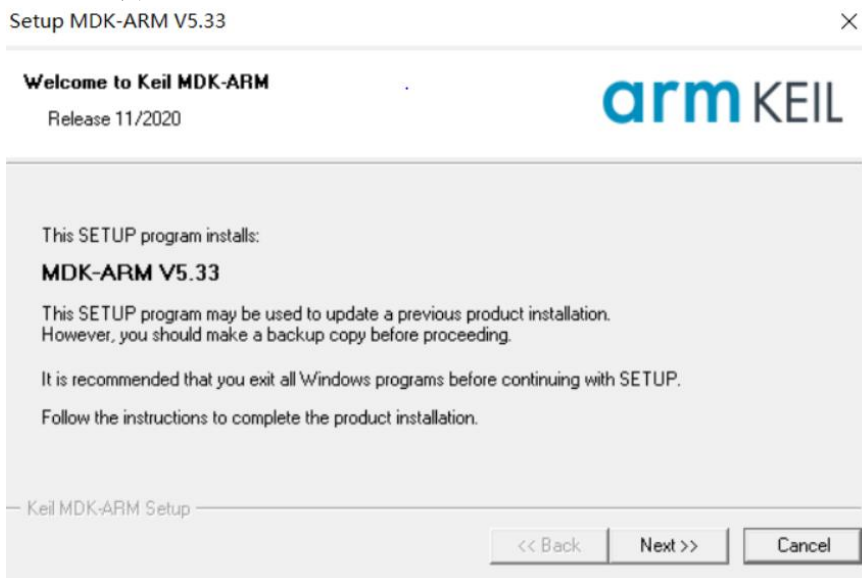




图2-37 安装keil并同意协议

第三步：选择安装路径，如图2-38所示。

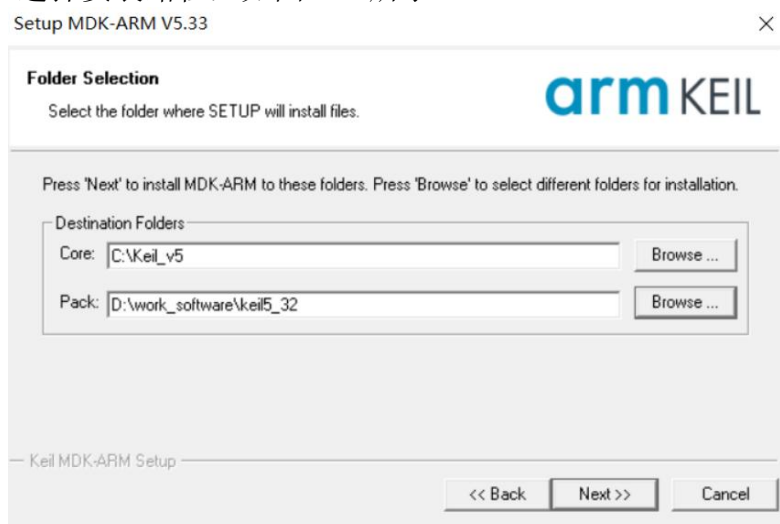


图2-38 选择路径

第四步：填写个人信息，可以随意填写（可全部填空格），但是必须填，如图2-39所示。

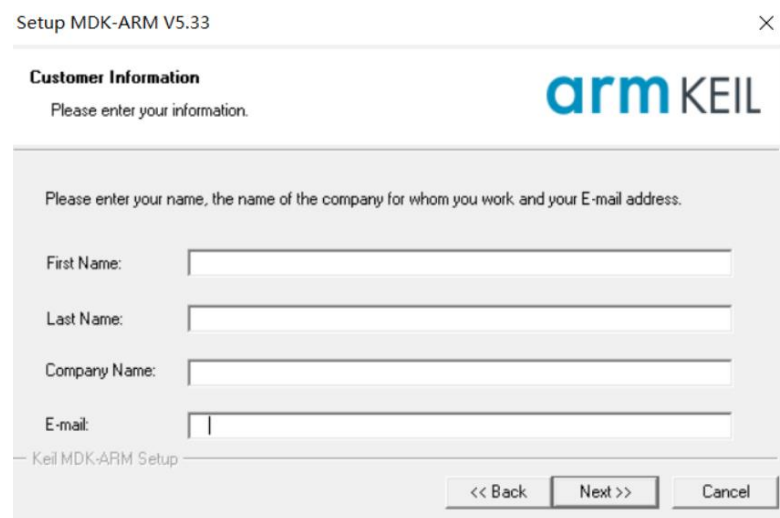


图2-39 填写信息

第五步：安装keil5结束。如图2-40所示

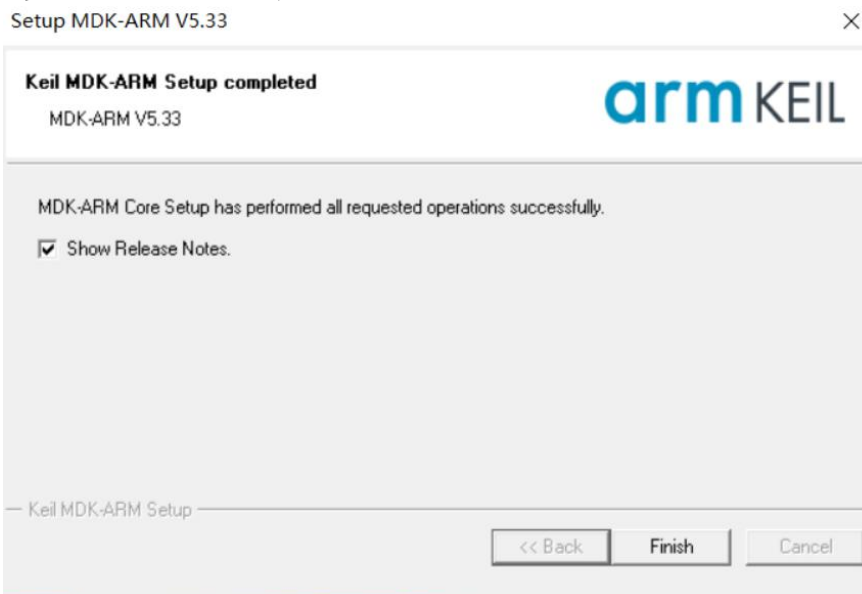


图2-40 结束安装

第六步：下载STM32芯片包，可以直接去keil的官网下载：

<http://www.keil.com/dd2/pack/>,

在官网可以STM32F1、STM32F4、STM32F7这三个系列的包，具体下载哪个根据使用的型号下载即可，我们使用的是STM32F103RCT6是属于STM32F1系列的。如图2-41所示



图2-40 下载STM32F1

第七步：双击安装包即可，选择路径和KEIL5一样的安装路径，安装成功之后，在KEIL5的Pack Installer中就可以看到我们安装的包。以后我们新建工程的时候，就有单片机的型号可选。

2.2.3 第一个STM32程序

(1) 如图2-54-1所示，单机【文件】菜单中的【New】菜单项。新建文件后窗口界面如图2-54-2所示。

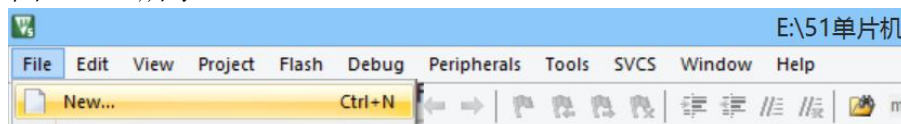


图2-54-1新建文件

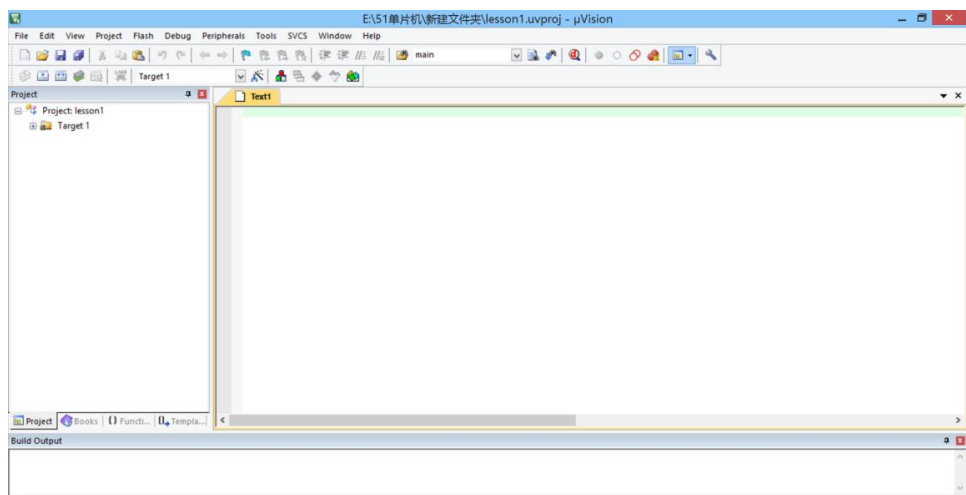


图2-54-2新建完文件后的窗口界面

新建文件后就等待用户输入程序。输入下面的一段C语言代码。

```
#include <stdio.h>
void main(void)
{
    while(1)
    {
        printf("Hello World!\n");//显示HelloWorld
    }
}
```

头文件定义`#include< reg52.h>`也可以使用`#include<reg51.h>`，其具体的区别在于：`reg52.h`定义了更多的地址空间。可以在Keil安装文件夹中，找到相应的文件。头文件中定义了很多符号常量，包括管脚和特殊寄存器，如`sbit P1_0 = P1^0`；定义了P1的第1个管脚。

“//”后面表示的是注释，在第一个程序中，暂时不考虑具体含义，以后我们会使用和解释。另外，Keil的文本编辑器对中文的支持欠佳，所以读者在写程序时，可以考虑用英文对程序代码进行注释。此处使用中文注释仅是为了使读者阅读方便。单击图标，将文件保存到已建立的文件夹下，窗口界面如图2-55所示。

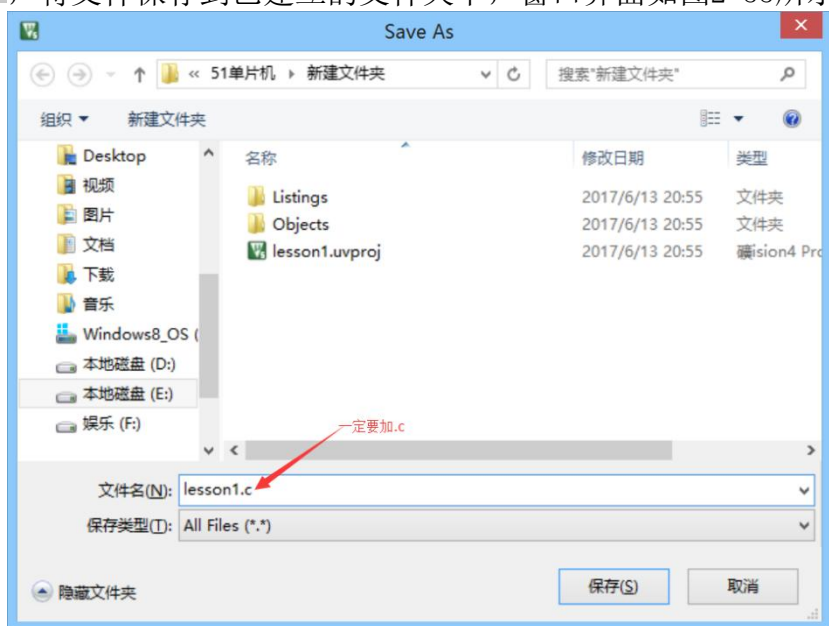


图2-55保存文件

(2) 此时的新建文件与我们刚刚建立的工程还没有直接的联系，回到编辑界面，单击【目标1】前面的十号，然后在【Source Group1】选项上单击右键，弹出如图2-56所示菜单。然后选择【添加到文件组‘源组1’】菜单项，对话框如图2-57所示。这样就把新建的文件加入到建立的工程中，如图2-58所示。

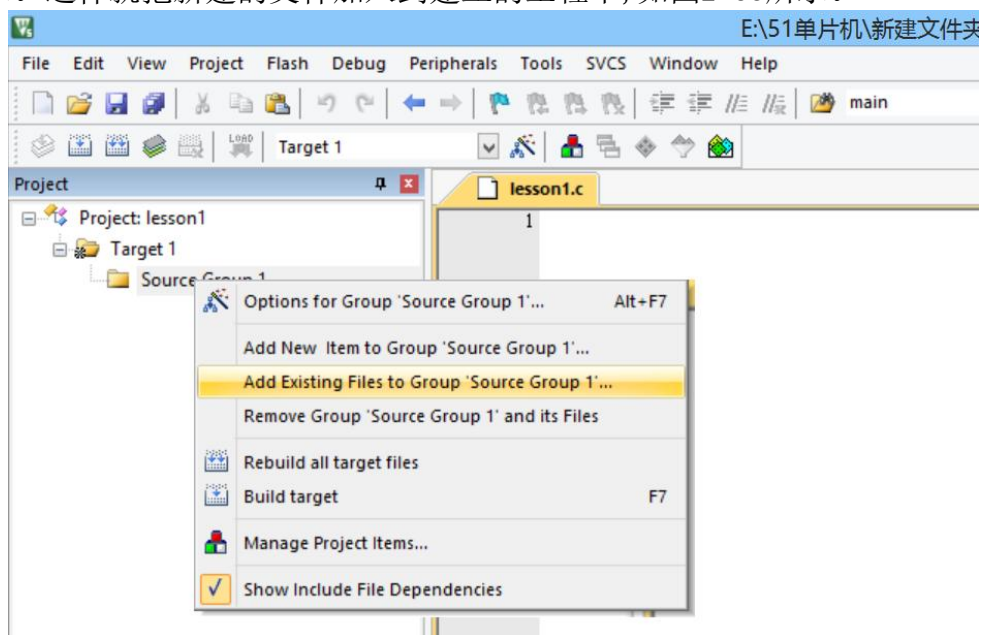


图2-56将文件加入工程

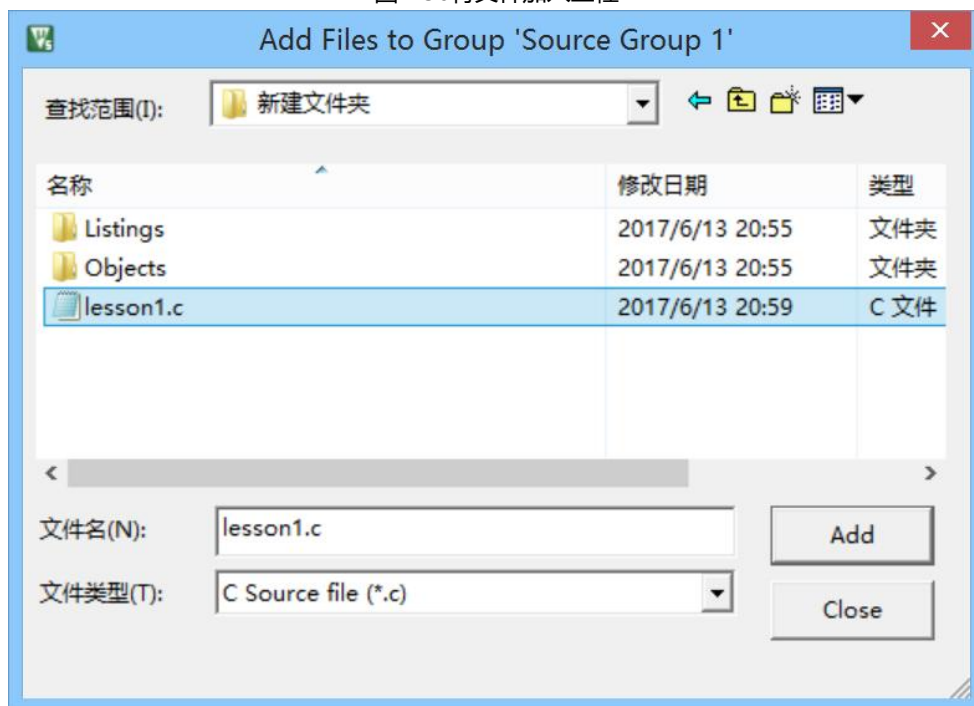


图2-57选中文件后的对话框

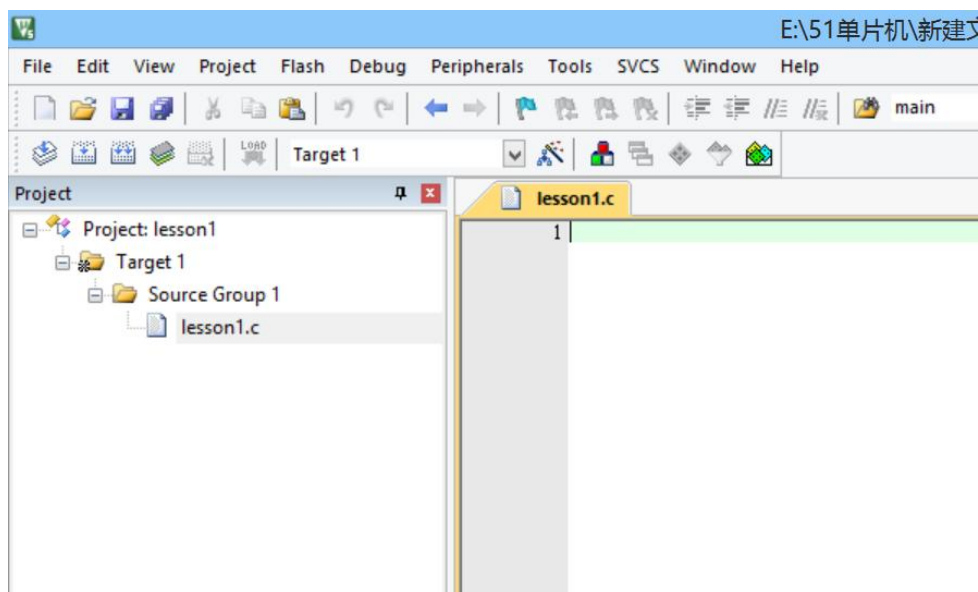


图2-58 将文件加入工程后的屏幕窗口

通过以上步骤我们已经完成了在Keil编译环境下建立一个工程。

(3) 在编写好程序，且C程序文件已经加到项目中了，就可以进行编译运行了。上面这个项目我们只是用做学习新建程序项目和编译运行仿真的基本方法，所以使用软件默认的编译设置，它不会生成用于芯片烧写的hex文件。我们先来看图2-59中1、2、3都是编译按钮。不同的是1是用于编译单个文件；2是编译当前项目，如果先前编译过一次之后，文件没有作过编辑改动，这时再点击是不会再次重新编译的；3是重新编译，每点击一次均会再次编译链接一次，不管程序是否有改动。在3右边的是停止编译按钮，只有点击了前三个中的任一个，停止按钮才会生效。这些按钮也有对应的菜单项，在project菜单项中，5是这三个按钮在菜单中相应的命令，效果和按钮是一样的。这个项目只有一个文件，你按1、2、3中的任一个都可以编译。在message框4中可以看到编译的错误信息和使用的系统资源情况等，以后我们要查错就靠它了。3的上面有一个小放大镜的按钮，这是开启/关闭调试模式的按钮，它在菜单“Debug” -> “Start/Stop Debug Session”中，快捷键为Ctrl+F5。

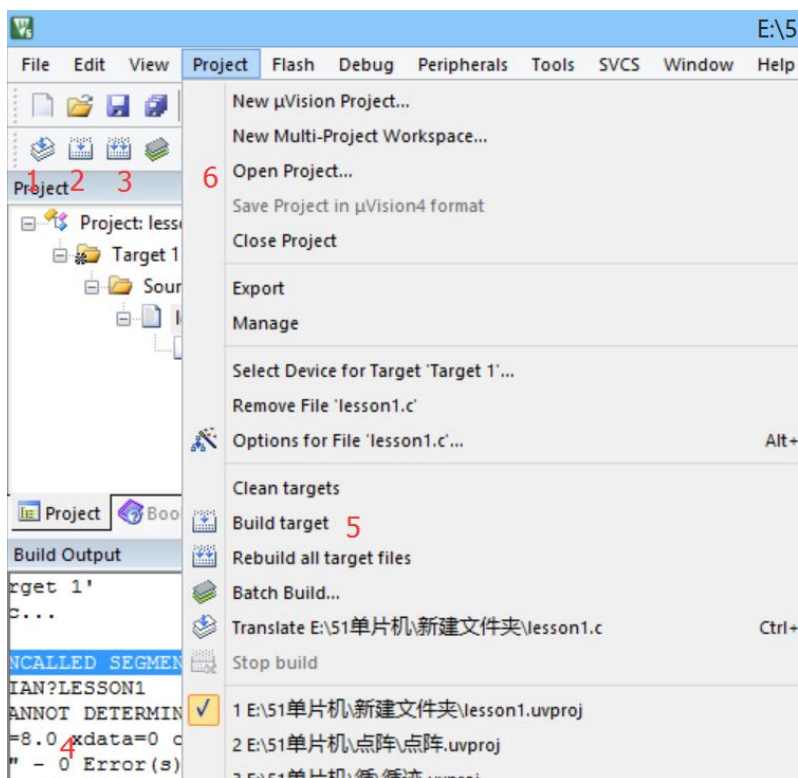


图2-59编译运行程序按钮

(4) 调试。进入调试模式，软件窗口样式大致如图2-60所示。图中1为运行，当程序处于停止状态时才有效；2为停止，程序处于运行状态时才有效；3是复位，模拟芯片的复位，程序回到最开头处执行。按4可以打开5中的串行调试窗口，这个窗口可以看到从51芯片的串行口输入输出的字符，上面的第一个项目也正是在这里看运行结果。这些按钮在菜单中也有。首先按4打开串行调试窗口，再按运行键，这时就可以看到串行调试窗口中不断的打印“Hello World! ”。这样就完成了第一个C项目。要停止程序运行回到文件编辑模式中，就要先按停止按钮再按开启/关闭调试模式按钮。然后就可以进行关闭Keil等相关操作了。

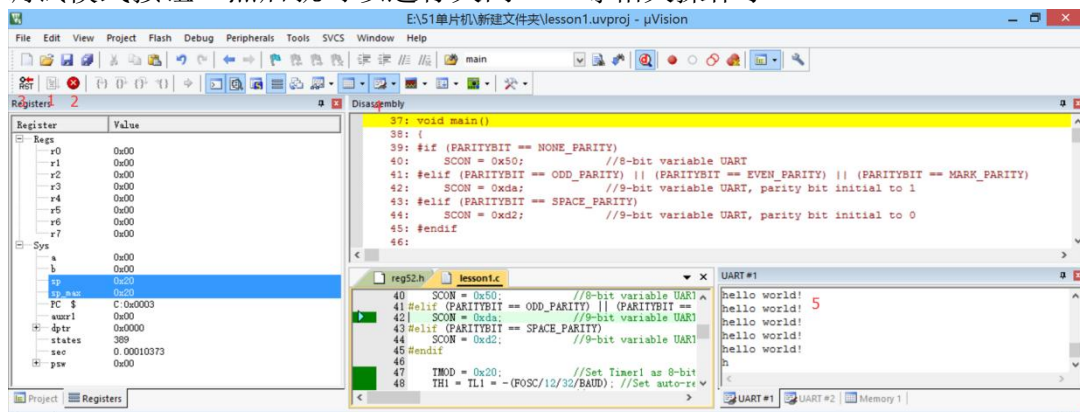


图2-60调试运行程序

(5) 生成hex文件。如果一个文件需要下载到单片机上运行，就需要用Keil µVision5来编译生成用于烧写芯片的hex文件。Hex文件格式是Intel公司提出的用来保存单片机或其他处理器的目标程序代码的文件格式。一般的编程器都支持这种格式。先打开刚做的第一个项目，打开它的所在目录，找到lesson1.Uv2的文件就可以打开先前的项目了。然后右击图2-58中的项目文件夹Target1，弹出项目功能菜单，选“Options for Target' Target1' ”，弹出项目选项设置窗口；也可以

先选中项目文件夹图标，在Project菜单中也有一样的菜单可选。打开项目选项窗口，转到“Output”选项页，如图2-61所示，在这里可以设置编译输出的路径，设置编译输出生成的文件名，以及是否要创建hex文件，选中它就可以输出hex文件到指定的路径中。

现在选中它，重新编译一次，很快在编译信息窗口中就显示hex文件创建到指定的路径中了，如图2-62所示。这样就可用编程器所附带的软件去读取并烧到芯片了。

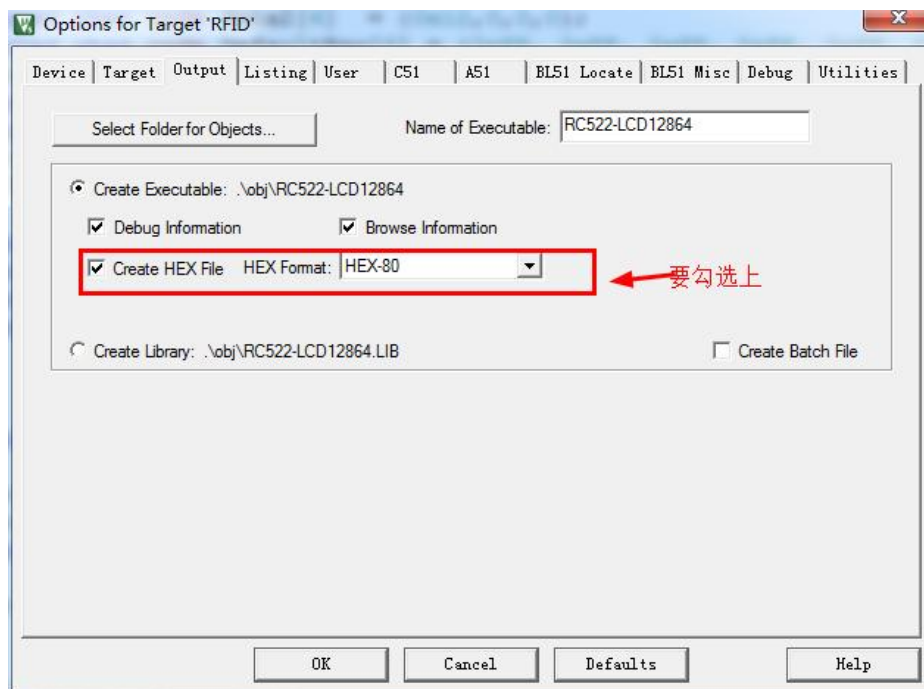


图2-61选择程序的编译输出

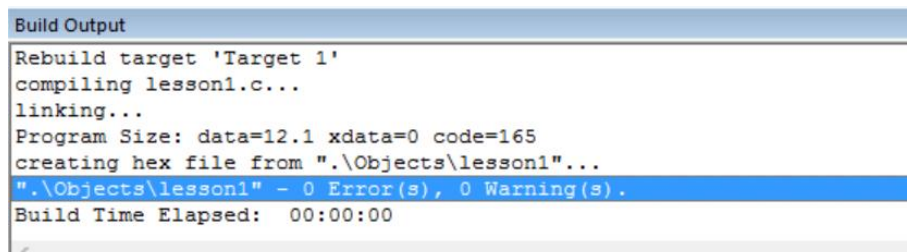


图2-62 编译信息窗口

2.3 ST-LINK 下载软件的使用

ST-Link是用于STM8和STM32微控制器的在线调试器和编程器，也是大家口中的下载器。ST-Link具有SWIM、JTAG / SWD等通信接口，用于与STM8或STM32微控制器进行通信（各版本有差异）。

三种接口说明：

- SWIM: Single Wire Interface Module, 单线接口模块。
- JTAG: Joint Test Action Group, 联合测试工作组，是一种国际标准测试协议。
- SWD: Serial Wire Debugging, 串行调试接口。

STM8使用USB全速接口与**ST Visual Develop（STVD集成开发环境）或ST Visual Program（STVP编程软件）**进行通信。STM32使用USB全速接口与ATOLLIC®, IAR™, KEIL™或TASKING™集成开发环境进行通信。同时，与STVP、STM32 ST-LINK

utility、STM32CubeProg编程软件进行通信

安装ST-link v2驱动程序，安装过程中的选项默认就好包括路径，一直点NEXT即可。

安装完成后插上STLINK打开设备管理器就可以看到STLINK的设备驱动了。成功完成ST-LINK 安装与连接后：

(1) 打开项目配置选项卡，点击Debug,配置仿真器为ST-LINK DEBUG,点击setting。

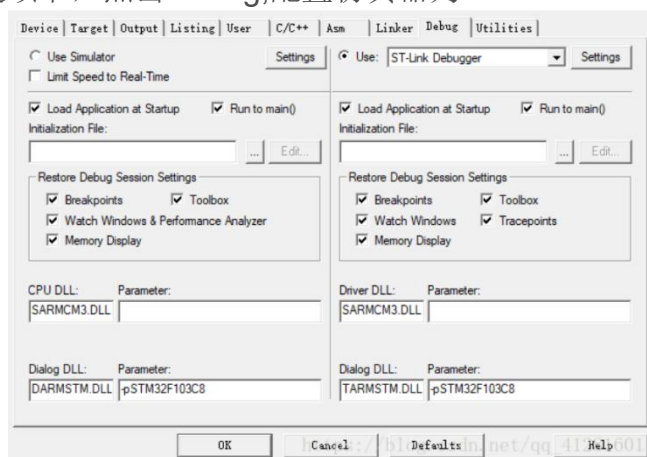


图2-63 工程配置

(2) 正常情况下会显示：

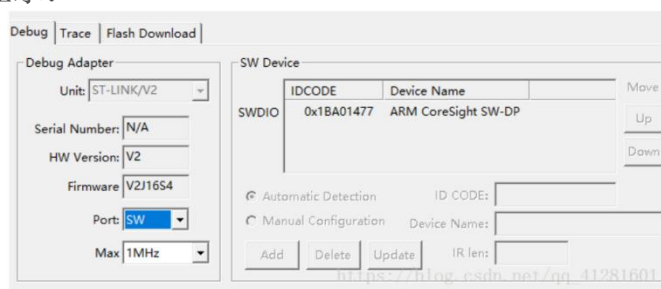


图2-64 仿真器信息与配置

选择SW，右侧SW/JATG Device 下会出现id号，这种情况下直接点确定即可。

(3) 还有可能出现：

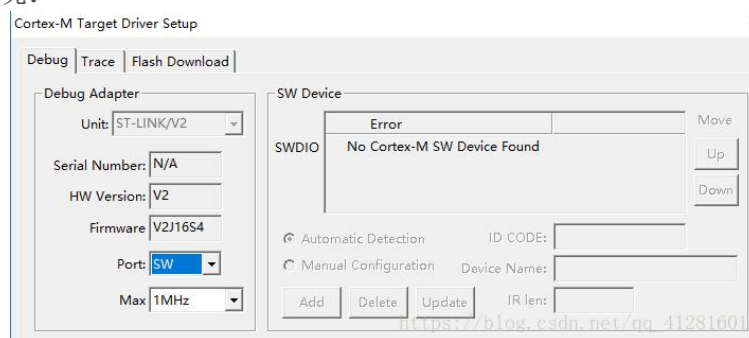


图2-65 仿真器无法连接实验板信息

这表示ST-LINK连接无误，而查找不到目标板，请检查插线和主板。

(4) 正常后点击Utilities选项卡：

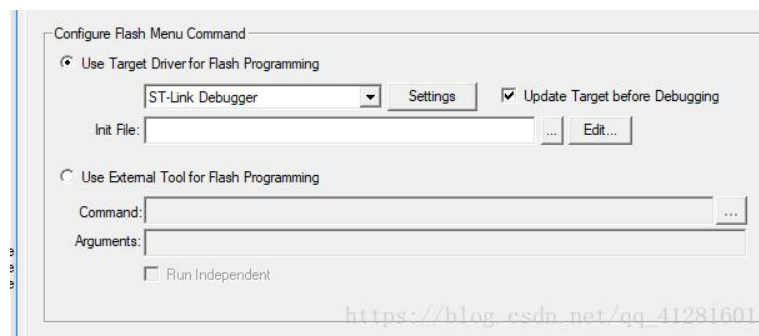


图2-66 下载工具选择

同样设置ST-LINK DEBUG 点击Setting

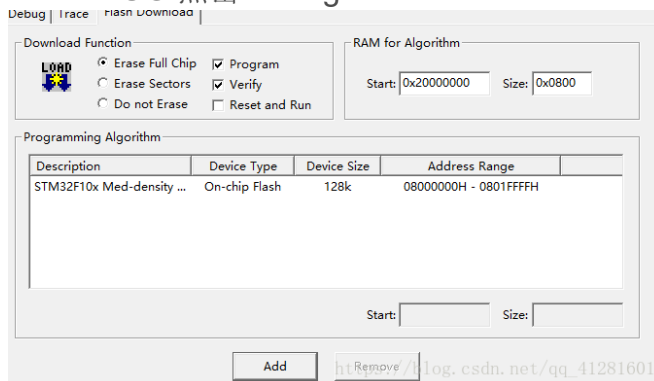


图2-67 Flash下载算法配置

(5) 选择你的开发板型号。

至此，ST-link 调试可以在 MDK 下下载程序。

2.4 思考与练习

- (1) 熟悉嵌入式系统开发过程中用到的各种软件的使用。
- (2) 制作嵌入式系统的硬件开发板的步骤有哪些？
- (3) 通过在 Keil 中编程和 FlyMcu 下载程序，理解开发环境、运行环境、交叉编译等概念。

第3章 实验硬件平台介绍

嵌入式实验硬件平台主要由：MiniSTM32 开发板、STLINK 调试下载器（仿真器）、miniIO 扩展板等部分组成。

3.1 MiniSTM32开发板

ALIENTEK MiniSTM32 开发板是一款迷你型的 STM32F103 开发板，小巧而不小气，简约而不简单。该开发板自推出以来，深得广大 STM32 学习者喜爱，总销量超过 1.6W 套。目前最新版本为 V3，最新 MiniSTM32 开发板资源图如图 3.1 所示。

这款 MiniSTM32 V3 开发板，设计精良，结构小巧！板子的设计充分考虑了成本与功能这两个矛盾面，再结合实际使用的经验及 STM32 的特点，最终确定了这样的设计。总体来说是该有的都有，不该有的坚决不要，可有可无的选择性价比最高的留下。



图 3.1 MiniSTM32 开发板实物图

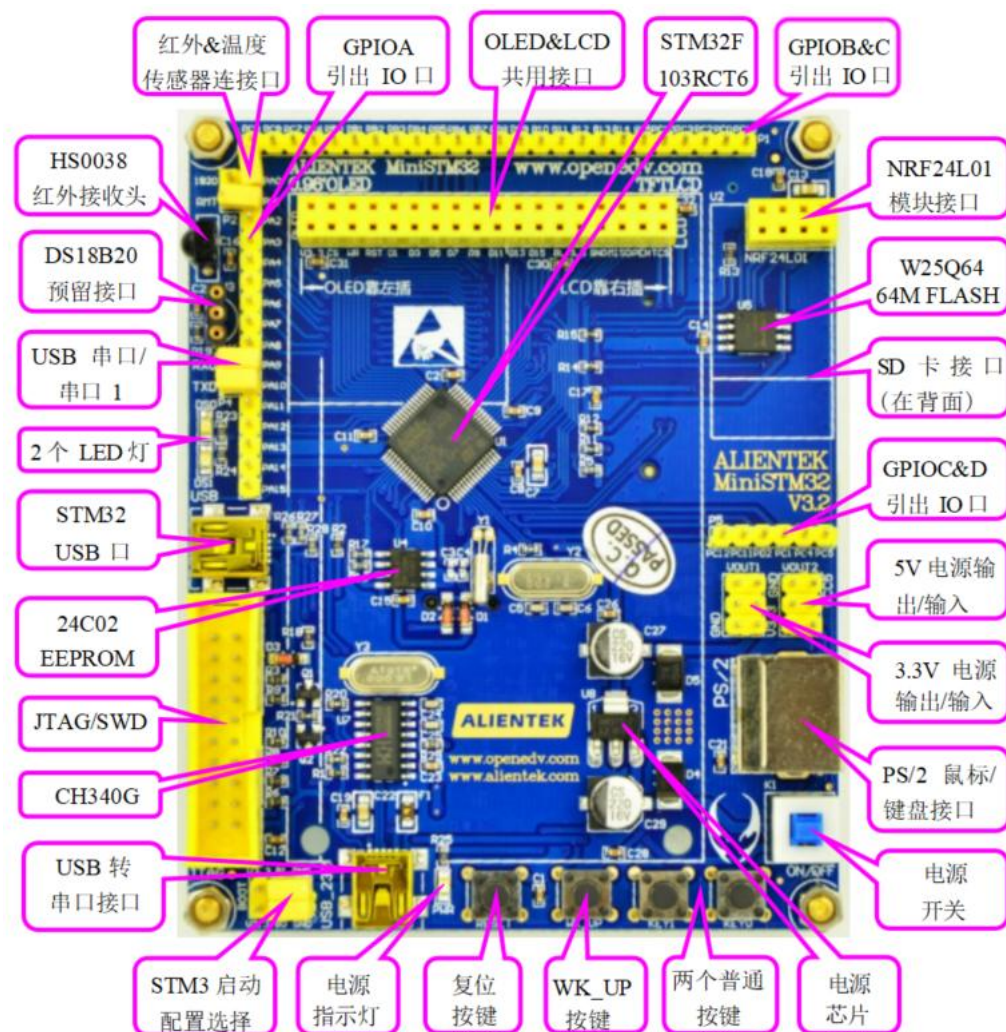


图 3.2 MiniSTM32 开发板资源图

ALIENTEK MiniSTM32 开发板板载资源如下：

- ◆ CPU: STM32F103RCT6 , LQFP64 , FLASH:256K, SRAM: 48K;
- ◆ 1 个标准的 JTAG/SWD 调试下载口
- ◆ 1 个电源指示灯 (蓝色)
- ◆ 2 个状态指示灯 (DS0: 红色, DS1: 绿色)
- ◆ 1 个红外接收头, 配备一款小巧的红外遥控器
- ◆ 1 个 IIC 接口的 EEPROM 芯片, 24C02, 容量 256 字节
- ◆ 1 个 SPI FLASH 芯片, W25Q64, 容量为 8M 字节 (即 64M bit)
- ◆ 1 个 DS18B20/DS1820 温度传感器预留接口
- ◆ 1 个标准的 2.4/2.8/3.5/4.3/7 寸 LCD 接口, 支持触摸屏
- ◆ 1 个 OLED 模块接口 (与 LCD 接口部分共用)
- ◆ 1 个 USB 串口接口, 可用于程序下载和代码调试
- ◆ 1 个 USB SLAVE 接口, 用于 USB 通信
- ◆ 1 个 SD 卡接口
- ◆ 1 个 PS/2 接口, 可外接鼠标、键盘

- ◆ 1 组 5V 电源供应/接入口
- ◆ 1 组 3.3V 电源供应/接入口
- ◆ 1 个启动模式选择配置接口
- ◆ 1 个 2.4G 无线通信接口
- ◆ 1 个 RTC 后备电池座，并带电池
- ◆ 1 个复位按钮，可用于复位 MCU 和 LCD
- ◆ 3 个功能按钮，其中 WK_UP 兼具唤醒功能
- ◆ 1 个电源开关，控制整个板的电源
- ◆ 3.3V 与 5V 电源 TVS 保护，有效防止烧坏芯片
- ◆ 独创的一键下载功能
- ◆ 除晶振占用的 IO 口外，其余所有 IO 口全部引出，其中 GPIOA 和 GPIOB 按顺序引出

从上面的板载资源可以看出，MiniSTM32 开发板的板载资源是很丰富的，加上灵活的设计，让您的开发变得更加简单。

ALIENTEK MiniSTM32 V3 开发板的特点包括：

- 1) 小巧。整个板子尺寸为 8cm*10cm*2cm(包括液晶，但不计算铜柱的高度)。
- 2) 灵活。板上除晶振外的所有的 IO 口全部引出，特别还有 GPIOA 和 GPIOB 的 IO 口是按顺序引出的，可以极大的方便大家扩展及使用，另外板载独特的**一键下载**功能，避免了频繁设置 B0、B1 带来的麻烦，直接在电脑上一键下载。
- 3) 资源丰富。板载十多种外设及接口，可以充分挖掘 STM32 的潜质。
- 4) 质量过硬。沉金 PCB+全新优质元器件+定制全铜镀金排针/排座+电源 TVS 保护，坚若磐石。
- 5) 人性化设计。各个接口都有丝印标注，使用起来一目了然；接口位置设计安排合理，方便顺手。资源搭配合理，物尽其用。

这里我们首先详细介绍 MiniSTM32 开发板的各个部分(图 1.3 中的标注部分)的硬件资源，我们将按逆时针的顺序依次介绍。

(1) HS0038 红外信号接收头

这是开发板板载的标准 38K 红外信号接收头，用于接收红外遥控器的信号，有了它，就可以用红外遥控器控制这款开发板了，也可以用来做红外解码等其他相关实验。ALIENTEK MiniSTM32 开发板标配了一个红外遥控器，其外观如图 1.4 所示：

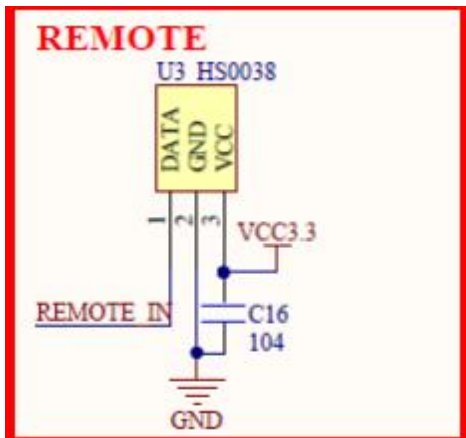


图 3.3 红外遥控原理图

(2) DS18B20 预留接口

这是开发板预留的数字温度传感器 DS18B20/DS1820 接口，采用的是镀金的圆孔母座。当要做 DS18B20 实验的时候，直接插到这个母座上即可，很方便。

DS18B20 需自备，插上就可以用的。同样 ALIENTEK 提供了 DS18B20 的相关例程。

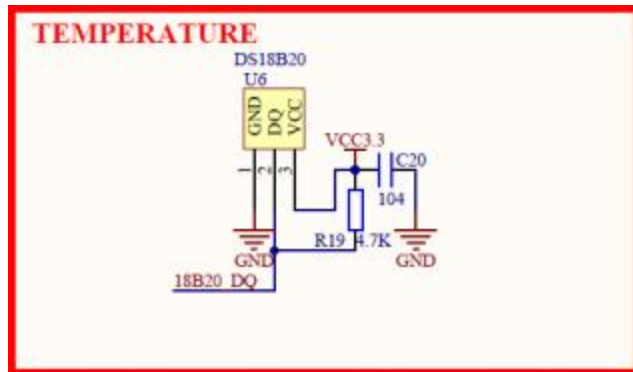
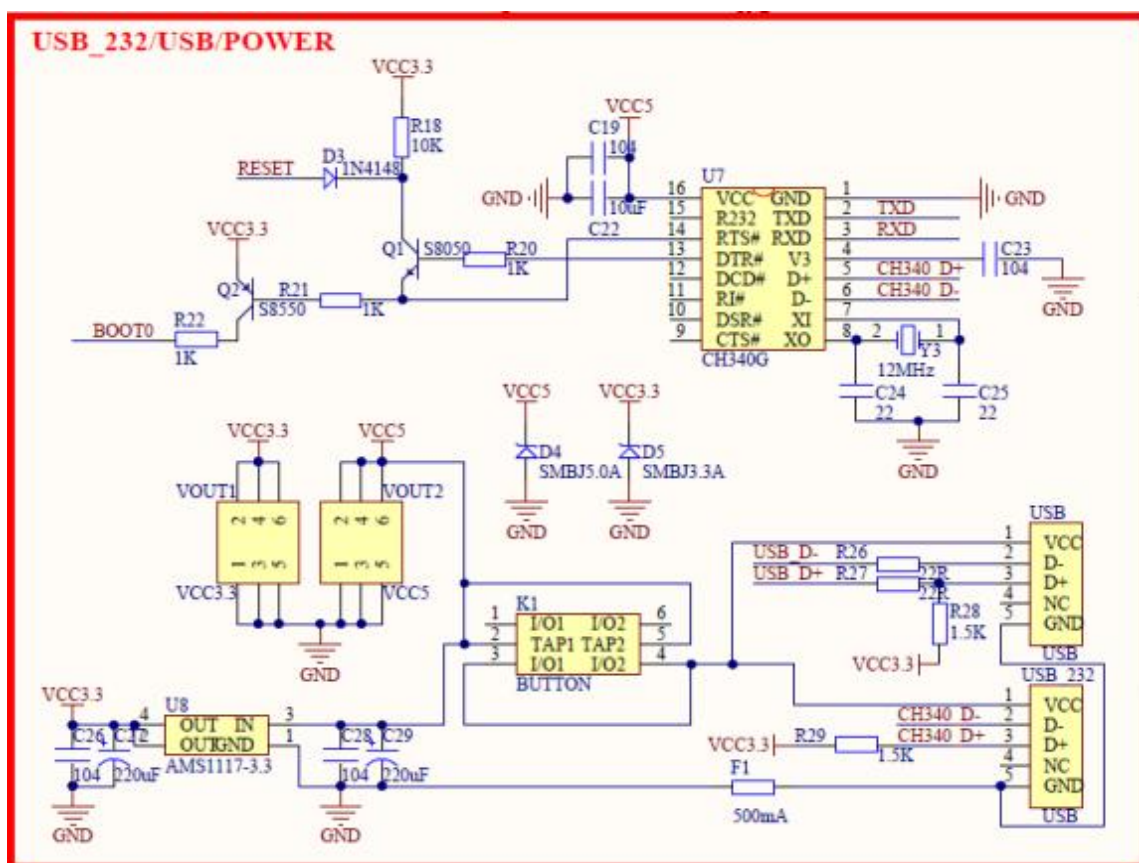


图 3.4 DS1820 接口图

(3) USB、串口、ISP 及电源

这是 USB 转串口 (P4) 同 STM32F103RCT6 的串口 1 进行连接的接口，标号 RXD 和 TXD 是 USB 转串口的 2 个数据口 (对 CH340G 来说)，而 PA9 (TXD) 和 PA10 (RXD) 则是 STM32 的串口 1 的两个数据口 (复用功能下)。他们通过跳线帽对接，就可以和连接在一起了，从而实现 STM32 的程序下载以及串口通信。



这里三个部分一起介绍，ALIENTEK MiniSTM32 开发板板载了 USB 串口，并且由 USB 提供电源，使得我们只需要一根 USB 线就可以使用 ALIENTEK MiniSTM32 开发板了，包括串口、下载代码、供电、串口通信 3 位一体。

开发板的供电部分还引出了 5V (VOUT2) 和 3.3V (VOUT1) 的排针，可以用来为外部设备提供电源或者从外部引入电源，这在很多时候是非常有用的，有时候你突然要一个 3.3V 的电源，但找半天就是没这样的电源，而我们的板子则可直接向外部提供 3.3V 电源，有了它，你就可以给外部设备提供 3.3V、5V 电源了。注意电流不能太大哦！

开发板的 USB 接口 (USB) 通过独立的 Mini USB 头引出, 不和 USB 转串口 (USB 232) 共用, 这样不但可以同时使用, 还可以给系统提供更大的电流。

图中的 Q1 和 Q2 外加几个电阻和一个二极管就构成了开发板的一键下载电路，此电路通过 RST 和 DTR 信号来控制 BOOT0 和 RESET 信号，从而实现一键下载的功能。很多朋友问一键 下载的原理，这里和大家讲一下，先说一个前提： DTR_N 和 RTS N 的输出和 DTR/RTS 的设 置是相反的。必须先记下这个前提。

一键下载电路的具体实现过程：首先， `mcuisp` 控制 `DTR` 输出低电平，则 `DTR_N` 输出高， 然后 `RTS` 置高，则 `RTS_N` 输出低， 这样 `Q2` 导通了，`BOOT0` 被拉高， 即实现设置 `BOOT0` 为 1， 同时 `Q1` 也会导通， `STM32` 的复位脚被拉低，实现复位。然后，延时 100ms 后， `mcuisp` 控制 `DTR` 为高电平，则 `DTR_N` 输

出低电平，RTS维持高电平，则RTS_N继续为低电平，此时STM32的复位引脚，由于Q1不再导通，变为高电平，STM32结束复位，但是BOOT0还是维持为1，从而进入ISP模式，接着mcuisp就可以开始连接STM32，下载代码了，从而实现一键下载。

另外，此部分还有一个开关K1，用来控制整个系统的供电，如果断开则整个系统的3.3V部分都将断电。而5V部分的电源还是开启的。图中F1为可恢复保险丝，用于保护USB。

图中的D4和D5这两个TVS管，用于保护开发板，防止外部高压脉冲/静电损坏开发板上的元器件，让大家用的更加放心。

(4) 两个LED灯

这是开发板板载的两个LED灯，它们在开发板上的标号为：DS0和DS1。DS0是红色的，DS1是绿色的，主要是方便大家识别。一般的应用2个LED足够了，在调试代码的时候，使用LED来指示程序状态，是非常不错的一个辅助调试方法。ALIENTEK开发板几乎每个实例都使用了LED来指示程序的运行状态。

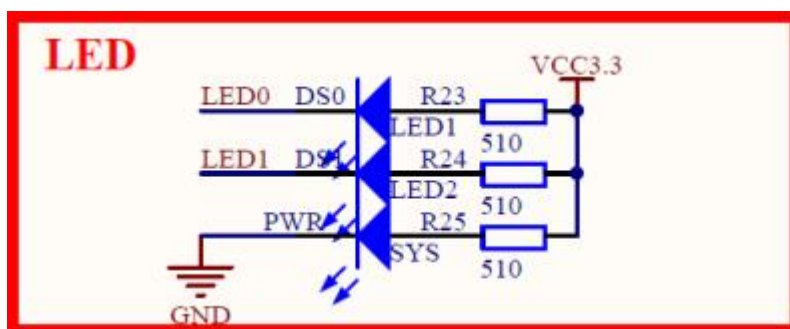


图 3.6 LED原理图

其中PWR是开发板电源指示灯，为蓝色。LED0和LED1分别接在PA8和PD2上，PA8还可以通过TIM1的通道1的PWM输出来控制DS0的亮度。为了方便大家判断，我们选择了DS0为红色，DS1为绿色的LED灯。

(5) 24C02 EEPROM

这是开发板板载的2Kbit（256个字节）EEPROM，型号为：24C02，用于掉电数据保存。因为STM32内部没有EEPROM，所以开发板外扩了24C02，用于存储重要数据，也可以用来做IIC实验，及其他应用。该芯片直接挂在STM32的IO口上。其原理图如下：

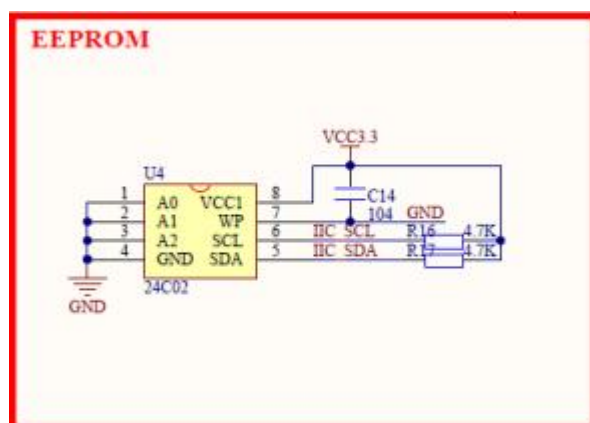


图 3.7 EEPROM 原理图

(6) JTAG/SWD

这是开发板板载的 20 针标准 JTAG 调试口，在开发板上的标号为：JTAG。该 JTAG 口直接可以和 ULINK 或者 JLINK 或者 STLINK 等调试器(仿真器)连接，同时由于 STM32 支持 SWD 调试，这个 JTAG 口也可以用 SWD 模式来连接。

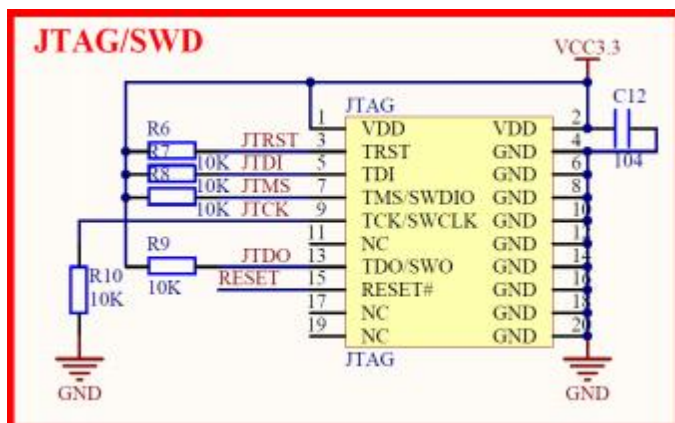


图 3.8 JTAG 原理图

用标准的 JTAG 调试，需要占用 5 个 IO 口，很多时候，可能造成 IO 口不够用，而用 SWD 则只需要 2 个 IO 口，大大节约了 IO 数量，但他们达到的效果是一样的。所以调试下载的时候，**强烈建议使用 SWD 模式!!!**

(7) USB 转串口

这是开发板板载的 USB 转串口芯片，型号为：CH340G。有了这个芯片，我们就可以实现 USB 转串口，从而能实现 USB 下载代码，串口通信等，原理如图 1.5。

这是开发板板载的另外一个 MiniUSB 头(USB_232)，用于 USB 连接 CH340G 芯片，从而实现 USB 转串口，所以串口下载代码的时候，USB 一定是要接在这个口上的。同时，此 MiniUSB 接头也是开发板电源的主要提供口。

(8) STM32 启动配置选择

这是开发板板载的启动模式选择开关，在开发板上的标号为：BOOT。STM32 有 BOOT0 (B0)和 BOOT1 (B1) 两个启动选择引脚，用于选择复位后 STM32 的启

动模式，默认 B0，B1 都是连接在 GND 的。作为开发板，这两个是必须的。在开发板上，我们通过跳线帽选择 STM32 的启动模式。

表 3.1 BOOT0、BOOT1 启动模式表

BOOT0	BOOT1	启动模式	说明
0	X	用户闪存存储器	用户闪存存储器，也就是FLASH启动
1	0	系统存储器	系统存储器启动，用于串口下载
1	1	SRAM启动	SRAM启动，用于在SRAM中调试代码

(9) 电源指示灯

这是开发板板载的一颗蓝色的 LED，用于指示电源状态，在开发板上的标号为：PWR。在电源开启的时候(通过板上的电源开关控制)，该灯会亮，否则不亮。通过这个 LED，可以判断开发板的上电情况，开发板必须在上电的条件下(电源灯亮)，才可以正常使用。原理如图 1.6 所示。

(10) 复位按键

这是开发板板载的复位按键，用于复位 STM32，同时还具有复位液晶的功能，因为液晶模块的复位引脚和 STM32 的复位引脚是连接在一起的，此按键在开发板上的标号为：RESET。当按下该键的时候，STM32 和液晶一并被复位。

(11) WK_UP 按键

这是开发板板载的一个唤醒按键，该按键连接到 STM32 的 WAKE_UP (PA0) 引脚，可用于待机模式下的唤醒，在不使用唤醒功能的时候，也可以做为普通按键输入使用，此按键在开发板上的标号为：WK_UP。

(12) 两个普通按键

这是开发板板载的两个普通按键，可以用于人机交互的输入，这两个按键是直接连接在 STM32 的 IO 口上的，两个按键在开发板上的标号分别为：KEY0、KEY1。

(13) 电源芯片

这是开发板的电源稳压芯片，型号为：AMS1117-3.3。因为 STM32 是 3.3V 供电的，所以我们需要将 USB 的 5V 电压转换为 3.3V，这个芯片就是将 5V 转换为 3.3V 的线性稳压芯片。

(14) 电源开关

这是开发板板载的电源开关，此开关在开发板上的标号为：K1，并标有 ON/OFF 丝印。该开关用于控制整个开发板的供电，如果切断，则整个开发板都将断电，电源指示灯(PWR)会随着此开关的状态而亮灭。

(15) PS/2 接口

这是开发板板载的一个标准 PS/2 接头，用于连接电脑鼠标和键盘等 PS/2 设备，在开发板上的标号为：PS/2。通过该接口，我们仅需要 2 个 IO 口，就可以扩展

一个键盘，所以大家不必要对板上只有3个按键而感到担忧。ALIENTEK 提供了标准的鼠标驱动例程，方便大家学习PS/2 协议。原理图如下：

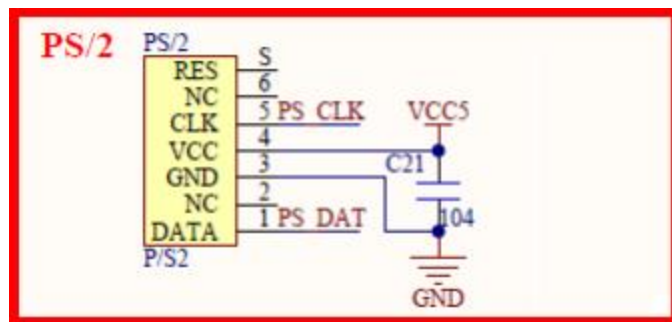


图 3.9 PS/2 接口原理图

PS_CLK 和 PS_DAT 分别接 PA15 和 PC5，PS/2 的信号线是需要外部提供上拉电阻的，这里 PS_CLK 与 JTCK 共用一个上拉电阻，而 PS_DAT 则需要使用 STM32 内部的上拉电阻了，在使用的时候，需要注意，记得开启 PC5 的上拉电阻。

(16) 3.3V 电源输出/输入

这是开发板板载的一组 3.3V 电源输入输出排针(2*3)，在开发板上的标号为：VOUT1。该排针用于给外部提供 3.3V 的电源，也可以用于从外部取 3.3V 的电源给板子供电。大家在实验的时候可能经常会为没有 3.3V 电源而苦恼不已，ALIENTEK 充分考虑到了大家需求，有了这组 3.3V 排针，您就可以很方便的拥有一个简单的 3.3V 电源(最大电流不能超过 500ma)，另外板载了 3.3V TVS 管，能有效吸收高压脉冲，防止外接设备/电源可能对开发板造成的损坏。

(17) 5V 电源输出/输入

这是开发板板载的一组 5V 电源输入输出排针(2*3)，在开发板上的标号为：VOUT2，用于给外部提供 5V 的电源，也可以用于从外部取 5V 的电源给板子供电。同样大家在实验的时候可能经常会为没有 5V 电源而苦恼不已，有了 ALIENTEK MiniSTM32 开发板，您就可以很方便的拥有一个简单的 5V 电源(最大电流不能超过 500ma)，另外板载了 5V TVS 管，能有效吸收高压脉冲，防止外接设备/电源可能对开发板造成的损坏。

(18) GPIOC&D 引出 IO 口

这是开发板板载的 GPIOC 与 GPIOD 等 IO 口的引出排针，在开发板上的标号为：P5。我们可以用这些引出的 IO 口来连接外部模块，方便大家外接其他模块。

(19) SD 卡接口

这是开发板板载的 SD 卡接口。SD 卡作最常见的存储设备之一，是很多数码设备的存储媒介，比如数码相框、数码相机、MP5、手机、平板电脑等。我们的开发板自带了 SD 卡接口(大卡)，可以用于 SD 卡实验，方便大家学习 SD 卡，TF 卡通过转接座也可以很方便的接到我们的开发板上。

有了它，开发板就相当于拥有了一个大容量的外部存储器，不但可以用来提

供数据，也可以用来存储数据，使得这款开发板可以完成更多的功能。

这里要特别说明一下：该 SD 卡卡座是在开发板的背面！

(20) W25Q64 64M FLASH

这是开发板板载的一颗 FLASH 芯片，型号为 W25Q64。这颗芯片的容量为 64M bit，也就是 8M 字节。有了这颗芯片，我们就可以存储一些不常修改的数据到里面，比如字库等，从而大大节省对 STM32 内部 FLASH 的占用。关于该芯片的使用见 SPI 实验这个章节。

(21) NRF24L01 模块接口

这是开发板板载的 NRF24L01 模块接口，只要插入 NRF24L01 无线模块，我们便可以实现无线通信功能。但是提醒大家：NRF24L01 通信，至少需要 2 个模块和 2 个开发板同时工作才可以。如果只有 1 个开发板或 1 个模块，是没法实现无线通信的。

(22) GPIOB&C 引出 IO 口

这是开发板板载的 GPIOB 与 GPIOC 的引出口，该接口用于将 STM32 的 GPIOB 和部分的 GPIOC 引出，方便大家的使用，在开发板上的标号为：P1。这里 GPIOB 全部使用顺序引出的方式，尤其适合外部总线型器件的接入。

(23) STM32F103RCT6

这是开发板的核心芯片，从 3.0 版本开始，升级到 RCT6，详细型号为：STM32F103RCT6。该芯片具有 48K SRAM、256K FLASH、2 个 16 位基本定时器、4 个 16 位通用定时器、2 个 16 位高级定时器、2 个 DMA 控制器、3 个 SPI、2 个 IIC、5 个串口、1 个 USB、1 个 CAN、3 个 12 位 ADC、1 个 12 位 DAC、1 个 SDIO 接口、51 个通用 IO 口。

(24) OLED&LCD 共用接口

这是 ALIENTEK 开发板的特色设计，一个接口，兼容两种模块。在此部分，LCD 的部分 IO 和 OLED 的 IO 共用，具体请参看后面的开发板原理图。这样我们一个接口既可以接 LCD 模块，又可以接 OLED 模块。OLED 模块使用的是 ALIENTEK 的 OLED 模块，分辨率为 128*64，模块大小为 2.6cm*2.7cm。而 LCD 模块，则可以使用 ALIENTEK 全系列 TFTLCD 模块，包括：2.4 寸(电阻屏，240*320)、2.8 寸(电阻屏，240*320)、3.5 寸(电阻屏，320*480)、4.3 寸(电容屏，800*480)、7 寸(电容屏，800*480)。

这里特别提醒：在使用的时候，OLED 模块是靠左插的，而 LCD 模块，则是靠右插，在后续章节我们将分别介绍 OLED 模块和 LCD 模块的使用。

(25) GPIOA 引出 IO 口

这是开发板 GPIOA 的引出排针，在开发板上的标号为 P3。ALIENTEK 开发板将所有的 IO 口(除了 2 个晶振占用的 4 个 IO 口)都用排针引出来了，而且

GPIOA 和 GPIOB 是按顺序引出的。按顺序引出，在很多时候能方便大家的实验和测试，比如外接带并行控制的器件，有了并行引出的排针，那么就可以很方便的通过这些排针连接到外部设备了。

将开发板的 IO 口全部排针引出，大家就可以用来外接其他模块等，不论调试还是功能扩展都是很方便的。

(26) 红外&温度传感器接口

这是开发板板载的红外与温度传感器的连接接口，开发板虽然自带了红外接收头和 DS18B20 的接口，但是并没有将这两个器件直接挂在 IO 口上，而是通过跳线帽来连接，以防止在不使用这两器件的时候，他们对 IO 口的干扰，当然我们也可以用跳线，把 DS18B20 和红外遥控接收模块接到其他电路上使用。

为了让大家更快更好的使用MiniSTM32-V3开发板，这里特地将MiniSTM32-V3开发板主芯片：STM32F103RCT6的IO资源分配做了一个总表，以便大家查阅。MiniSTM32-V3的IO引脚分配总表如表3.2所示：

表3.2 MiniSTM32 V3 IO 资源分配表

MiniSTM32 V3 IO 资源分配表					
引脚	GPIO	连接资源		独立	连接关系说明
14	PA0	WK_UP	18B20_DQ	Y	1, 按键 KEY_UP 2, 可以做待机唤醒脚(WKUP) 3, 可以接 DS18B20 传感器接口(P2 设置)
15	PA1	NRF_IRQ	REMOTE_IN	Y	1, NRF24L01 接口 IRQ 信号 2, 接 HS0038 红外接收头(P2 设置)
16	PA2	F_CS		N	W25Q64 的片选信号
17	PA3	SD_CS		N	SD 卡接口的片选脚
20	PA4	NRF_CE		Y	NRF24L01 接口的 CE 信号
21	PA5	SPI1_SCK		N	W25Q64、SD 卡和 NRF24L01 接口的 SCK 信号
22	PA6	SPI1_MISO		N	W25Q64、SD 卡和 NRF24L01 接口的 MISO 信号
23	PA7	SPI1_MOSI		N	W25Q64、SD 卡和 NRF24L01 接口的 MOSI 信号
41	PA8	LED0		N	接 DS0 LED 灯(红色)
42	PA9	U1_TXD		Y	串口 1 TX 脚，默认连接 CH340 的 RX(P4 设置)
43	PA10	U1_RXD		Y	串口 1 RX 脚，默认连接 CH340 的 TX(P4 设置)
44	PA11	USB_D-		Y	接 USB D-引脚
45	PA12	USB_D+		Y	接 USB D+引脚
46	PA13	JTMS	SWDIO	N	JTAG/SWD 仿真接口, 没接任何外设 注意：如要做普通 IO，需先禁止 JTAG&SWD
49	PA14	JTCK	SWDCLK	N	JTAG/SWD 仿真接口, 没接任何外设 注意：如要做普通 IO，需先禁止 JTAG&SWD

50	PA15	JTDI	PS_CLK KEY1	N	1, JTAG 仿真口(JTDI) 2, PS/2 接口的 CLK 信号 3, 接按键 KEY1
26	PB0	LCD_D0		Y	TFTLCD 接口的 D0 脚
27	PB1	LCD_D1		Y	TFTLCD 接口的 D1 脚
28	PB2	BOOT1	LCD_D2	N	1, BOOT1, 启动选择配置引脚(仅上电时用) 2, TFTLCD 接口的 D2 脚
55	PB3	JTD0	LCD_D3	N	1, JTAG 仿真口(JTD0) 2, TFTLCD 接口的 D3 脚(使用时, 需先禁止 JTAG, 才可以当普通 I/O 使用)
56	PB4	JTRST	LCD_D4	N	1, JTAG 仿真口(JTRST) 2, TFTLCD 接口的 D4 脚(使用时, 需先禁止 JTAG, 才可以当普通 I/O 使用)
57	PB5	LCD_D5		Y	TFTLCD 接口的 D5 脚
58	PB6	LCD_D6		Y	TFTLCD 接口的 D6 脚
59	PB7	LCD_D7		Y	TFTLCD 接口的 D7 脚
61	PB8	LCD_D8		Y	TFTLCD 接口的 D8 脚
62	PB9	LCD_D9		Y	TFTLCD 接口的 D9 脚
29	PB10	LCD_D10		Y	TFTLCD 接口的 D10 脚
30	PB11	LCD_D11		Y	TFTLCD 接口的 D11 脚
33	PB12	LCD_D12		Y	TFTLCD 接口的 D12 脚
34	PB13	LCD_D13		Y	TFTLCD 接口的 D13 脚
35	PB14	LCD_D14		Y	TFTLCD 接口的 D14 脚
36	PB15	LCD_D15		Y	TFTLCD 接口的 D15 脚
8	PC0	T_SCK		Y	TFTLCD 接口触摸屏 SCK 信号
9	PC1	T_PEN		Y	TFTLCD 接口触摸屏 PEN 信号
10	PC2	T_MISO		Y	TFTLCD 接口触摸屏 MISO 信号
11	PC3	T_MOSI		Y	TFTLCD 接口触摸屏 MOSI 信号
24	PC4	NRF_CS		Y	NRF24L01 接口的 CS 信号
25	PC5	PS_DAT	KEY0	Y	1, PS/2 接口的 DAT 信号 2, 接按键 KEY0
37	PC6	LCD_RD		Y	TFTLCD 接口的 RD 脚
38	PC7	LCD_WR		Y	TFTLCD 接口的 WR 脚
39	PC8	LCD_RS		Y	TFTLCD 接口的 RS 脚
40	PC9	LCD_CS		Y	TFTLCD 接口的 CS 脚
51	PC10	LCD_BL		Y	TFTLCD 接口的 BL 脚
52	PC11	IIC_SDA		N	接 24C02 的 SDA
53	PC12	IIC_SCL		N	接 24C02 的 SCL
2	PC13	T_CS		Y	TFTLCD 接口触摸屏 CS 信号
3	PC14		RTC 晶振	N	接 32.768K 晶振, 不可用做 I/O
4	PC15		RTC 晶振	N	接 32.768K 晶振, 不可用做 I/O
5	PD0		HSE 晶振	N	接 HSE 晶振, 不可用做 I/O
6	PD1		HSE 晶振	N	接 HSE 晶振, 不可用做 I/O

54	PD2	LED1	N	接 DS1 LED 灯 (绿色)
----	-----	------	---	------------------

3.2 miniIO扩展板

miniIO扩展板具有如下模块：

- ❖ 电磁式蜂鸣器
- ❖ 继电器
- ❖ MAX7219驱动8位数码管
- ❖ 交流电压检测
- ❖ 交流电流检测
- ❖ 漏电电流检测

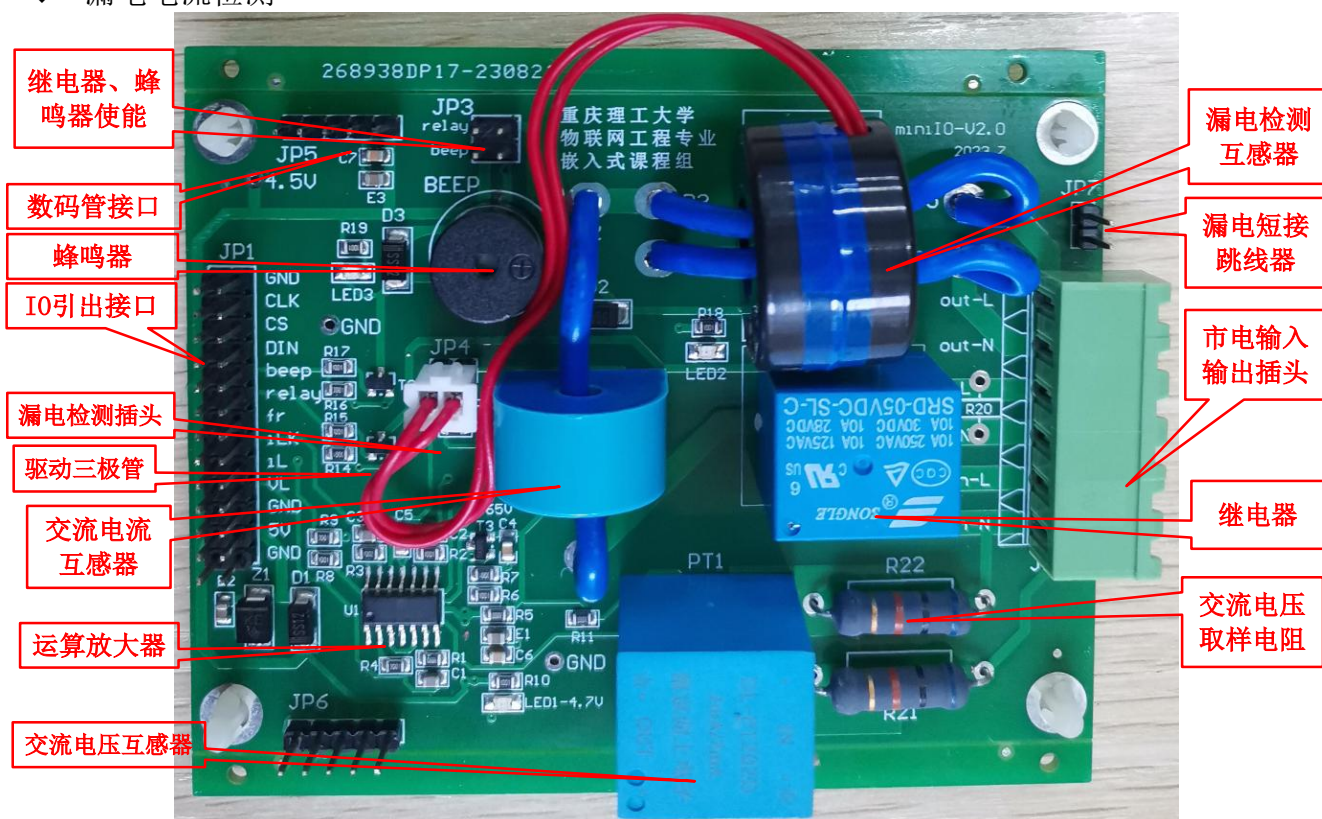


图 3.10 miniIO 扩展接口板资源图

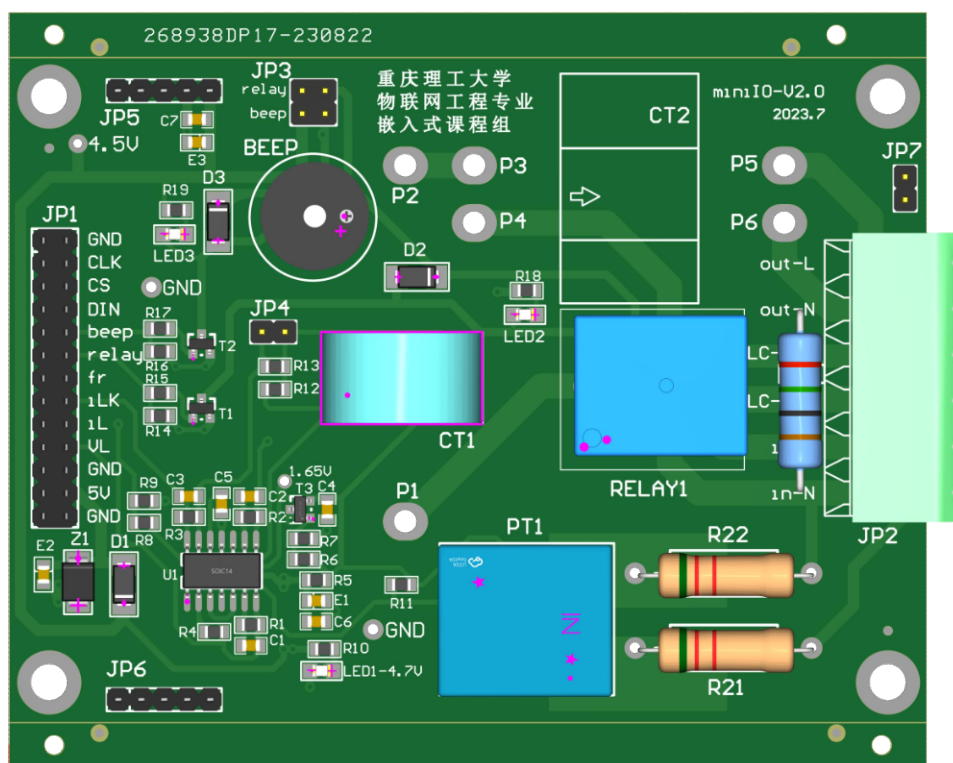


图 3.11 miniIO 元件布置补效果图

miniIO原理图如下：

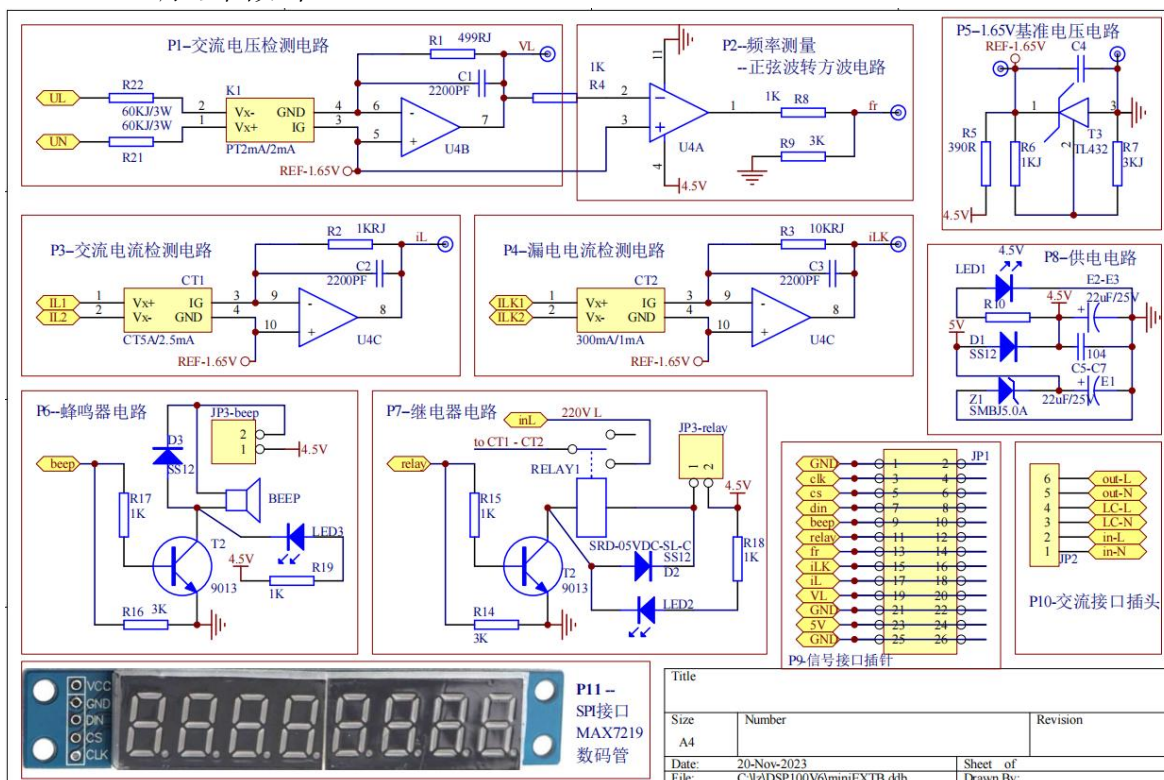
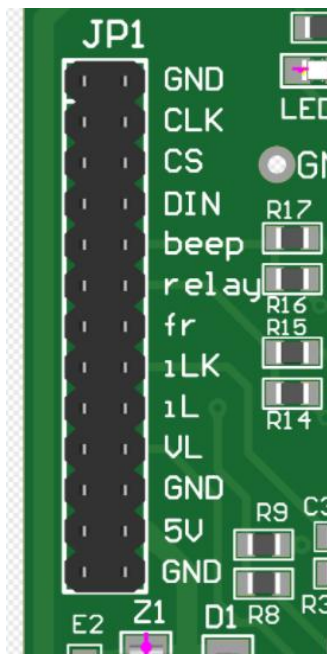


图 3.12 miniIO原理图

miniIO接口信号分配



miniIO 信号分配如左面图 1.12 所示，图中信号引出为 JP1，横向 2 针为同一个信号，主要信号如下：

- (1) 5V，供电电源，占用 JP1-23、24 针；
- (2) GND，电源地，占用 JP1-1、2、21、22、25、26 针；
- (3) CLK，SPI 时钟，占用 JP1-3、4 针；
- (4) CS，SPI 片选，占用 JP1-5、6 针；
- (5) DIN，SPI 的 MOSI，占用 JP1-7、8 针；
- (6) beep，蜂鸣器控制，占用 JP1-9、10 针；
- (7) relay，继电器控制，占用 JP1-11、12 针，高电平有效（继电器动作）；
- (8) fr，市电交流频率方波，占用 JP1-13、14 针；
- (9) iLK，市电交流漏电电流检测，占用 JP1-15、16 针；
- (10) iL，市电交流电流检测，占用 JP1-17、18 针；
- (11) VL，市电交流电压检测，占用 JP1-19、20 针；

除 5V 供电外，其余信号都是幅值为 3.3V 信号，可兼容 5V 电压。

图 3.13 调理后的输入输出信号分配图

第 4 章 PWM 频率测试信号产生与测频

本章讨论交流频率测量方法，以及测试频率产生，我们测频采用通用定时器输入捕获频率源周期，进而计算频率。为了检测频率测量有效性，我们需要通过通用定时器 PWM 输出通道产生测试用频率源。

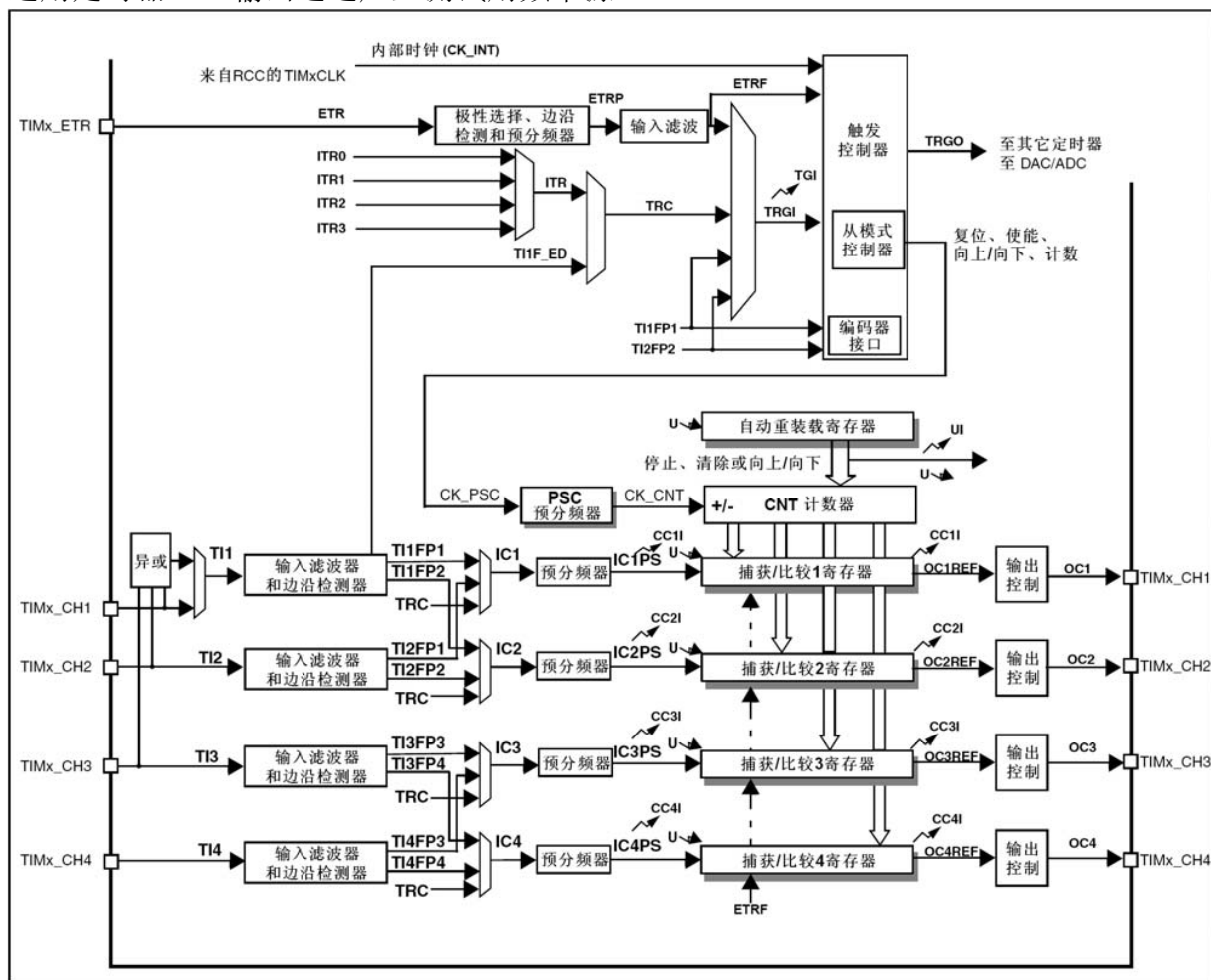


图 4-1 通用定时器框图

如图 4-1 所示，生成测试频率源可以使用 TIMx_CHy（OC1~OC4）输出占空波 50% 的 PWM 波形，通过通过串口及通讯协议修改输出频率。测量频率信号的周期可以使用 TIMx_CHy（TI1~TI4），捕捉边沿时刻，继而计算出信号源周期。

4.1 基于 PWM 的频率信号源产生

PWM 工作原理如图 4-2 所示，STM32F103 的 PWM（脉宽调制）工作原理基于其定时器（TIM）模块，通过配置定时器的计数、比较和输出控制来生成可调占空比的方波信号。以下是其核心工作原理和配置步骤：

(1) 定时器基础

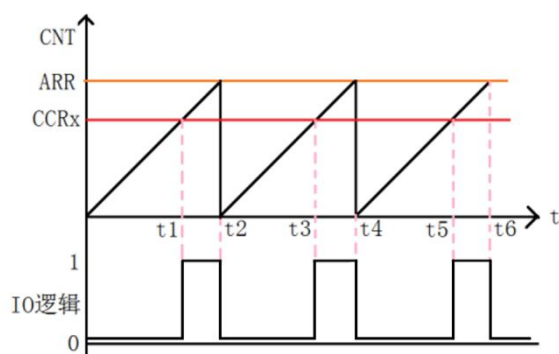


图 4-2 PWM 时序原理图

STM32F103 的 PWM 由通用或高级定时器（如 TIM1-TIM8）生成，核心组件包括：

- ❖ 计数器寄存器（CNT）：向上、向下或双向计数。
- ❖ 自动重载寄存器（ARR）：决定 PWM 周期（频率）。
- ❖ 比较寄存器（CCR_x）：决定占空比（x 为通道号，如 CCR1）。
- ❖ 预分频器（PSC）：调整定时器时钟频率，进一步控制周期。

(2) PWM 模式

定时器支持两种 PWM 模式（通过寄存器 TIM_x_CCMR_x 配置）：

- ❖ PWM 模式 1：
 - ✧ 计数器值 CNT < CCR_x 时，输出有效电平（高或低）；
 - ✧ 计数器值 CNT ≥ CCR_x 时，输出无效电平。
- ❖ PWM 模式 2：
 - ✧ 与模式 1 逻辑相反。

(3) 输出极性控制

通过 TIM_x_CCER 寄存器的 CCxP 位可设置输出极性：

CCxP=0：有效电平为高。

CCxP=1：有效电平为低。

(4) PWM 周期与占空比计算

- ❖ 周期（T）：

$$T = \frac{(ARR + 1) \times (PSC + 1)}{TIM_CLK} \quad (4-1)$$

- ✧ TIM_CLK：定时器时钟频率（如 72MHz）。
- ✧ ARR：自动重载值（决定周期）。
- ✧ PSC：预分频值（分频输入时钟）。

- ❖ 占空比（D）：

$$D = \frac{CCR_x}{ARR + 1} \times 100\% \quad \text{或} \quad D = 100\% - \frac{CCR_x}{ARR + 1} \times 100\% \quad (4-2)$$

- ✧ CCR_x：比较值（决定高电平时间）。

(5) 关键配置步骤（以 TIM1_CH1 为例）

① 使能时钟：

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM1, ENABLE); //使能 TIM1 时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); //使能 GPIO 外设时钟
```

② 配置 GPIO 为复用推挽输出：

```
GPIO_InitTypeDef g1;
//设置 PA8 引脚为复用输出功能,输出 TIM1 CH1 的 PWM 脉冲波形
g1.GPIO_Pin = GPIO_Pin_8; //TIM_CH1
g1.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
g1.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &g1);
```

③ 配置定时器参数：

```
TIM_TimeBaseInitTypeDef tini;
tini.TIM_Period = arr; //设置更新事件装入的自动重载寄存器周期的值
tini.TIM_Prescaler = psc; //设置用来作为 TIMx 时钟频率除数的预分频值
tini.TIM_ClockDivision = TIM_CKD_DIV1; //设置时钟分割:TDTS = Tck_tim
tini.TIM_CounterMode = TIM_CounterMode_Up; //TIM 向上计数模式
```

TIM_TimeBaseInit(TIM1, &tini); //根据 tini 中指定的参数初始化 TIM1

④ 配置 PWM 模式:

```
TIM_OCInitTypeDef ocini;  
ocini.TIM_OCMode = TIM_OCMode_PWM2; //选择定时器模式:TIM 脉冲宽度调制模式 2  
ocini.TIM_OutputState = TIM_OutputState_Enable; //比较输出使能  
ocini.TIM_Pulse = 0; //设置待装入捕获比较寄存器的脉冲值  
ocini.TIM_OCPolarity = TIM_OCPolarity_High; //输出极性:TIM 输出比较极性高  
TIM_OC1Init(TIM1, &ocini); //根据 ocini 中指定的参数初始化 TIM1_OC1
```

⑤ 禁能预装载

```
TIM_OC1PreloadConfig(TIM1, TIM_OCPreload_Disable); //CH1 预装载禁能  
TIM_ARRPreloadConfig(TIM1, DISABLE); //禁能 TIM1 在 ARR 上的预装载寄存器  
禁能预装载后, 写入的值立即生效。
```

⑥ 启动定时器:

```
TIM_CtrlPWMOutputs(TIM1, ENABLE); //启动 OC1 输出  
TIM_Cmd(TIM1, ENABLE); //使能 TIM1
```

示例: 生成 1kHz PWM (占空比 50%)

假设系统时钟 72MHz, 预分频 PSC=71, ARR=999:

周期计算:

$$T = \frac{(999 + 1) \times (71 + 1)}{72 \text{ MHz}} = 1 \text{ ms (1 kHz)}$$

CCR1 = 500, 占空比=500/1000=50%。

通过灵活配置 ARR、CCR_x 和 PSC, STM32F103 可生成精确的 PWM 信号, 广泛应用于电机控制、LED 调光、电源转换等场景。

4.2 基于捕捉功能的频率测量

STM32F103 的定时器输入捕捉 (Input Capture) 功能用于测量外部信号的频率或脉冲宽度, 其工作原理基于捕获信号边沿并记录定时器计数值, 通过计算两次捕获的差值来得到信号周期或频率。以下是详细的工作原理和配置步骤:

(1) 输入捕捉功能核心原理

- ❖ 边沿触发: 通过检测外部信号 (如方波) 的上升沿或下降沿, 触发捕获事件。
- ❖ 计数记录: 当边沿到来时, 定时器的当前计数值 (CNT) 被自动保存到捕获/比较寄存器 (CCR_x)。
- ❖ 差值计算: 两次捕获的 CCR_x 值之差即为信号周期对应的计数值, 结合定时器时钟可计算出实际频率。

(2) 关键硬件组件

- ❖ 定时器通道: 支持输入捕捉的通道 (如 TIM_x_CH1-CH4)。
- ❖ 捕获寄存器 (CCR_x): 存储边沿触发时的 CNT 值。
- ❖ 输入滤波器和边沿检测:
 - ✧ 通过 TIM_x_CCMR_x 寄存器配置边沿 (上升/下降/双边沿)。
 - ✧ 可设置数字滤波器 (抗抖动)。

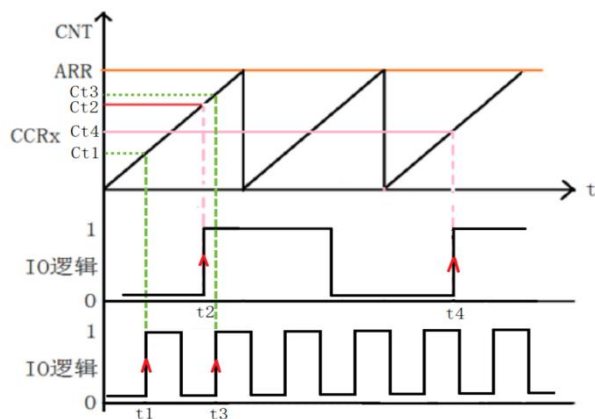


图 4-2 捕捉功能时序原理图

(3) 频率测量方法

方法一：直接周期测量（适用于低频信号）

- ① 配置为上升沿触发：

捕获连续两个上升沿的 CNT 值（CCRx1 和 CCRx2）。

- ② 计算周期：

若未发生计数器溢出：

$$\text{差值} = \text{CCRx2} - \text{CCRx1} \quad (4-3)$$

若发生溢出：

$$\text{差值} = (\text{ARR}+1) + \text{CCRx2} - \text{CCRx1} \quad (4-4)$$

- ③ 频率公式：

$$f = \frac{\text{TIM_CLK}}{(\text{PSC} + 1) \times \text{差值}} \quad (4-5)$$

方法二：脉冲计数+定时器（适用于高频信号）

✧ 外部信号作为时钟：将信号输入定时器的外部时钟模式（ETR 引脚）。

✧ 定时器作为计数器：在固定时间内（如 1 秒）统计脉冲数。

(4) 配置步骤（以 TIM2_CH1 为例）

硬件连接

信号输入引脚：PA0（TIM2_CH1）。

代码配置

- ① 使能时钟和 GPIO：

// 1. 开启时钟

RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

- ② 配置 GPIO 为输入（浮空或上拉）：

GPIO_InitTypeDef g1;

// 2. 配置 PA0 为输入（TIM2_CH1）

g1.GPIO_Pin = GPIO_Pin_0;

g1.GPIO_Mode = GPIO_Mode_IN_FLOATING; // 浮空输入

GPIO_Init(GPIOA, &g1);

- ③ 配置定时器基础参数：

TIM_TimeBaseInitTypeDef tini;

// 3. 配置 TIM2 时基（1MHz 计数频率，72MHz/72=1MHz）

tini.TIM_Period = 0xFFFF; // ARR 最大值

tini.TIM_Prescaler = 72 - 1; // 分频系数

tini.TIM_ClockDivision = TIM_CKD_DIV4;

tini.TIM_CounterMode = TIM_CounterMode_Up;

TIM_TimeBaseInit(TIM2, &tini);

- ④ 配置输入捕捉通道：

TIM_ICInitTypeDef capini;

// 4. 配置输入捕获通道 1（上升沿触发）

capini.TIM_Channel = TIM_Channel_1;

capini.TIM_ICPolarity = TIM_ICPolarity_Rising;

capini.TIM_ICSelection = TIM_ICSelection_DirectTI;

capini.TIM_ICPrescaler = TIM_ICPSC_DIV1; // 无分频

capini.TIM_ICFilter = 0x03; // 0(无滤波)~15(最大滤波)

TIM_ICInit(TIM2, &capini);

- ⑤ 使能捕获中断：

```

    // 5. 启用捕获中断
    TIM_ITConfig(TIM2, TIM_IT_CC1, ENABLE);
⑥ 配置 NVIC
    NVIC_InitTypeDef n1;
    // 6. 配置 NVIC
    n1.NVIC_IRQChannel = TIM2_IRQn;
    n1.NVIC_IRQChannelPreemptionPriority = 0;
    n1.NVIC_IRQChannelSubPriority = 1;
    n1.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&n1);
⑦ 启动 TIM2
    TIM_Cmd(TIM2, ENABLE); // 启动定时器
⑧ 中断服务函数中处理捕获值:
    volatile uint32_t last_capture = 0, period = 0;
    void TIM2_IRQHandler(void)
    {
        uint16_t current_capture;
        if (TIM_GetITStatus(TIM2, TIM_IT_CC1) == SET)
        {
            current_capture = TIM2->CCR1; //TIM_GetCapture1(TIM2);
            period = current_capture - last_capture; // 计算周期计数值
            last_capture = current_capture;
            TIM_ClearITPendingBit(TIM2, TIM_IT_CC1);
        }
    }
⑨ 计算频率:
    float fr;
    fr = 1000000.0f / period; // 单位 Hz (假设定时器时钟为 1MHz)

```

(5) 注意事项

- ✧ 高频信号：若信号频率过高（周期小于定时器分辨率），需减小 PSC 或使用更高时钟。
- ✧ 低频信号：若周期超过 ARR 值，需启用定时器溢出中断并统计溢出次数。
- ✧ 抗干扰：通过 TIM_ICFilter 设置滤波（如 0x0F 表示 16 个时钟周期滤波）。

(6) 示例：测量 1kHz 方波

配置：

TIM_CLK=72MHz, PSC=71 → 定时器时钟=1MHz (1us/计数)。

信号周期=1ms → 预期捕获差值=1000。

结果：

$f = 1\text{MHz}/1000 = 1\text{kHz}$

通过输入捕捉功能，STM32F103可精确测量外部信号的频率或脉宽，适用于编码器、转速传感器等应用场景。

第 5 章 STM32F103 的 DMA 传送

DMA(Direct Memory Access, 直接存储器访问)是STM32微控制器中的一个重要外设, 它允许外设与内存之间或内存与内存之间直接进行数据传输, 而无需CPU的干预。使用DMA可以大大提高数据传输效率, 减少CPU负担。

STM32F103系列微控制器内置了2个DMA控制器(DMA1和DMA2)，个通道专门用来管理来自于一个或多个外设对存储器访问的请求。还有一个仲裁器来协调各个DMA请求的优先权。

其中:

- DMA1有7个通道
- DMA2(仅在大容量产品中有)有5个通道

5.1 DMA主要特性

- ◆ 支持存储器到存储器、存储器到外设、外设到存储器的传输
- ◆ 可编程的数据传输量：最大65535
- ◆ 支持8位、16位和32位数据宽度
- ◆ 支持循环缓冲管理
- ◆ 每个通道都有独立的硬件中断

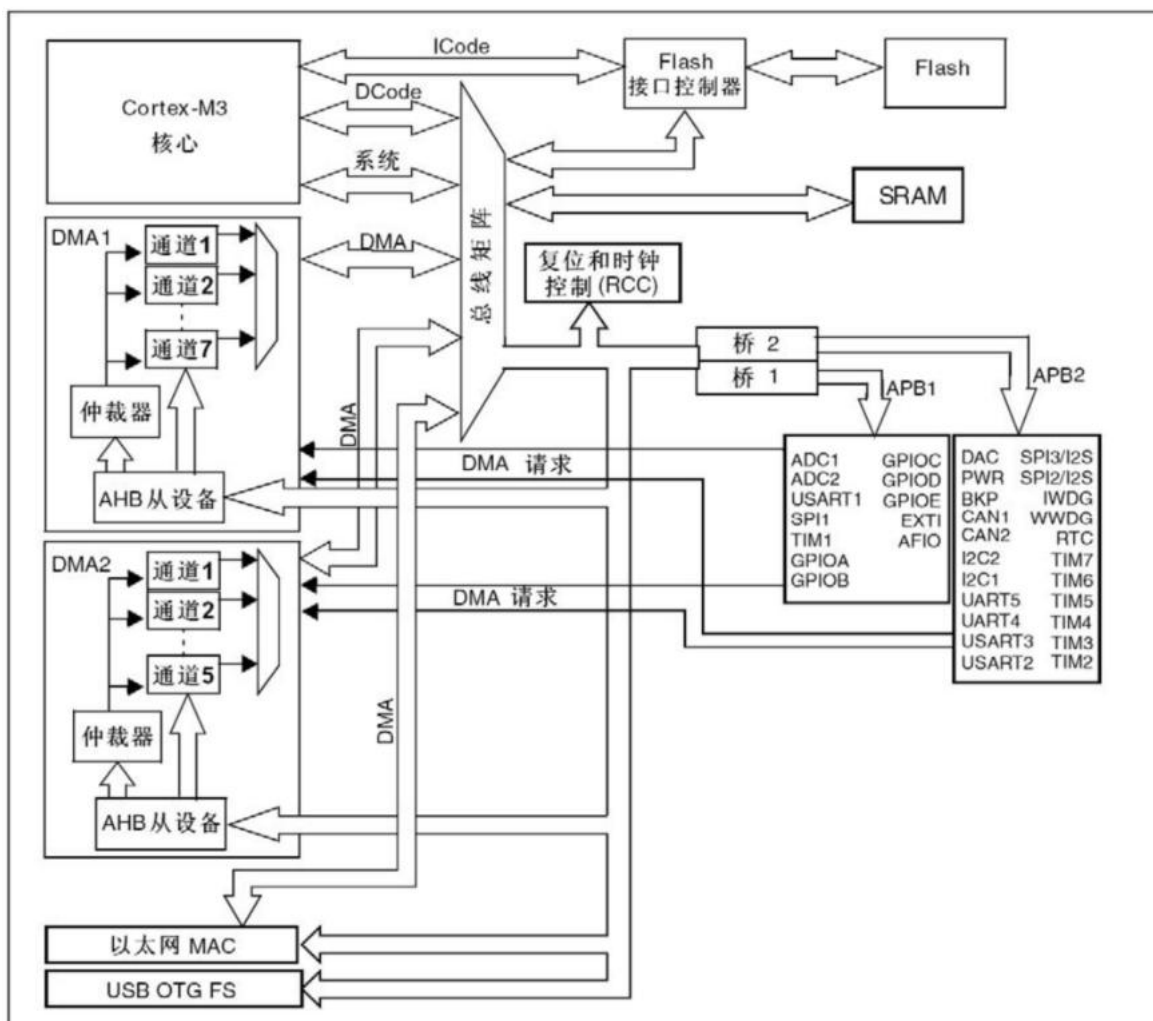


图5-1 STM32的DMA结构框图

5.2 DMA请求映像

STM32外设的DMA请求，可以通过设置相应外设寄存器中的控制位，被独立地开启或关闭。图中多个DMA请求通过逻辑或输入到DMA控制器，这意味着同时只能有一个请求有效。参见下面的DMA请求映像。

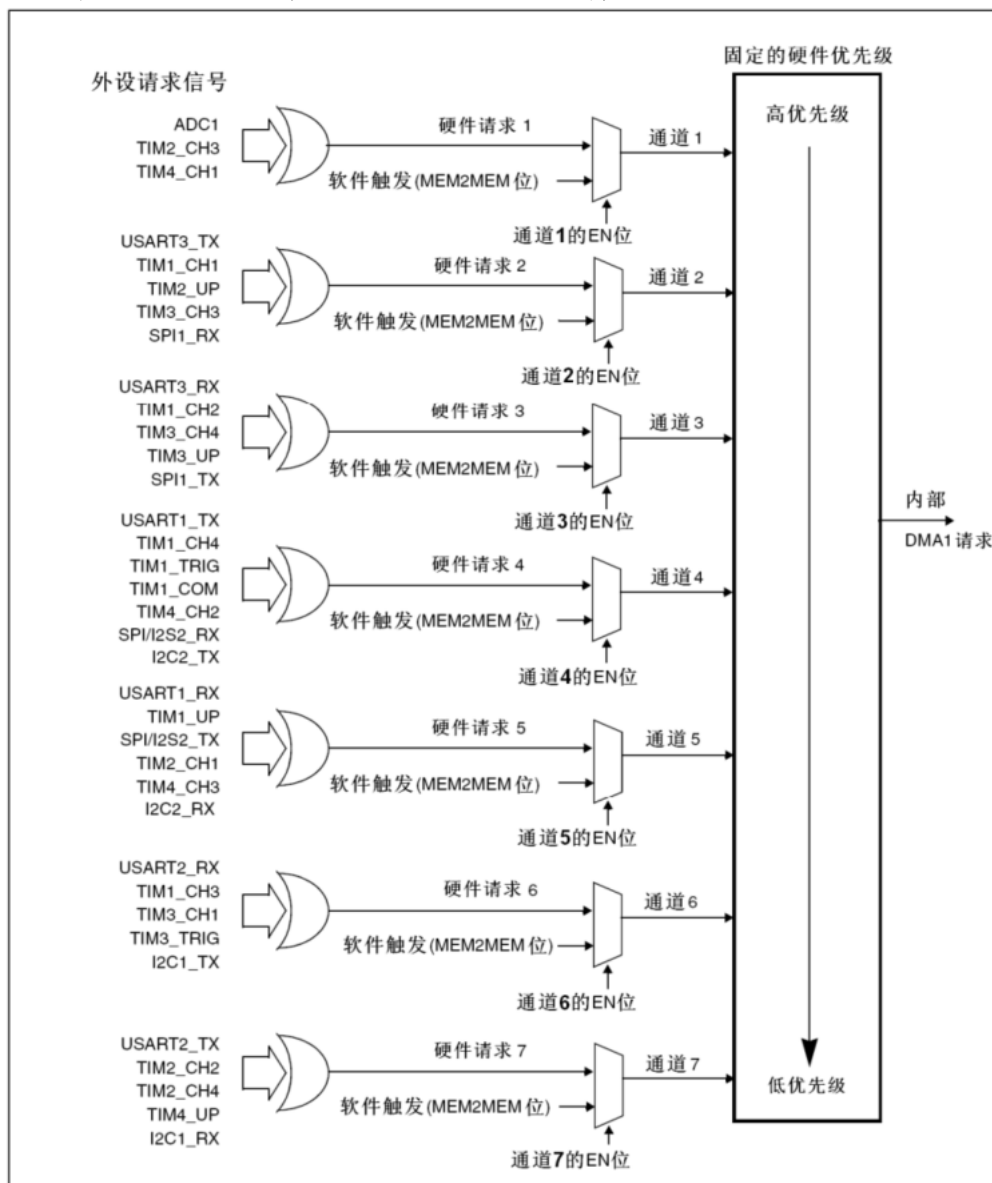


图5-2 DMA1请求映像图

表 5-1 DMA1中断请求各个通道的请求一览

外设	通道1	通道2	通道3	通道4	通道5	通道6	通道7
ADC1	ADC1						
SPI/I ² S		SPI1_RX	SPI1_TX	SPI/I2S2_RX	SPI/I2S2_TX		
USART		USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
I ² C				I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
TIM1		TIM1_CH1	TIM1_CH2	TIM1_TX4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
TIM2	TIM2_CH3	TIM2_UP			TIM2_CH1		TIM2_CH2 TIM2_CH4
TIM3		TIM3_CH3	TIM3_CH4 TIM3_UP			TIM3_CH1 TIM3_TRIG	
TIM4	TIM4_CH1			TIM4_CH2	TIM4_CH3		TIM4_UP

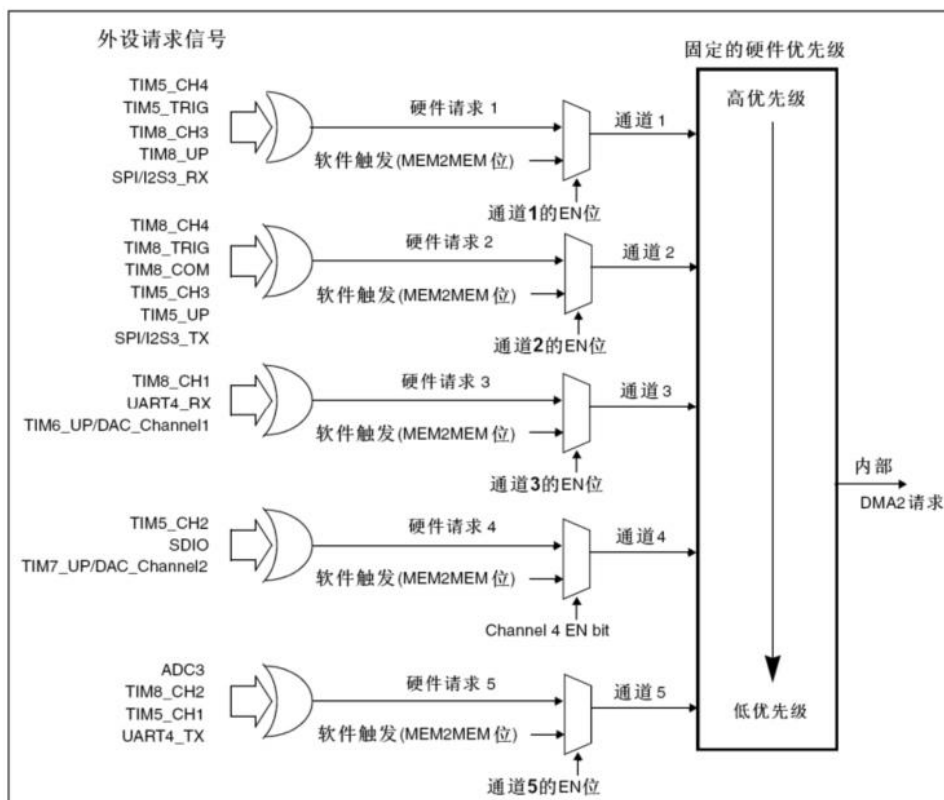


图5-3 DMA2请求映像图

表5-2 DMA2中断请求各个通道的请求一览

外设	通道1	通道2	通道3	通道4	通道5
ADC3 ⁽¹⁾					ADC3
SPI/I2S3	SPI/I2S3_RX	SPI/I2S3_TX			
UART4			UART4_RX		UART4_TX
SDIO ⁽¹⁾				SDIO	
TIM5	TIM5_CH4 TIM5_TRIG	TIM5_CH3 TIM5_UP		TIM5_CH2	TIM5_CH1
TIM6/ DAC通道1			TIM6_UP/ DAC通道1		
TIM7/ DAC通道2				TIM7_UP/ DAC通道2	
TIM8 ⁽¹⁾	TIM8_CH3 TIM8_UP	TIM8_CH4 TIM8_TRIG TIM8_COM	TIM8_CH1		TIM8_CH2

5.3 DMA配置步骤

(1) 使能DMA时钟

首先需要开启DMA的时钟：

```
RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1, ENABLE);
```

(2) 初始化DMA通道

使用DMA_Init() 函数初始化DMA通道：

```
DMA_InitTypeDef ds1;
```

```
ds1.DMA_PeripheralBaseAddr = (uint32_t)&ADC1->DR; // 外设地址
```

```
ds1.DMA_MemoryBaseAddr = (uint32_t)ADC_ConvertedValue; // 内存地址
```

```
ds1.DMA_DIR = DMA_DIR_PeripheralSRC; // 传输方向：外设到内存
```

```

    ds1.DMA_BufferSize = 1; // 传输数据量
    ds1.DMA_PeripheralInc = DMA_PeripheralInc_Disable; // 外设地址不递增
    ds1.DMA_MemoryInc = DMA_MemoryInc_Enable; // 内存地址递增
    ds1.DMA_PeripheralDataSize=DMA_PeripheralDataSize_HalfWord;//外设数据宽度
16位

```

```

    ds1.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord; // 内存数据宽度16位
    ds1.DMA_Mode = DMA_Mode_Circular; // 循环模式
    ds1.DMA_Priority = DMA_Priority_High; // 高优先级
    ds1.DMA_M2M = DMA_M2M_Disable; // 禁用内存到内存模式
    DMA_Init(DMA1_Channel1, &ds1);

```

(3) 使能DMA通道

```

    DMA_Cmd(DMA1_Channel1, ENABLE);

```

(4) 配置外设使用DMA

根据需要使用的外设，配置其DMA请求：

// 以ADC为例

```

    ADC_DMACmd(ADC1, ENABLE);

```

5.4 DMA传输示例

(1) 示例1：内存到内存传输

```

#define BUFFER_SIZE 32
uint32_t srcBuffer[BUFFER_SIZE] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};
uint32_t dstBuffer[BUFFER_SIZE];
void DMA_MemToMem_Config(void)
{
    DMA_InitTypeDef ds1;
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1, ENABLE);
    DMA_DeInit(DMA1_Channel6);
    ds1.DMA_PeripheralBaseAddr = (uint32_t)srcBuffer;
    ds1.DMA_MemoryBaseAddr = (uint32_t)dstBuffer;
    ds1.DMA_DIR = DMA_DIR_PeripheralSRC;
    ds1.DMA_BufferSize = BUFFER_SIZE;
    ds1.DMA_PeripheralInc = DMA_PeripheralInc_Enable;
    ds1.DMA_MemoryInc = DMA_MemoryInc_Enable;
    ds1.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word;
    ds1.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
    ds1.DMA_Mode = DMA_Mode_Normal;
    ds1.DMA_Priority = DMA_Priority_High;
    ds1.DMA_M2M = DMA_M2M_Enable;
    DMA_Init(DMA1_Channel6, &ds1);
    DMA_Cmd(DMA1_Channel6, ENABLE);
    // 等待传输完成
    while(DMA_GetFlagStatus(DMA1_FLAG_TC6) == RESET);
    DMA_ClearFlag(DMA1_FLAG_TC6);
}

```

(2) 示例2：USART DMA传输

```

#define USART_TX_DMA_BUFFER_SIZE 64
uint8_t USART_TX_DMA_Buffer[USART_TX_DMA_BUFFER_SIZE];
void USART_DMA_Config(void)

```

```

{
    DMA_InitTypeDef ds1;
    // 填充发送缓冲区
    for(int i=0; i<USART_TX_DMA_BUFFER_SIZE; i++) {
        USART_TX_DMA_Buffer[i] = i;
    }
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1, ENABLE);
    DMA_DeInit(DMA1_Channel4);
    ds1.DMA_PeripheralBaseAddr = (uint32_t)&USART1->DR;
    ds1.DMA_MemoryBaseAddr = (uint32_t)USART_TX_DMA_Buffer;
    ds1.DMA_DIR = DMA_DIR_PeripheralDST;
    ds1.DMA_BufferSize = USART_TX_DMA_BUFFER_SIZE;
    ds1.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    ds1.DMA_MemoryInc = DMA_MemoryInc_Enable;
    ds1.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte;
    ds1.DMA_MemoryDataSize = DMA_MemoryDataSize_Byte;
    ds1.DMA_Mode = DMA_Mode_Normal;
    ds1.DMA_Priority = DMA_Priority_Medium;
    ds1.DMA_M2M = DMA_M2M_Disable;
    DMA_Init(DMA1_Channel4, &ds1);
    USART_DMACmd(USART1, USART_DMAREq_Tx, ENABLE);
    DMA_Cmd(DMA1_Channel4, ENABLE);
}

```

5.5 DMA中断

每个DMA通道都可以在DMA传输过半、传输完成和传输错误时产生中断。为应用的灵活性考虑，通过设置寄存器的不同位来打开这些中断。

表 5-3 DMA中断请求

中断事件	事件标志位	使能控制位
传输过半	HTIF	HTIE
传输完成	TCIF	TCIE
传输错误	TEIF	TEIE

DMA传输完成可以产生中断，需要在NVIC中配置：

```

// 配置DMA中断
NVIC_InitTypeDef nv1;
nv1.NVIC_IRQChannel = DMA1_Channel1_IRQn;
nv1.NVIC_IRQChannelPreemptionPriority = 0;
nv1.NVIC_IRQChannelSubPriority = 0;
nv1.NVIC_IRQChannelCmd = ENABLE;
NVIC_Init(&nv1);
// 使能DMA传输完成中断
DMA_ITConfig(DMA1_Channel1, DMA_IT_TC, ENABLE);
// DMA中断处理函数
void DMA1_Channel1_IRQHandler(void)
{
    if(DMA_GetITStatus(DMA1_IT_TC1) != RESET)
    {
        // 处理传输完成
    }
}

```

```
        DMA_ClearITPendingBit(DMA1_IT_TC1);  
    }  
}
```

5.6 注意事项

- ✧ 确保DMA通道与外设的对应关系正确（参考芯片参考手册）
- ✧ 在DMA传输期间不要修改源或目标地址
- ✧ 使用循环模式时，确保缓冲区大小足够
- ✧ 对于内存到内存传输，只能使用DMA1的通道6和7
- ✧ 传输完成后及时清除标志位

5.7 常见问题

- DMA不工作：检查时钟是否使能，通道配置是否正确，外设DMA请求是否开启
- 数据传输不完整：检查缓冲区大小设置是否正确
- 数据错位：检查地址递增和数据宽度配置是否匹配
- 中断不触发：检查NVIC配置和中断使能是否正确

第 6 章 测试正弦波的生成

本章我们介绍测试信号源—双路可串口修改幅值、频率、相位的正弦波信号发生器的设计。使用STM32F103的双路DA输出，采用DMA数据传送。

本方案使用STM32F103的定时器触发DMA传输，将预先计算好的正弦波数据通过DAC输出，实现50Hz正弦波信号生成。系统主要由以下部分组成：

- ◆ 计算的正弦波数据表数组
- ◆ DMA控制器用于高效传输数据到DAC
- ◆ 定时器(TIM)用于精确触发DMA传输
- ◆ 数模转换器(DAC)输出模拟信号

6.1 STM32F103的DA介绍

数字/模拟转换模块(DAC)是12位数字输入，电压输出的数字/模拟转换器。DAC可以配置为8位或12位模式，也可以与DMA控制器配合使用。DAC工作在12位模式时，数据可以设置成左对齐或右对齐。DAC模块有2个输出通道，每个通道都有单独的转换器。在双DAC模式下，2个通道可以独立地进行转换，也可以同时进行转换并同步地更新2个通道的输出。DAC可以通过引脚输入参考电压VREF+以获得更精确的转换结果。

DAC主要特征

- 2个DAC转换器：每个转换器对应1个输出通道
- 8位或者12位单调输出
- 12位模式下数据左对齐或者右对齐
- 同步更新功能
- 噪声波形生成
- 三角波形生成
- 双DAC通道同时或者分别转换
- 每个通道都有DMA功能
- 外部触发转换
- 输入参考电压VREF+

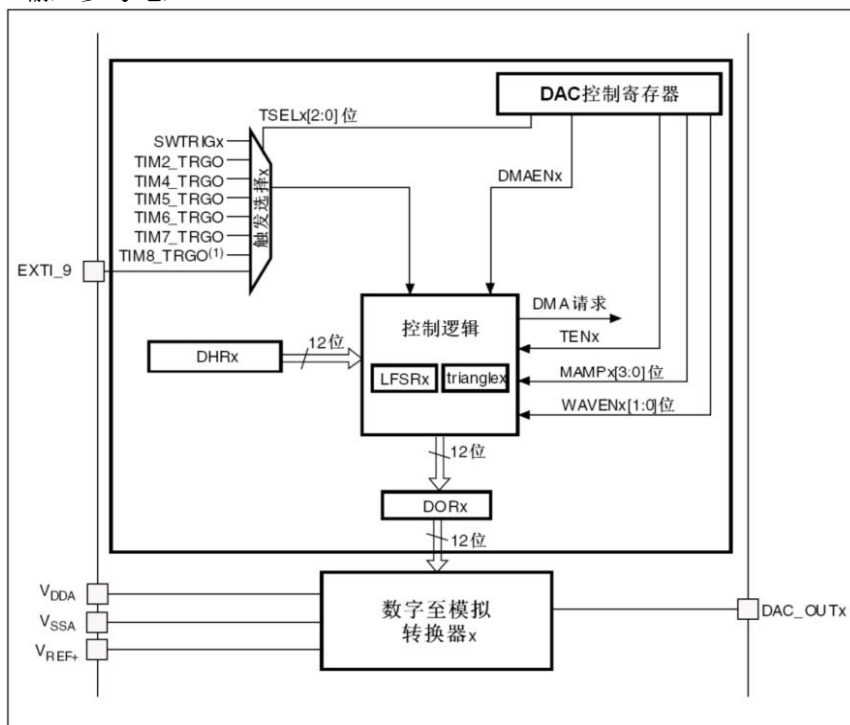


图 5.1 DAC 模块框图

STM32F103的DAC1使用引脚PA4、DAC2使用引脚PA5，一旦使能DACx通道，相应的GPIO引脚（PA4或者PA5）就会自动与DAC的模拟输出相连（DAC_OUTx）。为了避免寄生的干扰和额外的功耗，引脚PA4或者PA5在之前应当设置引脚成模拟输入模式（AIN）。

6.2 正弦波DA输出数组的生成

STM32F103的DAC为12位，数据范围0~4095，正弦波零点为2047，也就是输出数据为： $2047+2047*\sin(\alpha)$ 。生成360点的正弦波数组，有2种常见方式：

(1) sin函数生成数据表：

```
#define SINE_WAVE_POINTS 360 // 一个周期采样点数
uint16_t sineWave[SINE_WAVE_POINTS];
void generateSineWave(void) // 生成 12 位 DAC 值 (0-4095)，偏移到中间值 2047
{
    for(int i = 0; i < SINE_WAVE_POINTS; i++)
        sineWave[i] = (uint16_t)(2047*(1+sin(2*3.1416*i/SINE_WAVE_POINTS)));
}
```

(2) 事先存储数据表：

通过VC++6.0等PC软件提前准备好数组数据表，嵌入式编程时直接拷贝该常数数组（存放于代码区）。该方式占用STM32运行时间较少，但需要占用程序区存储数组。PC程序如下：

```
#include <stdio.h>
#include <math.h>
#define PI 3.14159265358979323846
#define SAMPLES 360
#define FILENAME "sine_wave.txt"
int main() {
    FILE *file;
    double angle;
    short sine_value;
    int i;
    file = fopen(FILENAME, "w");// 打开文件用于写入
    if (file == NULL) {printf("Error opening file\n");return 1;}
    fprintf(file, "const int16_t sine_wave[SINE_WAVE_POINTS] = {"); //数组名
    for (i = 0; i < SAMPLES; i++)// 写入正弦波数据到文件
    {
        angle = ((double)i / SAMPLES) * 2 * PI;// 计算角度（以弧度为单位）
        sine_value = 2047+2047*sin(angle);// 计算正弦值，偏移到2047
        if(i%10==0) fprintf(file, "\n");//每行10个数据
        fprintf(file, "%4d,", sine_value);// 将角度和正弦值写入文件
    }
    fprintf(file, "\n};\n");
    fclose(file);// 关闭文件
    printf("Sine wave data has been written to %s\n", FILENAME);
    return 0;
}
```

6.3 设置幅值频率相位

初始化或程序运行过程中需要设置波形频率 $f*0.01\text{Hz}$ 、电压幅值系数 $k1\%$ 、

电流幅值系数k2%以及相位angle度（电流滞后电压的角度）。

频率设定：设定输出正弦波频率，数据范围2000~10000，也就是20.00Hz~100.00Hz。对本系统，每个正弦周期DA输出360点，定时器TIM6工作频率为72MHz，为了完成波形频率f*0.01Hz，那么DAC每输出一个点的时间：

$$tp = 1 \times 10^6 / ((f \times 0.01) \times 360) \quad (\mu s) \quad (\text{式5.1})$$

当f=5000，也就是50Hz时，解（式5.1）得：tp≈55.56（μs）

定时器TIM6的重装载单元：

$$ARR = 72 \times 10^6 / (f \times 0.01) / 360 - 1 = 20000000 / f - 1 \quad (\text{式5.2})$$

当f=5000，也就是50Hz时，解（式5.2）得：ARR=4000-1。

幅值设定：幅值系数单位为0.01%，数据范围0~10000，当等于10000表示幅值最大100.00%（1.0000）。

相位设定：设置电流与电压之间的相位差，单位是度，数据范围-180~180，比如-30代表电流滞后电压30度角度，30代表电流超前电压30度角度。360个点的正弦数组中，DAC提前和推后输出1数据点正好对应超前和滞后1度。

函数如下：

```
short SetWaveData(short f, short k1, short k2, short angle)
{
    short i, j;
    TIM6->CNT=0;
    TIM6->ARR=20000000L/f-1;
    for(i=0; i<SINE_WAVE_POINTS; i++)
    {
        sine_wave1[i]=(sine_wave[i]-2047)*(long)k1/10000L+2047;
        j=i+angle;    //滞后角度
        sine_wave2[i]=(sine_wave[j]-2047)*(long)k2/10000L+2047;
    }
}
```

6.4 DAC输出步骤

为了高效、高精度的产生正弦波，DAC采用定时启动，从图5-2中可以看出，启动DAC可以使用定时器TIM2到TIM8中的任意一个TRGO事件，以下实例使用TIM6的TRGO事件触发DAC。DAC数据输出采用DMA传送循环输出一个周期360个点的正弦波。

需要配置：DAC输出引脚、DAC初始化、启动定时器、DMA输出等。基本步骤如下。

（1）配置引脚

```
// DAC通道1、2配置在PA4、PA5
GPIO_InitTypeDef gl;
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
gl.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
gl.GPIO_Mode = GPIO_Mode_AIN;    // 模拟输入模式(减少DAC输出噪声)
GPIO_Init(GPIOA, &gl);
```

（2）DAC初始化

```
DAC_InitTypeDef dai;
RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE); //使能DAC时钟
// 配置DAC
dai.DAC_Trigger = DAC_Trigger_T6_TRGO;    //使用TIM6的TRGO事件启动DAC
dai.DAC_WaveGeneration = DAC_WaveGeneration_None;
```

```

dai.DAC_LFSRUnmask_TriangleAmplitude=DAC_LFSRUnmask_Bit0;
dai.DAC_OutputBuffer = DAC_OutputBuffer_Enable;
DAC_Init(DAC_Channel_1, &dai);
DAC_Init(DAC_Channel_2, &dai);
// 使能DAC的DMA功能和使能DAC
DAC_DMAMCmd(DAC_Channel_1, ENABLE);
DAC_DMAMCmd(DAC_Channel_2, ENABLE);
DAC_Cmd(DAC_Channel_1, ENABLE);
DAC_Cmd(DAC_Channel_2, ENABLE);

```

(3) 定时器配置

```

TIM_TimeBaseInitTypeDef t6;
RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM6, ENABLE); // 使能定时器时钟
// 配置定时器50Hz频率
t6.TIM_Period = 4000-1;
t6.TIM_Prescaler = 1 - 1; //72/4=18MHz
t6.TIM_ClockDivision = 0;
t6.TIM_CounterMode = TIM_CounterMode_Up;
TIM_TimeBaseInit(TIM6, &t6);
// 使能定时器更新事件触发DAC
TIM_SelectOutputTrigger(TIM6, TIM_TRGOSource_Update);
TIM_Cmd(TIM6, ENABLE); // 使能定时器

```

接下来我们讨论“t6.TIM_Period = 4000-1;”，也就是TIM6→ARR = 4000-1，其中数据4000如何计算出来的。首先，当psc=1-1时，TIM6的计数频率fck=72MHz/1=72MHz；然后，当DAC输出fs=50Hz的正弦波，每个正弦波输出360点，因此我们启动DAC的频率fdac=360*fs；因此TIM6的arr=fck/fdac = 72000000/(360*fs)=200000/fs。当fs=50Hz时，arr=200000/50=4000。当我们用定点小数fx=5000代表50.00Hz时，可以得到如下改变正弦波频率的设定语句：“TIM6→ARR=20000000L/fx;”。

(4) DAC寄存器地址

```

/* DHR registers offsets */
#define DHR12R1_OFFSET ((uint32_t)0x00000008)
#define DHR12R2_OFFSET ((uint32_t)0x00000014)
#define DHR12RD_OFFSET ((uint32_t)0x00000020)
#define DAC_DHR12R1_Address ((uint32_t)0x40007408)
//((uint32_t)DAC_BASE+DHR12R1_OFFSET+DAC_Align_12b_R)
#define DAC_DHR12R2_Address ((uint32_t)0x40007414)

```

(5) DMA配置

```

DMA_InitTypeDef dm1;
RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA2, ENABLE); // 使能DMA时钟
// 配置DMA
dm1.DMA_PeripheralBaseAddr = DAC_DHR12R1_Address;
dm1.DMA_MemoryBaseAddr = (uint32_t)sine_wavel;
dm1.DMA_DIR = DMA_DIR_PeripheralDST;
dm1.DMA_BufferSize = SINE_WAVE_SIZE;
dm1.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
dm1.DMA_MemoryInc = DMA_MemoryInc_Enable;
dm1.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
dm1.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;

```

```

dm1.DMA_Mode = DMA_Mode_Circular;
dm1.DMA_Priority = DMA_Priority_High;
dm1.DMA_M2M = DMA_M2M_Disable;
DMA_Init(DMA2_Channel3, &dm1);
dm1.DMA_PeripheralBaseAddr = DAC_DHR12R2_Address;
dm1.DMA_MemoryBaseAddr = (uint32_t)sine_wave2;
DMA_Init(DMA2_Channel4, &dm1);
// 使能DMA
DMA_Cmd(DMA2_Channel3, ENABLE);
DMA_Cmd(DMA2_Channel4, ENABLE);

```

6.5 串行通信协议

串口通信(UART/USART)作为基础通信方式，在实际应用中通常需要配合帧协议和各种软件协议来实现可靠的数据传输和设备控制。通过串口我们可以把STM32检测的数据发到PC串口助手显示，也可以将需要设定的参数通过PC串口助手下发到STM32。

6.5.1 通信格式

当我们需要修改正弦输出参数时，我们可以直接调用5.3节的函数“short SetWaveData(short f, short k1, short k2, short angle);”来实现。该函数的4个形参决定：

- (1) f：波形频率 $f \times 0.01\text{Hz}$ ；
- (2) k1：电压幅值系数 $k1 \times 0.01\%$ ；
- (3) k2：电流幅值系数 $k2 \times 0.01\%$ ；
- (4) angle：相位angle度。

调用时的配置参数（实参）需要通过人机接口修改，常见的数据修改可以通过触摸屏、按键、串行通信等人机接口完成。

在STM32中，串口通信帧协议主要通过以下方式配置：

- (1) 波特率：常见值有9600，19200，115200等，单位bps
- (2) 数据位：通常8位
- (3) 停止位：1位、1.5位或2位
- (4) 奇偶校验：可选无校验、奇校验或偶校验
- (5) 流控制：可选无、RTS或CTS硬件流控

以下是常见的串口软件协议（数据包）：

- (1) Modbus ASCII（字符传输协议）
 - 用途：工业控制领域
 - 特点：ASCII字符表示十六进制数据，以“:”开始，LRC校验
 - 示例：:0103000000001FB
- (2) Modbus RTU（二进制传输协议）
 - 用途：工业领域最常用协议
 - 特点：紧凑二进制格式，CRC校验
 - 帧结构：[地址][功能码][数据][CRC16]
- (3) 自定义二进制协议

通用结构：

[包头][长度][命令字][数据][校验][包尾]

典型实现：

包头：0xAA 0x55等固定值
 校验：累加和、CRC、异或等
 包尾：0x0D 0x0A等

(4) 自定义字符协议

通用结构：

[包头][命令字][数据][校验][包尾]

典型实现：

包头：‘:’等字符
 命令字：‘>’等字符
 校验：累加和、CRC、异或等
 包尾：‘\r’ ‘\n’等

比如采用自定义字符协议，为了实时显示电压电流等测量参数，我们可以直接使用printf向PC发送需要显示的字符串；为了设定DA输出信号频率、幅值、相位，可以设计如下软件协议，比如PC向STM32发送数据包（字符串）：

PC→STM32：“>5000,8000,6000,-30\r\n”

同时STM32收到数据并正确解析后向PC返回回复信息，比如：

PC→STM32：f=%5.2f,k1=%6.4f,k2=%6.4f,angle=%d.\r\n

其含义为：包头‘:’、命令字‘>’、频率50.00Hz、电压幅值系数0.8000、电压幅值系数0.6000、电流滞后电压30度角度、包尾‘\r’ ‘\n’。

6.5.2 演示程序通讯命令字

表 6-1 通讯功能表

序号	功能	命令字符	命令示例	示例含义
1	修改DA输出数据	>	>5000,8000,6000,30\r\n	修改DA输出数据为： f=50.00Hz, k1=80.00%, k2=60.00%, 角度滞后30度
2	继电器控制	#	#-ON\r\n #-OFF\r\n	继电器合上 继电器断开
3	修改显示格式	\$	\$-P\r\n \$-E\r\n	串口输出标么值 串口输出实际工程量

演示程序中用到的3个字符串命令如表5-1所示：

(1) 修改DA输出数据

串口助手发送：>5000,8000,6000,30\r\n

STM32返回：f=50.00,k1=0.8000,k2=0.6000,angle=30.\r\n

(2) 继电器控制

串口助手发送：#-ON\r\n

STM32返回：Relay is ON.\r\n

串口助手发送：#-OFF\r\n

STM32返回：Relay is OFF.\r\n

(3) 修改显示格式

串口助手发送：\$-P\r\n

STM32返回：Display in percent unit.\r\n

标么值显示格式：U= 83.45%, I= 2.69%, S= 2.24%, P= 2.24%, Q= -0.04%, COS=1.000

串口助手发送：\$-E\r\n

STM32返回: Display in engineering unit.\r\n

工程量显示格式: $U=222.7V$, $I=0.066A$, $S=14.6VA$, $P=14.7W$, $Q=0.2var$, $COS=1.000$

6.5.3 软件设计

(1) 串口初始化

串口初始化主要包括: 串口与GPIO 时钟使能、GPIO 端口模式设置、采用串口结构体参数初始化串口、开启中断并且初始化NVIC、使能串口等。

基本程序如下:

```
void uart_init(u32 bound)
{
    //GPIO端口设置
    GPIO_InitTypeDef g1;
    USART_InitTypeDef u1;
    NVIC_InitTypeDef n1;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1|RCC_APB2Periph_GPIOA, ENABLE);
    //使能USART1, GPIOA时钟
    //USART1_TX   GPIOA.9      //USART1_RX   GPIOA.10初始化
    g1.GPIO_Pin = GPIO_Pin_9; //PA.9
    g1.GPIO_Speed = GPIO_Speed_50MHz;
    g1.GPIO_Mode = GPIO_Mode_AF_PP; //复用推挽输出
    GPIO_Init(GPIOA, &g1); //初始化GPIOA.9
    g1.GPIO_Pin = GPIO_Pin_10; //PA10
    g1.GPIO_Mode = GPIO_Mode_IN_FLOATING; //浮空输入
    GPIO_Init(GPIOA, &g1); //初始化GPIOA.10

    //Usart1 NVIC 配置
    n1.NVIC_IRQChannel = USART1_IRQn;
    n1.NVIC_IRQChannelPreemptionPriority=3 ;//抢占优先级3
    n1.NVIC_IRQChannelSubPriority = 3; //子优先级3
    n1.NVIC_IRQChannelCmd = ENABLE; //IRQ通道使能
    NVIC_Init(&n1); //根据指定的参数初始化VIC寄存器

    //USART 初始化设置
    u1.USART_BaudRate = bound; //串口波特率
    u1.USART_WordLength = USART_WordLength_8b; //字长为8位数据格式
    u1.USART_StopBits = USART_StopBits_1; //一个停止位
    u1.USART_Parity = USART_Parity_No; //无奇偶校验位
    u1.USART_HardwareFlowControl = USART_HardwareFlowControl_None; //无流控
    u1.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式

    USART_Init(USART1, &u1); //初始化串口1
    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); //开启串口接受中断
    USART_Cmd(USART1, ENABLE); //使能串口1
}
```

(2) 接收中断及包尾判断

假设我们采用字符串通讯格式, 一般用回车换行字符表示一个完整的数据包传输完成, STM32的中断接收函数把接收到的字符存入字符数组RX_BUF中, 中断接收函数判断当收到回车换行后设置包接收完成标志, 通知主函数: 数据包接收完成, 主函数完成协议判断并做相应功能。

```

u8 RX_BUF[REC_LEN];    //接收缓冲,最大REC_LEN个字节.
u16 RX_STA=0;          //接收状态标记
//bit15, 接收完成标志 '\r' 0x0a
//bit14, 接收到 '\n' 0x0d
//bit13~0, 接收到的有效字节数目
void USART1_IRQHandler(void) //串口1中断服务程序
{
    u8 Res;
    if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) //接收中断
    {
        Res =USART_ReceiveData(USART1); //读取接收到的数据
        if(RX_STA&0x4000) //前一字节已经接收到了'\r', 需要判断下一个是否'\n'
        {
            if(Res!='\n') RX_STA=0; //如果不是'\n' 接收错误, 重新开始
            else {RX_STA |= 0x8000; RX_BUF[RX_STA&0X3FFF]='\0';} //是'\n' 接收完成
        }
        else //还没收到'\r'
        {
            if(Res=='\r') RX_STA|=0x4000;
            else
            {
                RX_BUF[RX_STA&0X3FFF]=Res; //存放收到字符
                RX_STA++; //收到的字符数
                if(RX_STA>(REC_LEN-1)) RX_STA=0; //接收数据错误, 重新开始接收
            }
        }
    }
}

```

(3) 从完整数据包中解析数据

解析数据需要约定通信格式（协议），假设我们要从串口接收参数（x, y），我们可以约定格式如下：

```

包头: ':'
命令字: '>'
数据: "*****,#####"
包尾: '\r' '\n'

```

其中: x=*****, y=#####

那么我们的协议解析部分函数代码如下：

```

if((RX_STA&0xC000)==0xC000) //接收完成, 该标志在UART中断接收服务函数完成
{
    len=RX_STA&0x3fff; //接收到的字节数
    if(len>=5 && RX_BUF[0]==':' && RX_BUF[1]=='>') //字符数、包头、命令字判断
    {
        i=2;
        x=0;
        for(;i<len;i++) //把收到数字字符串转换为数据存放到x
            if(RX_BUF[i]>='0' && RX_BUF[i]<='9') x=x*10+(RX_BUF[i]-'0');
            else break;
        if(RX_BUF[i]!='(',')') {RX_STA=0;return -1;} //间隔符判断
        i++;
        y=0;
        for(;i<len;i++) //把收到数字字符串转换为数据存放到y

```

```
        if (RX_BUF[i]>='0' && RX_BUF[i]<='9') y=y*10+(RX_BUF[i]-'0');  
        else break;  
    }  
}
```

第 7 章 交流信号采样

本章我们介绍如何用定时器信号触发多通道AD输入、采用DMA数据传送至指定数组存放。

7.1 ADC介绍

STM32F103的12位ADC是一种逐次逼近型模拟数字转换器。它有多达18个通道，可测量16个外部和2个内部信号源。各通道的A/D转换可以单次、连续、扫描或间断模式执行。ADC的结果可以左对齐或右对齐方式存储在16位数据寄存器中。ADC的输入时钟不得超过14MHz，它是由PCLK2经分频产生。

7.2 ADC主要特征

- 12位分辨率
- 转换结束、注入转换结束和发生模拟看门狗事件时产生中断
- 单次和连续转换模式
- 从通道0到通道n的自动扫描模式
- 自校准
- 带内嵌数据一致性的数据对齐
- 采样间隔可以按通道分别编程
- 规则转换和注入转换均有外部触发选项
- 间断模式
- 双重模式(带2个或以上ADC的器件)
- ADC转换时间：
 - STM32F103xx增强型产品：时钟为56MHz时为1μs(时钟为72MHz为1.17μs)
 - STM32F101xx基本型产品：时钟为28MHz时为1μs(时钟为36MHz为1.55μs)
 - STM32F102xxUSB型产品：时钟为48MHz时为1.2μs
 - STM32F105xx和STM32F107xx产品：时钟为56MHz时为1μs(时钟为72MHz为1.17μs)
- ADC供电要求：2.4V到3.6V
- ADC输入范围： $V_{REF-} \leq V_{IN} \leq V_{REF+}$
- 规则通道转换期间有DMA请求产生。

7.3 ADC功能描述

下图是ADC模块的方框图。

有16个多路通道。可以把转换组织成两组：规则组和注入组。在任意多个通道上以任意顺序进行的一系列转换构成成组转换。例如，可以如下顺序完成转换：通道3、通道8、通道2、通道2、通道0、通道2、通道2、通道15。

● 规则组由多达16个转换组成。规则通道和它们的转换顺序在ADC_SQRx寄存器中选择。规则组中转换的总数应写入ADC_SQR1寄存器的L[3:0]位中。

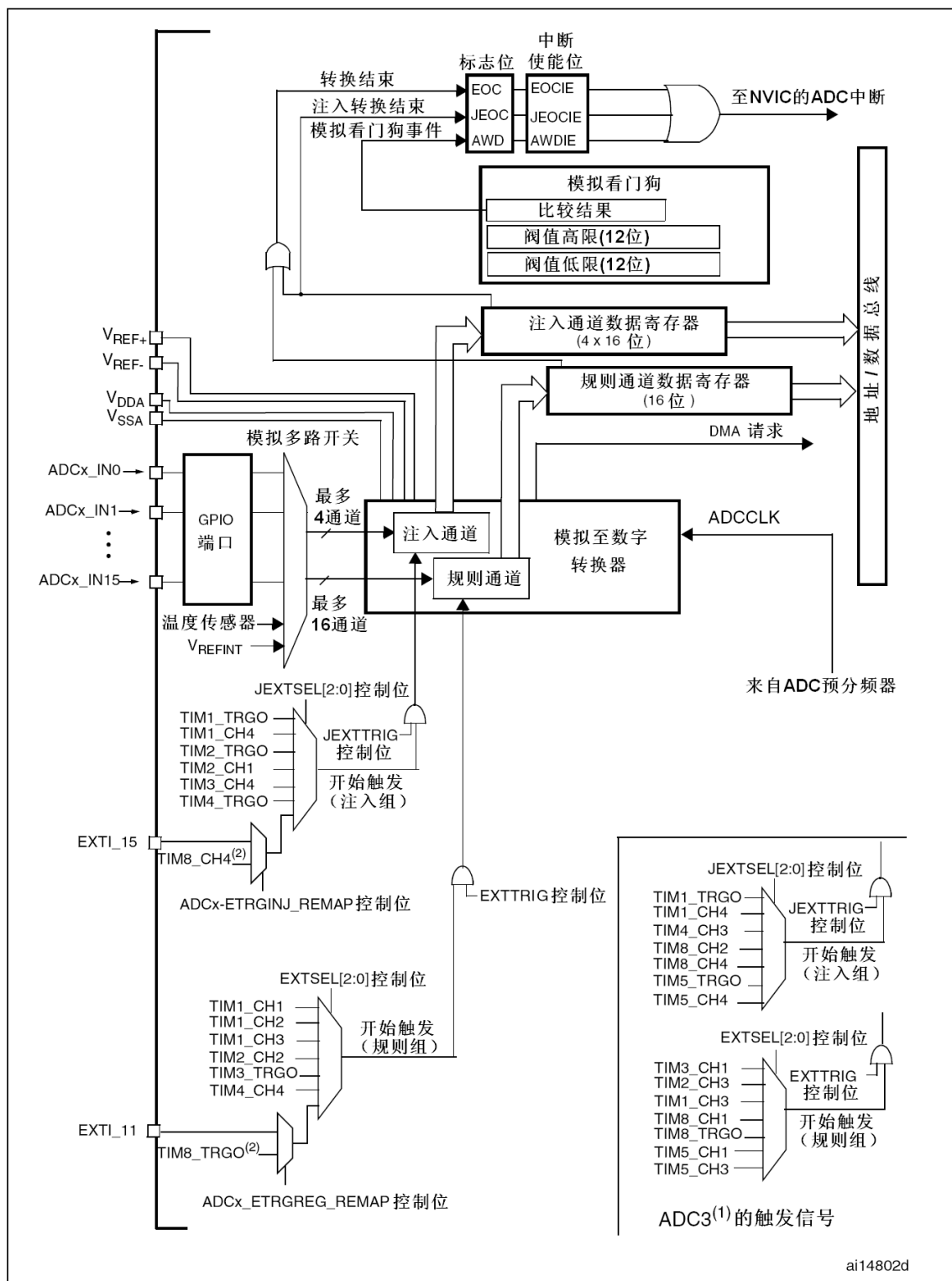
● 注入组由多达4个转换组成。注入通道和它们的转换顺序在ADC_JSQR寄存器中选择。注入组里的转换总数目应写入ADC_JSQR寄存器的L[1:0]位中。

如果ADC_SQRx或ADC_JSQR寄存器在转换期间被更改，当前的转换被清除，一个新的启动脉冲将发送到ADC以转换新选择的组。

温度传感器/ VREFINT内部通道：

温度传感器和通道ADC1_IN16相连接，内部参照电压VREFINT和ADC1_IN17相连

接。可以按注入或规则通道对这两个内部通道进行转换。



用于常规、连续的ADC采样，通常处理多个通道的循环扫描（如多路传感器数据采集）。优先级较低，会被注入通道打断。

➤ **注入通道**

用于高优先级、突发性的采样需求（如紧急信号监测、故障检测）。类似于“中断”机制，可以抢占规则通道的转换。

表6-1 规则通道与注入通道主要区别

特性	规则通道	注入通道
通道数量	最多16个（依赖型号）	通常4个（如STM32F1/F4）
触发方式	软件触发/硬件触发（定时器、外部事件）	支持外部事件（如定时器、GPIO边沿）
数据寄存器	单个规则数据寄存器（需DMA或轮询）	独立的4个数据寄存器（JDR1~JDR4）
优先级	低	高（可打断规则通道）
自动触发后的行为	完成当前转换后继续	立即插入，完成后返回规则通道

规则通道运行过程中

ADC按预设顺序转换规则通道（如通道1→2→3）。
若注入通道被触发（如外部事件），规则通道**立即暂停**，ADC转而处理注入通道（如通道J1→J2）。
注入通道完成后，规则通道**从断点恢复**。

数据存储

规则通道数据需通过DMA或轮询读取（仅1个DR寄存器，需及时处理）。
注入通道数据直接存入独立的JDR寄存器，避免覆盖风险。

7.3.2 ADC启动方式

在STM32F103中，ADC（模数转换器）的启动方式多样，可以通过软件触发或硬件触发来启动规则通道（Regular Channels）和注入通道（Injected Channels）。

(1) 软件启动 (Software Trigger)

单次转换模式（Single Conversion），通过软件直接控制ADC开始转换，适用于非周期性的单次或连续采样。每次启动后只转换一次，需手动重新触发。

(2) 硬件启动 (Hardware Trigger)

通过外部事件或定时器自动触发ADC，适用于精确时序控制的应用（如电机控制、同步采样）。STM32F103的ADC支持的硬件触发源：

- ✧ 定时器（TIMx_TRGO）：如TIM1/TIM2/TIM3/TIM4的更新事件、比较匹配等。
- ✧ 外部中断（EXTI_11）：对应引脚GPIO_PA0（ADC_EXTTRIG）。

7.3.3 STM32的双ADC模式

STM32在有2个或以上ADC模块的产品中，可以使用双ADC模式，在双ADC模式里，根据ADC1_CR1寄存器中DUALMOD[2:0]位所选的模式，转换的启动可以是ADC1主和ADC2从的交替触发或同步触发。

注意：在双ADC模式里，当转换配置成由外部事件触发时，用户必须将其设置成仅触发主ADC，从ADC设置成软件触发，这样可以防止意外的触发从转换。但是，主和从ADC的外部触发必须同时被激活。

下图为双ADC模式结构图：

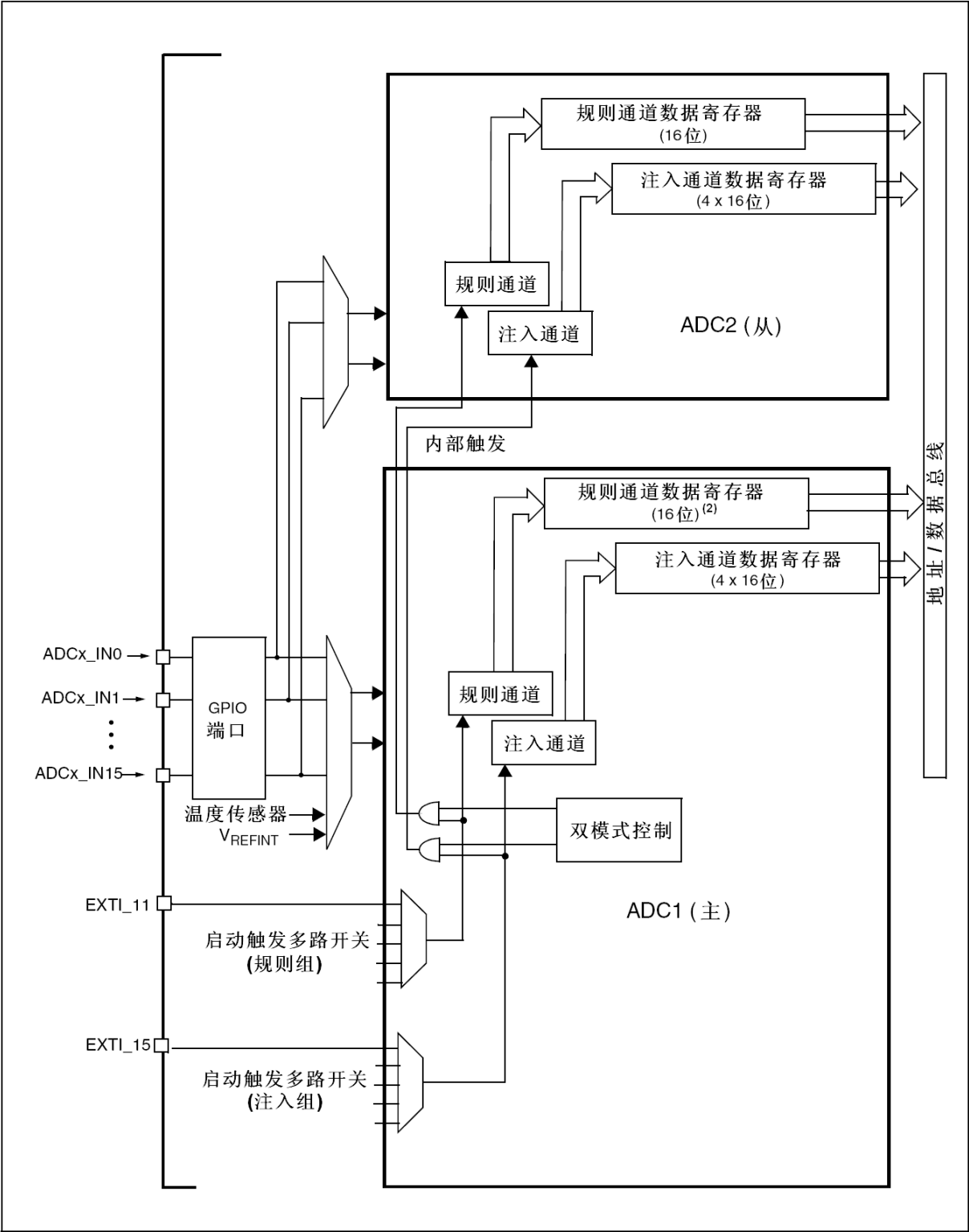


图 6-2 双ADC框图

7.3.4 DMA传输

在STM32F103中，使用DMA（直接内存访问）传输ADC数据可以显著提高效率，

避免CPU频繁中断处理数据，特别适用于多通道扫描或高速连续采样的场景。

(1) DMA传输的基本原理

作用：ADC转换完成后，DMA自动将数据从ADC数据寄存器（DR）搬运到用户指定的内存（如数组），无需CPU干预。

优势：

- ◆ 减少CPU开销，适合实时系统。
- ◆ 避免数据丢失（尤其是高速采样时）。

(2) ADC的DMA通道

STM32F103的ADC1和ADC3支持DMA传输（ADC2需与ADC1配合使用，如双ADC模式）。查询表3.1和表3.2可知ADC的DMA通道如下：

表 6-2 ADC的DMA通道表

ADC	DMA控制器	DMA通道
ADC1	DMA1	Channel1
ADC3	DMA2	Channel5

7.4 ADC配置过程

为了检测交流电压电流信号，我们需要提前做如下安排：

(1) ADC引脚分配

需要分配2个ADC引脚，本例中：电压信号连接PC4、电流信号连接PC5；

(2) ADC模式通道选择

选择ADC1、规则通道、定时启动、2路扫描模式；

(3) ADC启动硬件选择

本例选择定时器TIM3_TRGO触发启动ADC。欲采用每个电网周期（20ms）采样128点，因此定时器TIM3定时时间为20ms/128；

(4) DMA目的缓冲区定义

用于存放2路，每路128点的ADC结果；

```
u16 adval[2*128];
```

(5) DMA选择模式选择

源为ADC转换数据结存起，目的为ADC缓冲区；

DMA中断选择缓冲区满中断；

DMA循环传送模式（缓冲区满以后继续回到缓冲区块首继续传输）。

完成交流采样程序详细步骤如下：

(1) GPIO 配置

将GPIOC的Pin_4和Pin_5配置为模拟输入，作为 ADC 的输入通道。

```
// 配置ADC通道对应的GPIO为模拟输入
g1.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
g1.GPIO_Mode = GPIO_Mode_AIN;
GPIO_Init(GPIOC, &g1);
```

(2) 定时器配置：

配置定时器3，使其每20ms/128产生一次触发信号。

定时器的时钟频率为 72MHz，预分频器设置为 72，计数器周期设置为

$0.02 \times 72 \times 10^6 / 128 = 11250$ ，实现20ms/128的定时。

```
// 配置定时器
void TIM3_AD_Init(void)
{
    TIM_TimeBaseInitTypeDef t3;
    // 使能定时器时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);

    // 配置定时器
    t3.TIM_Period = 11250 - 1; // 128*50Hz
    t3.TIM_Prescaler = 1-1; // 72MHz计数频率
    t3.TIM_ClockDivision = TIM_CKD_DIV1;
    t3.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM3, &t3);
    // 配置定时器触发ADC
    TIM_SelectOutputTrigger(TIM3, TIM_TRGOSource_Update);
    TIM_Cmd(TIM3, ENABLE);
}
```

(3) ADC 配置

- 使能GPIO和ADC时钟
- 配置GPIO引脚
- 配置为规则同步模式，启用扫描模式
- 转换由定时器的触发输出（TRGO）启动
- 配置两个通道（通道0和通道1）
- 配置ADC通道及采样时间
- 激活外部触发、启动ADC、使能ADC的DMA请求。
- 校准ADC

ADC初始化代码：

```
// 初始化ADC
void myADC_Init(void)
{
    GPIO_InitTypeDef g1;
    ADC_InitTypeDef adc1;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1 | RCC_APB2Periph_ADC2 |
        RCC_APB2Periph_GPIOC, ENABLE); // 使能ADC和GPIO时钟
    RCC_ADCCLKConfig(RCC_PCLK2_Div8); // 设置ADC分频因子8，72M/8=9MHz，最大不能超过
    14MHz

    // 配置ADC通道对应的GPIO为模拟输入
    g1.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
    g1.GPIO_Mode = GPIO_Mode_AIN;
    GPIO_Init(GPIOC, &g1);
    // 配置ADC1
    adc1.ADC_Mode = ADC_Mode_Regular; // 规则同步模式
    adc1.ADC_ScanConvMode = ENABLE; // 扫描模式
    adc1.ADC_ContinuousConvMode = DISABLE; // 非连续转换
    adc1.ADC_ExternalTrigConv = ADC_ExternalTrigConv_T3_TRGO; // 定时器3触发
    adc1.ADC_DataAlign = ADC_DataAlign_Right;
    adc1.ADC_NbrOfChannel = 1; // ADC_CHANNEL_COUNT;
```

```

ADC_Init(ADC1, &adc1l);
// 配置ADC2
adc1l.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None; //从ADC设置成软件触
发, 以防止意外的触发
ADC_Init(ADC2, &adc1l);
// 配置ADC通道及采样时间
ADC_RegularChannelConfig(ADC1, ADC_Channel_14, 1, ADC_SampleTime_239Cycles5);
ADC_RegularChannelConfig(ADC2, ADC_Channel_15, 1, ADC_SampleTime_239Cycles5);

ADC_ExternalTrigConvCmd(ADC2, ENABLE); //激活外部触发
ADC_DMAMCmd(ADC2, DISABLE); //从ADC不启动DMA (主ADC启动)
ADC_Cmd(ADC2, ENABLE); // 使能ADC2
ADC_ExternalTrigConvCmd(ADC1, ENABLE); //激活外部触发
ADC_DMAMCmd(ADC1, ENABLE); //主ADC启动DMA
ADC_Cmd(ADC1, ENABLE); // 使能ADC1

ADC_ResetCalibration(ADC1); // 复位校准寄存器
while(ADC_GetResetCalibrationStatus(ADC1)); // 校准ADC
ADC_StartCalibration(ADC1); // 开始校准
while(ADC_GetCalibrationStatus(ADC1)); // 等待校准结束
ADC_ResetCalibration(ADC2);
while(ADC_GetResetCalibrationStatus(ADC2));
ADC_StartCalibration(ADC2);
while(ADC_GetCalibrationStatus(ADC2));
}

```

(4) DMA 配置:

- 将 ADC 的数据寄存器作为外设地址。
- 将adval数组作为内存地址。
- 采用循环模式, 不断传输数据。

ADC数据寄存器内存地址为:

```

/* ADC1 DR register base address */
#define DR_ADDRESS ((uint32_t)0x4001244C)

```

程序如下:

```

void DMA1_1_Init(void)
{
    NVIC_InitTypeDef n1;
    DMA_InitTypeDef dma1;
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1, ENABLE); // 使能DMA时钟
    // 配置DMA
    dma1.DMA_PeripheralBaseAddr = (uint32_t)DR_ADDRESS; //ADC数据寄存器地址&ADC1-
    >DR
    dma1.DMA_MemoryBaseAddr = (uint32_t)adval; //内存地址
    dma1.DMA_DIR = DMA_DIR_PeripheralSRC;
    dma1.DMA_BufferSize = ADC_BUFF_SIZE;
    dma1.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
    dma1.DMA_MemoryInc = DMA_MemoryInc_Enable;
    dma1.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Word; //高2字节是ADC2结
    果, 低2字节是ADC1结果
    dma1.DMA_MemoryDataSize = DMA_MemoryDataSize_Word;
    dma1.DMA_Mode = DMA_Mode_Circular;
}

```

```

    dmai.DMA_Priority = DMA_Priority_High;
    dmai.DMA_M2M = DMA_M2M_Disable;
    DMA_Init(DMA1_Channel1, &dmai);
    n1.NVIC_IRQChannel = DMA1_Channel1_IRQn; // DMA2通道1中断
    n1.NVIC_IRQChannelPreemptionPriority = 0;
    n1.NVIC_IRQChannelSubPriority = 1;
    n1.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&n1); //根据指定的参数初始化VIC寄存器
    DMA_Cmd(DMA1_Channel1, ENABLE);
    DMA_ITConfig(DMA1_Channel1, DMA_IT_TC | DMA_IT_TE, ENABLE); //DMA_IT_HT 使
    能DMA通道1传输完成中断
}

```

(5) DMA传输完成中断服务程序

该中断服务程序，完成ADC结果数据从DMA缓冲区到应用缓冲区的传送。首先DMA缓冲区（short adval[2*128]）为128个字（2*128个short型的半字）；其次：DMA缓冲区的DAC1和DAC2交替存放（DAC1是电压通道，DAC2是电流通道）；最后，应用缓冲区（short adv[2][4*128]）的adv[0]存放4个周期电压数据，adv[1]存放4个周期电流数据。那么传输程序基本内容如下：

```

void DMA1_Channel1_IRQHandler(void)
{
    short i;
    if(DMA_GetFlagStatus(DMA1_FLAG_TE1))
        DMA_ClearFlag(DMA1_FLAG_TE1); // 清除出错完成标志
    if(DMA_GetFlagStatus(DMA1_FLAG_TC1)) // 检查通道1传输完成标志
    {
        DMA_ClearFlag(DMA1_FLAG_TC1); // 清除传输完成标志
        // 以下处理传输完成的逻辑
        ps=adval; //+256
        p0d=&adv[0][wavecn*128];
        p1d=&adv[1][wavecn*128];
        for(i=0; i<128; i++)
        {
            *p0d++=*ps++;
            *p1d++=4095-*ps++;
        }
        wavecn++;
        if(wavecn>=4) wavecn=0;
        tu=wavecn+1;
    }
}

```

第8章 交流电参量计算方法

本章学习如何把ADC采样转换得到的离散数据经计算得出交流电量参数的原理和方法，常见的有直接计算、DFT等方法。其中DFT方法能全面计算电能各参数，相比直接计算有更好的稳定性和精度。

8.1 交流电参量的直接计算方法

(1) 电压有效值 (RMS)

电压有效值 (Root Mean Square) 是衡量交流电压大小的一个重要参数，计算公式为：

$$V_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} V_i^2} \quad (8-1)$$

其中， V_i 是第 i 个采样点的电压值， N 是一个周期内的采样点数。对于交流电压满足如下公式：

$$\sum_{i=0}^{N-1} V_i = 0 \quad (8-2)$$

(2) 电流有效值 (RMS)

电流有效值的计算方法与电压有效值类似：

$$I_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} I_i^2} \quad (8-3)$$

其中， I_i 是第 i 个采样点的电流值。对于交流电流满足如下公式：

$$\sum_{i=0}^{N-1} I_i = 0 \quad (8-4)$$

(3) 有功功率 (P)

有功功率是电压和电流瞬时值乘积的平均值，计算公式为：

$$P = \frac{1}{N} \sum_{i=0}^{N-1} V_i \times I_i \quad (8-5)$$

(4) 无功功率 (Q)

无功功率可以通过以下公式计算：

$$Q = \sqrt{S^2 - P^2} \quad (8-6)$$

其中， θ_i 是第 i 个采样点的相位差。在实际应用中，如果相位差未知，可以通过电压和电流的离散傅里叶变换 (DFT) 来计算。

(5) 视在功率 (S)

视在功率可以通过以下公式计算：

$$S = V_{\text{RMS}} \times I_{\text{RMS}} \quad (8-7)$$

(6) 功率因数 (PF)

功率因数是有功功率与视在功率的比值，计算公式为：

$$\text{PF} = \frac{P}{S} \quad (8-8)$$

其中有功功率、无功功率、功率因数的计算是假设电压电流间为同时采样，无功功率 Q 无法计算是否超前滞后，因此上述计算为理想状态算法计算。对于包含多次谐波无法计算，并且基波分量计算误差也较大。

因此，在工业应用中常使用DFT或者FFT算法。

8.2 交流电参量的DFT计算方法

(1) DFT基础公式

对于长度为N的离散序列 $x[n]$ ，其DFT变换 $X[k]$ 定义为：

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi}{N}kn} \quad (k = 0, 1, \dots, N-1) \quad (8-9)$$

展开为实部和虚部如下：

$$X[k] = \underbrace{\sum_{n=0}^{N-1} x[n] \cos\left(\frac{2\pi kn}{N}\right)}_{\text{实部}} - j \underbrace{\sum_{n=0}^{N-1} x[n] \sin\left(\frac{2\pi kn}{N}\right)}_{\text{虚部}} \quad (8-10)$$

(2) 电参量计算中的关键公式

① 频率索引k的取值

在计算离散傅里叶变换（DFT）时，频率索引 k 的值是根据DFT的定义和采样频率来确定的。DFT的公式为：

$$V[k] = \sum_{n=0}^{N-1} v[n] e^{-i2\pi kn/N} \quad (8-11)$$

其中N是样本数， $v[n]$ 是时间域的样本， $V[k]$ 是频率域的样本， k 是频率索引。

频率索引 k 的值从0到N-1。对于128点的DFT， k 的值从0到127。每个 k 值对应一个频率，这个频率由以下公式给出：

$$f_k = \frac{k f_s}{N} \quad (8-12)$$

其中 f_s 是采样频率。

因此，对于128点的DFT，频率索引 k 的值从0到127，每个 k 值对应一个频率

$$f_k = \frac{k f_s}{128}。$$

频率索引 $k = 0, 1, 2, \dots, 127$

② 电压/电流基波分量提取

对电压 $v[n]$ 和电流 $i[n]$ 分别计算50Hz（基波）对应的DFT：

$$V_{\text{phasor}} = \frac{2}{N} \sum_{n=0}^{N-1} v[n] \cdot e^{-j\frac{2\pi}{N}k_0 n} \quad (8-13)$$

$$I_{\text{phasor}} = \frac{2}{N} \sum_{n=0}^{N-1} i[n] \cdot e^{-j\frac{2\pi}{N}k_0 n} \quad (8-14)$$

其中 $k_0 = \frac{50 \times N}{\text{SampleRate}}$ 是50Hz对应的频点索引，系数 $\frac{2}{N}$ 用于归一化。

(3) 电压有效值 V_{rms} 计算

$$V_{\text{rms}} = \frac{|V_{\text{phasor}}|}{\sqrt{2}} = \frac{\sqrt{\text{Re}(V)^2 + \text{Im}(V)^2}}{\sqrt{2}} \quad (8-15)$$

(4) 电流有效值 I_{rms} 计算

$$I_{\text{rms}} = \frac{|I_{\text{phasor}}|}{\sqrt{2}} = \frac{\sqrt{\text{Re}(I)^2 + \text{Im}(I)^2}}{\sqrt{2}} \quad (8-16)$$

(5) 有功功率 P 计算

$$P = \frac{1}{2} \text{Re}(V \cdot I^*) = \frac{\text{Re}(V)\text{Re}(I) + \text{Im}(V)\text{Im}(I)}{2} \quad (8-17)$$

(6) 无功功率 Q 计算

$$Q = \frac{1}{2} \text{Im}(V \cdot I^*) = \frac{\text{Im}(V)\text{Re}(I) - \text{Re}(V)\text{Im}(I)}{2} \quad (8-18)$$

(7) 视在功率 S 计算

$$S = V_{\text{rms}} \times I_{\text{rms}} \quad (8-19)$$

(8) 功率因数 PF 计算

$$\text{PF} = \frac{P}{S} = \cos(\theta), \quad \theta = \angle V - \angle I \quad (8-20)$$

同时DFT还能计算谐波，这里不再赘述。总结电参量计算如表8-1所示：

表 8-1 电参量计算表

电参量	DFT公式
电压有效值	$V_{\text{rms}} = \sqrt{\text{Re}(V)^2 + \text{Im}(V)^2} / \sqrt{2}$
有功功率	$P = \frac{1}{2} (\text{Re}(V)\text{Re}(I) + \text{Im}(V)\text{Im}(I))$
无功功率	$Q = \frac{1}{2} (\text{Im}(V)\text{Re}(I) - \text{Re}(V)\text{Im}(I))$
功率因数	$\text{PF} = P/S$

通过DFT算法，不仅能精确计算基波参数，还能分析谐波成分，适用于电能质量监测等高精度场景。

常规计算方法的优点：

- ① 计算量小：
常规计算方法（如直接计算）的计算量较小，适用于实时计算和处理大量数据的应用。
- ② 简单易实现：
常规计算方法通常较为简单，易于实现和调试。
- ③ 无时间延迟：
常规计算方法可以实时计算，没有时间延迟。

常规计算方法的缺点：

- ① 精确性低：
常规计算方法可能无法精确地计算出信号的频率成分，对于含有多个频率成分的复杂信号，计算的准确性可能较低。

② 无法计算相位:

常规计算方法通常无法计算出信号的相位,这对于计算有功功率和无功功率等参数不利。

③ 滤波能力差:

常规计算方法通常无法有效地滤除信号中的噪声和干扰,计算的准确性可能受到噪声的影响。

DFT算法的优点在于:

① 精确性:

DFT可以精确地计算出信号的频率成分,对于含有多个频率成分的复杂信号,DFT能够提供详细的频谱信息。

② 相位信息:

DFT不仅能够计算出信号的幅度,还能计算出信号的相位,这对于计算有功功率和无功功率等参数非常重要。

③ 滤波能力:

DFT可以用于滤除信号中的噪声和干扰,提高信号的信噪比,从而提高计算的准确性。

④ 适用性:

DFT适用于各种类型的信号,包括非正弦波信号,因此在实际应用中具有广泛的适用性。

DFT算法的缺点在于:

① 计算量大:

DFT的计算量较大,对于实时计算和处理大量数据的应用,可能需要较高的计算资源。

② 时间延迟:

DFT需要一个完整的周期来计算,因此在实时应用中可能存在时间延迟。

③ 频谱泄漏:

DFT在计算时可能会出现频谱泄漏现象,即信号的频率成分在频谱中扩散,影响计算的准确性。

附录 A 常用工具使用方法

A-1 焊台使用方法

该部分主要讲解在实验室完成相应实验应该注意的问题，其中包含配套设备的使用方法和使用过程中注意的问题。

(1) 焊锡五步操作法

- 1) 准备焊接：准备焊锡丝和烙铁。
- 2) 加热焊件：烙铁接触焊接点，使焊件均匀受热。
- 3) 熔化焊料：当焊件加热到能熔化焊料温度后将焊丝置于焊点，焊料开始熔化并湿润焊点。
- 4) 移开焊锡：当熔化一定量的焊锡后将焊锡丝移开。
- 5) 移开烙铁：当焊锡完全湿润焊点后移开烙铁。

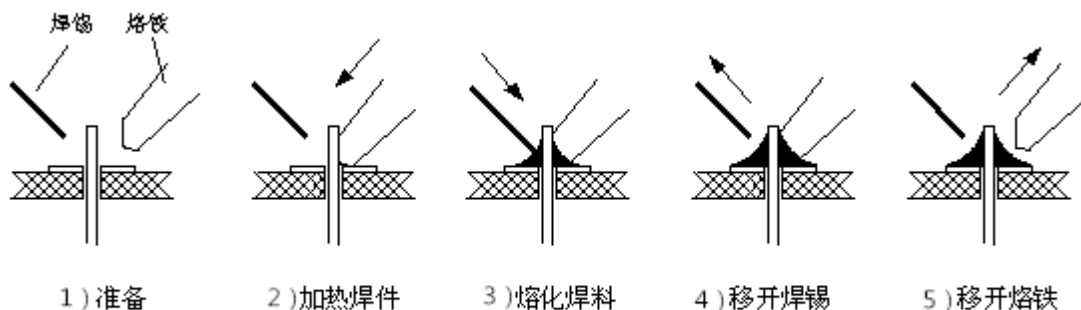


图 A-1 焊锡五步操作法

(2) 焊台使用注意事项

- 1) 切勿触及烙铁头及其附近的金属部分。（加热之后）
- 2) 切忌在易燃物体附近使用烙铁头。
- 3) 休息时或完工后关掉电源。（烙铁闲置超过 20 分钟）
- 4) 更换部件或装置烙铁头时，应关掉电源并待烙铁头冷却后在进行。
- 5) 切勿自动改动电焊台。
- 6) 切勿将烙铁手柄敲击工作台清理焊剂，此举可能严重振损烙铁手柄组件及烙铁头。
- 7) 切勿使用烙铁头进行焊接以外的工作。
- 8) 切勿弄湿焊台，手湿时不能使用和拆开焊台，也不能拉扯电源线。

(3) 焊接注意事项

- 1) 烙铁温度要适宜

可用烙铁头放到松香上去检验，一般以松香熔化较快又不冒烟的温度为适宜。

- 2) 焊接时间要适当

从加热焊料到焊料熔化并流满焊接点，一般应在三秒钟之内完成，但焊接时间也不易过短，时间过短焊料不能充分融化，易造成虚焊。

- 3) 焊料与焊剂的使用要适当

若使用的焊料过多，则多余的会流入管座的底部，降低管脚之间的绝缘性；若使用的助焊剂过多，则易在管脚周围形成绝缘层，造成管脚和底座之间的接触不良。反之，焊料和焊

剂过少易造成虚焊。

4) 焊接过程中不要触动焊接点

在焊接点上的焊料未完全冷却凝固时，不应移动被焊元件及导线，否则焊点易变形，也可能形成虚焊现象。焊接过程中也要注意不要烫伤周围的元器件及导线。

(4) 焊台保养方法

1) 温度过高会降低烙铁头的使用寿命，因此选择尽可能低的温度。

2) 长时间使用烙铁时，应每周一次拆下烙铁头清除氧化物，防止烙铁头受损而减低温度。不使用焊台时，应尽量关闭电源来延长使用寿命。不可让烙铁头长时间处在高温状态，会使烙铁头上的焊剂转化为氧化物，致使烙铁头导热功能大为减退。

3) 每次使用后浸润新鲜焊锡，可以阻止烙铁头的氧化而延长使用寿命。

4) 应定期使用清洁海绵清理烙铁头。焊接后，烙铁头的残余焊剂所衍生的氧化物和碳化物会损坏烙铁头，造成焊接误差，或使烙铁头导热功能减退。

5) 在能够工作的情况下尽量使用较低的温度，低温可以减少烙铁头的氧化，也容易焊接元器件。

6) 只有在必须时才使用细的烙铁头，细小烙铁头的镀层没有粗顿的烙铁头耐用。

7) 不要使用烙铁头作为探测工具，烙铁头弯曲会使镀层破裂，缩短使用寿命。

8) 用有较少活性的松香焊剂，因为含量高的活性松香会加速烙铁头镀层的腐蚀。

9) 不要对烙铁头施加重压，因为较大的压力不等于传热快，为提高热传输，必须使焊锡熔化，使烙铁头与焊点之间形成一个热传递的焊锡桥梁。

A-2 镊子、螺丝刀等工具使用方法

(1) 镊子的使用方法

1) 在焊接过程中，镊子是配合使用不可缺少的工具，特别是在焊接小零件时，用手扶拿会烫手，既不方便，有时还容易引起短路。有了镊子，可用它接引线、送管脚，灵活得很

2) 镊子的种类一般使用的镊子有两种：一种是用铝合金制成的尖头镊子，它不易磁化，可用来夹持怕磁化的小元器件；另一种是不锈钢制成的平头镊子，它的硬度较大，除了可用来夹持元器件管脚外，还可以帮助加工元器件引脚，做简单的成形工作。使用镊子进行协助焊接时，还有助于电极的散热，从而起到保护元器件的作用。

3) 镊子的选择选用的镊子，要求其弹性要适中。镊子口应对齐、严实，但不发软。比较适用的规格是 100~150mm 的医用镊子或修钟表的小镊子。

(2) 螺丝刀的使用方法

1) 使用时，不可用螺丝刀当撬棒或凿子使用。

2) 在使用前应先擦净螺丝刀柄和口端的油污，以免工作时滑脱而发生意外，使用后也要擦拭干净。

3) 正确的方法是以右手握持螺丝刀，手心抵住柄端，让螺丝刀口端与螺栓或螺钉槽口处于垂直吻合状态。

4) 当开始拧松或最后拧紧时，应用力将螺丝刀压紧后再用手腕力扭转螺丝刀；当螺栓松动后，即可使手心轻压螺丝刀柄，用拇指、中指和食指快速转动螺丝刀。

5) 选用的螺丝刀口端应与螺栓或螺钉上的槽口相吻合。如口端太薄易折断，太厚则不能完全嵌入槽内，易使刀口或螺栓槽口损坏。

6) 螺丝刀是一种用于拧紧或拧松带有槽口的螺栓或螺钉的手用工具。有一字形螺丝刀、十字形螺丝刀、偏置螺丝刀等几种。

附录 B 常见元器件识别

B-1 对于插针式的电阻

它的外部被涂上了不一样的颜色的色环，用来区分电阻的阻值。保证在安装电阻时不管从什么方向来安装，都可以清楚的读出它的阻值。色环电阻的基本单位有：欧姆（Ω）、千欧（KΩ）、兆欧（MΩ）。1 兆欧（MΩ）=1000 千欧（KΩ）=1000000 欧（Ω）。平常使用的色环电阻可以分为四环和五环，通常用四环。其中四环电阻前二环为数字，第三环表示阻值倍乘的数，最后一环为误差；五环电阻前三环为数字，第四环表示阻值倍乘的数，最后一环为误差。误差通常也是金、银和棕三种颜色，金的误差为 5%，银的误差为 10%，棕色的误差为 1%，无色的误差为 20%，另外偶尔还有以绿色代笔误差的，绿色的误差为 0.5%。



图 B-1 插针电阻

有一个小口诀：棕一红二橙是三，四黄五绿六为蓝，七紫八灰九对白，黑是零，金五银十表误差。

表 B-1 口诀对照表

黑	棕	红	橙	黄	绿	蓝	紫	灰	白	金	银
0	1	2	3	4	5	6	7	8	9	5%	10%

表 B-2 色环对照表

色环	第一环	第二环	第三环	第四环
			（乘法）	（误差环）
黑	0	0	1	+/-1%
棕	1	1	10	+/-2%
红	2	2	100	
橙	3	3	1000	
黄	4	4	10000	+/-0.5%
绿	5	5	100000	+/-0.2%
蓝	6	6	1000000	+/-0.1%
紫	7	7	10000000	
灰	8	8	100000000	
白	9	9	1000000000	+5~-20%
银				5%
金				10%
无色环				20%

四色环电阻：前二条色环用来表示阻值，第三环表示数字后面添加“0”的个数，这三条色环是相隔比较靠近的，而第四环相对距离较大，这是表示误差的。如果电阻色环不好分辨

出那个是第一个色环，最简单的方法就是“第四环”不是金色就是银色，而其它颜色会出现的银少（只对四环电阻有用，五环电阻不适用）。

例如：

（1）红，黄，棕，金 $24 \times 10 = 240$ 欧金误差为 5%

（2）绿，红，黄，银 $52 \times 10000 = 520K$ 欧误差为 10%

五色环电阻：第一道色环表示阻值的第一位数字；第二道色环表示阻值的第二位数字；第三道色环表示阻值的第三位数字，第四道色环表示阻值的倍乘数；第五道色环表示误差范围。一般五环电阻是相对较精密的电阻。

例如：

（1）红，红，黑，黑，棕 $220 \times 1 = 220$ 欧误差为 1%；

（2）紫，红，棕，红，绿 $521 \times 100 = 52.1K$ 欧误差为 0.5%。

B-2 贴片电阻

贴片电阻很小，通常有一面是白色的还有一面是黑色的。在黑色的那一面通常有用白色的字体标明了电阻的阻值的大小。贴片电阻表面数字一般为 3 位或者 4 位，4 位数字表示的是精密电阻。当采用 3 位数字时，用 ABC 表示，其大小为 $AB \times 10^C$ ，例如 103，其阻值大小为 $10 \times 10^3 = 10k\Omega$ ；当采用 4 位数字时，我们用 ABCD 表示，其大小为 $ABC \times 10^D$ ，例如 5110，其阻值大小为 $511 \times 100 = 511\Omega$ 。但有一个特殊情况，例如 5R6，中间的 R 表示小数点，即 5.6Ω 。

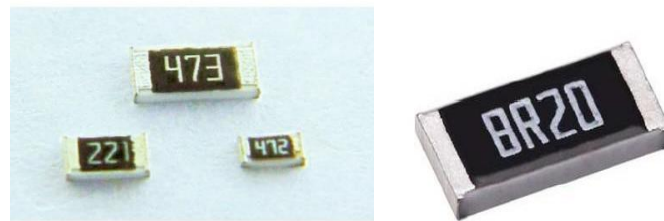


图 B-2 贴片电阻

B-3 插针式电容

插针式的电容按照极型划分可划分为有极性电容和无极性电容。我们平时用的比较多的有极性的电容通常会在电容上标明电容的正负极，电容的大小以及所能承受的电压或者电流的大小。有极性电容有灰色条纹的那一边是负极，同时也可以通过电容的引脚的长短来判断电容的极性，通常长的引脚的那一边是正极。



图 B-3 插针电阻

无极性电容的大小会标识在电容上的，电容的值的大小一般标记为三位，我们用 ABC 表示，其大小为 $AB \times 10^C$ ，例如 103，其阻值大小为 $10 \times 10^3 = 10000pf$ ；若电容的标没有给出符号，就被认定为使用的单位为 pf。各个单位之间的转化为： $1F = 10^6\mu F = 10^9nF = 10^{12}pF$ 。

附录 C 万用表使用方法

C.1 万用表的外表结构

如图 C-1 所示，为万用表的外表结构图。

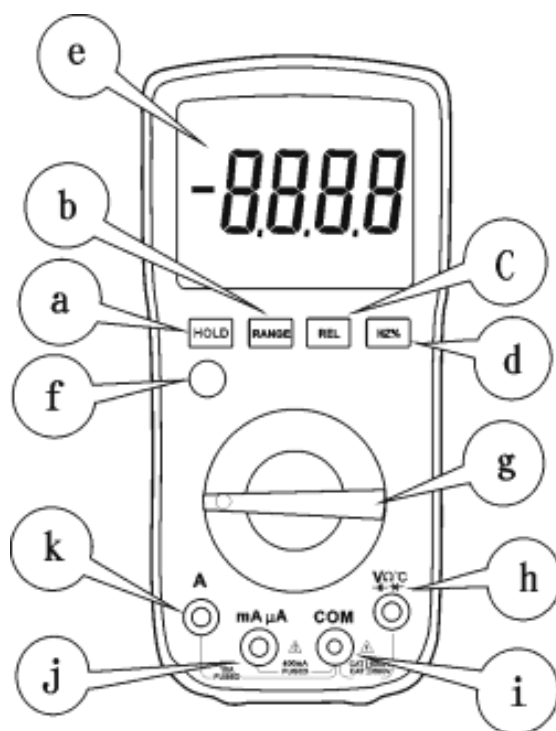


图 C-1 万用表结构图

万用表介绍：

- (1) 数据保持（部分表有这个功能）
- (2) 量程选择键（部分表有这个功能）
- (3) 相对值测量键（部分表有这个功能）
- (4) 频率测量/占空比（部分表有这个功能）
- (5) LCD 显示屏
- (6) 功能选择切换键（部分表有这个功能）
- (7) 功能转换开关（有些表的功能转换开关带有量程的选择）
- (8) 输入插口
- (9) COM 输入插口
- (10) mA、 μ A 电流输入插口
- (11) A 电流输入插口

C-2 万用表显示屏

如图 C-2 所示，为万用表显示屏的图。

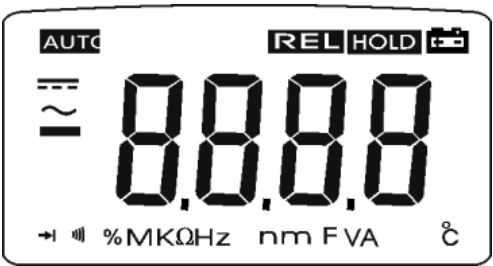


图 C-2 显示屏图

如表 C-1 所示，为显示屏上可能出现的一些符号的说明。

表 C-1 显示屏图标说明

符号	说明	符号	说明
AUTO	自动量程启用	~	AC 交流电
HOLD	数据保持	低电压电池图标	电池低电压
REL	相对测量值	M/KΩ	电阻
二极管符号	二极管测试	Hz	频率
蜂鸣器符号	电路通断测试	— — —	DC 直流电
%	占空比	uF、nF	电容
mA、A	电流	Mv、V	电压

C-3 端子

数字万用表的下端会有四个插孔，我们称之为端子，黑色的表笔始终插在公共输入端，红色的表笔根据测量的需求，需要插入不同的端子内。图c-3 就是一个万用表的端子示意图。

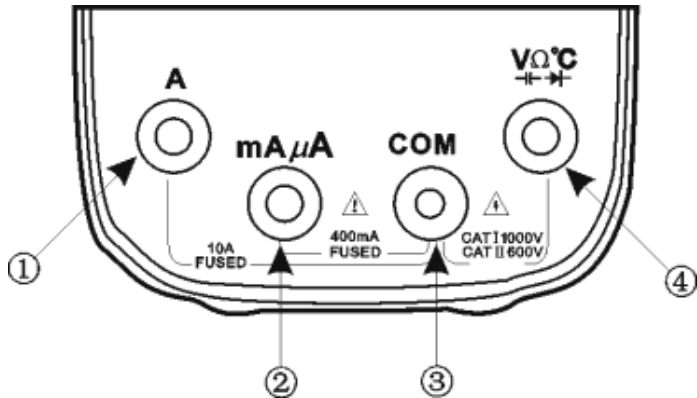


图 C-3 万用表端子图表

C-2 万用表端子说明表

部件	说明
1	20A 电流输入端
2	200mA 电流输入端
3	公共输入端
4	电压、电阻、通断性、二极管、电容频率的输入端

C-4 测量

(1) 开机

有些万用表的开关键就是一个 POWER 键，有些万用表的开关键有的是 ON/OFF 表示的。开启开关键后，万用表就进入了工作状态。

(2) 测量的步骤

- 1) 根据测量的需求，将表笔插入对应的端子中。
- 2) 估计出待测量的取值范围。
- 3) 转动 g，将 g 调到需要测量的量的对应的符号和合适的量程下（有些表的量程选择和功能选择键不在一起）。
- 4) 将两支表笔分别放在待测点处进行测量。

(3) 注意事项

- 1) 如果无法预先估计被测电压或电流的大小，则应先拨至最高量程挡测量一次，再视情况逐渐把量程减小到合适位置。测量完毕，应将量程开关拨到最高电压挡，并关闭电源。
- 2) 满量程时，仪表仅在最高位显示数字“1”，其它位均消失，这时应选择更高的量程。
- 3) 测量电压时，应将数字万用表与被测电路并联。测电流时应与被测电路串联，测直流量时不必考虑正、负极性。若正负极接反了，则在显示屏上会显示出一个负值，此时只需要将红黑表笔互换一下位置即可。
- 4) 当误用交流电压挡去测量直流电压，或者误用直流电压挡去测量交流电压时，显示屏将显示“000”，或低位上的数字出现跳动。
- 5) 禁止在测量高电压（220V 以上）或大电流（0.5A 以上）时换量程，以防止产生电弧，烧毁开关触点。
- 6) 当显示“—”、“BATT”或“LOWBAT”时，表示电池电压低于工作电压。

C-5 常用测量的方法介绍

(1) 电流(电压)的测量

1) 电流（电压）的测量分为交流电的测量和直流电的测量，测量的步骤大致相同。将黑色表笔插入公共输入端，根据待测电流的大小将红色表笔插入 20A 电流输入端或者是 200mA 电流输入端（电压的测量需要将红色的表笔插入端子 4，即带有  符号的端子）。

2) 调节功能转换开关，将开关调  （交流）或  （直流）电流下的合适的量程下。

3) 将表笔串联到电路中进行电流的测量。

4) 若显示屏上只显示了数字 1，则表示量程选择过小，需要将量程调大，此时应加大量程，直到测出数据为止。若显示屏上显示的是“000”，则是万用表上选择的电流（电压）测量的类型出错了。

(2) 电阻的测量

将黑色表笔插入公共的输入端，将红色的表笔插入端子 4，即带有  符号的端子。调节功能转换开关，将开关调到 Ω 挡下合适的量程下。

3) 将红黑表笔接在电阻的两端。（电阻的测量最好不要在电路中进行，因为电路中的电阻可能与其他的电阻并联，这样测出来的值不准确）

4) 若显示屏上只显示了数字 1，则表示量程选择过小，需要将量程调大。此时应加大

量程，直到测出数据为止。

(3) 电容的测量

将黑色表笔插入公共的输入端，将红色的表笔插入端子 4，即带有  符号的端子。调节功能转换开关，将开关调到 F 挡下合适的量程下。

3) 将红黑表笔接在电容的两端。(电容的测量最好不要在电路中进行，因为电路中的电容可能与其他电容相连接，这样测出来的值不准确)。

4) 若显示屏上只显示了数字 1，则表示量程选择过小，需要将量程调大。此时应加大量程，直到测出数据为止。

(4) 二极管的测量

1) 将黑色的表笔插入公共的输入端，将红色的表笔插入端子 4，即带  符号的端子。

2) 调节功能转换开关，将开关调到带  符号下。

3) 将红色表笔探针接对待测的二极管的正极而黑色表笔探针接到负极；此时显示屏上的读数就是被测二极管的正向偏压值；若测试表笔的极性与二极管的极性反接，则显示屏读数会是 [OL](或者显示一个非常大的数值)，以此可以用来区分二极管的正极和负极。

注:若为发光二极管，将红色表笔探针接对待测的发光二极管的正极而黑色表笔探针接到负极，此时发光二极管会亮。若接反了，则不会亮，显示屏上会显示一个非常大的数值。

(5) 通断性测试

将黑色表笔插入公共的输入端，将红色表笔插入端子 4，即带  符号的端子。调节功

能转换开关，将开关调到带  符号下。

3) 当被测电路的电阻不超过 80 欧姆时，蜂鸣器会发出连续音，表明被测电路已经短路。若蜂鸣器不响则表示被测电路是断路。



C-4 数字万用表的使用